

The preparation guide. September 2026 to April 2027.

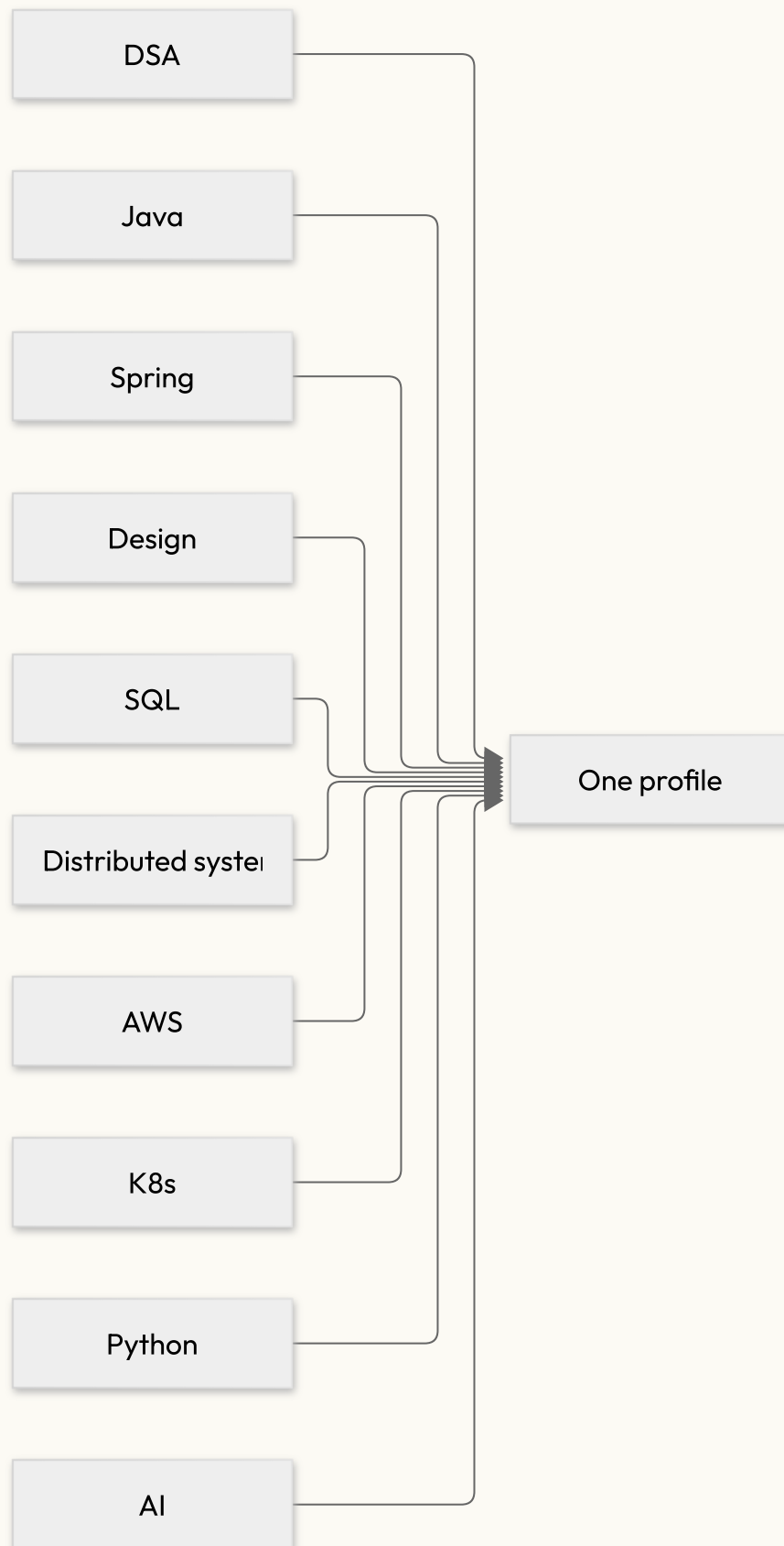
This is the study text, not only the calendar. Each skill from the target stack has a lesson: the idea, the Java or the design, the mistake, and the question. Dell SDE II, Java and Spring in production.

Every skill in the target stack, with a slot

The job posts share one stack. Each row below has a weight, a phase, and a lesson in this guide. Nothing in the stack is a separate career.

1. Very high skills are the interview
2. High skills are the project
3. Go and certificates support
4. One profile receives all of it

Figure



DSA – very high

Phase 1 through January. About 40 patterns and 120 to 150 problems, five passes. The DSA section of this guide is the set: recognition, a Java template, and the representative problems. January stops at basic DP. MCM, trie, KMP, and Kosaraju wait.

Java – very high

You already write it. The gap is interview depth: HashMap, collections choice, equals and hashCode, streams, generics, immutability, executors, CompletableFuture, volatile, locks, and a plain JVM story. Lessons are in the Java section. Say them from the Dell async work, with a failure mode.

Spring Boot – very high

IoC and the proxy, transactions and propagation, JPA and N+1, REST with validation and a single error shape, Kafka with crash and idempotency, outbox, Redis, security, resilience, tests, actuator. The 45-minute block in phase 2 is this list, one topic a day.

System design HLD – very high

Same checklist every time: requirements, scale, API, data model, architecture, database, cache, queue, consistency, failure, observability, security, trade-off. Phase 2 owns URL shortener, rate limiter, notification, file storage. Phase 4 owns five to eight full designs, including cache, chat, feed, search, payments, scheduler, logs, and the RAG agent.

LLD, OOP, and design patterns – very high

A 45-minute class design shows up next to HLD in SDE II loops. You need SOLID in one sentence each, and Strategy, Factory, Observer, Decorator, Adapter as tools you pick, not a poster. The LLD lesson is a rate limiter drawn as classes.

Distributed systems – very high

This is the language of your Kafka and storage work: replication, partitions, timeouts, retries, idempotency, ordering, and what a crash repeats. The distributed-systems lesson is the theory. The Kafka lessons are the concrete system.

SQL and database internals – very high

Indexes, EXPLAIN, isolation, MVCC, and deadlocks. JPA is how you meet the database. The SQL lesson is what the database is doing under the repository.

AWS – very high

Solutions Architect Associate level, not Cloud Practitioner. IAM and VPC first, then EC2, ALB, autoscaling, S3, RDS, DynamoDB, ElastiCache, SQS, SNS, Lambda, ECS, EKS, CloudWatch, CloudTrail, API Gateway, ECR, Secrets Manager, KMS, Route 53. Phase 3. Then deploy the project: EKS, S3, RDS, ElastiCache, SQS, CloudWatch, IAM.

Docker and Kubernetes – high

Order is fixed: image on the laptop, Deployment, Service, probes, HPA, ConfigMap, Secret, RBAC, volumes, Helm, then CKAD. CKAD does not replace that order. You already touch OpenShift. The public proof is the same objects on the flagship.

Kafka – high

Why a log instead of REST, delivery, consumer crash, duplicates, ordering, retry, DLQ, outbox. Phase 2, and the project publishes object events through the outbox by 25 November.

Redis – high

Cache, rate-limit counter, and short-lived agent session. Not the source of truth. Eviction and a stampede belong in the same answer.

Linux and networking – high

Enough to debug a pod and an API: process, port, DNS, TCP timeout versus connection refused, HTTP, and the commands you actually run. One lesson, used every time a deploy fails.

Observability and reliability – high

Logs, metrics, traces, a correlation id, and one alert that a human can act on. The diagnostics agent is this skill with a model in front of it.

Python – secondary, start now

Phase 1 is syntax, collections, functions, OOP, venv, pip, typing, requests. Then pytest, httpx, asyncio, FastAPI, Pydantic, SQLAlchemy, Docker. You are building the agent service, not collecting a Python certificate.

Go – tertiary

Apple storage and Cisco distributed roles mention Go beside Java. Read it. Write a small main after the offer. Do not pause Java to become a beginner in three languages.

GenAI, RAG, agents, MCP, evals – the differentiator

Phase 3: tokens, embeddings, chunking, retrieval, rerank, citations, tool calling, FastAPI. Phase 4: the four agents, human approval, traces, and an eval score in the README. MCP is how a tool is exposed. It is not a personality.

Terraform, certificates, LinkedIn, BITS

Terraform is one module that describes the network you already drew. Certificates are SAA, then CKAD, then a GenAI professional exam when the project is real. AI-901 and AWS AI Practitioner are optional. LinkedIn is one technical post a week and a short list of people. BITS M.Tech AI and ML is a parallel degree if the batch, fee, and hours fit. It does not sit in the January plan.

Do it in this order

1. Very high, January: DSA, Java, Spring, HLD, LLD, distributed systems, SQL, AWS, Kafka, GenAI on the project.
2. High: Docker, Kubernetes, Redis, Linux and networking, observability. They are practiced by deploying the same project.
3. Secondary: Python from phase 1. Go is read-only until after the offer. Terraform is one module, not a course.
4. Supporting: SAA after you can draw a VPC, CKAD after you can debug a pod, LinkedIn once a week. BITS is parallel and does not replace this list.

Worked example

DSA	20%	weekday 75 min, Patterns tab
Java + Spring	20%	weekday 45 min
System design	20%	Saturday 2 hours
Cloud + K8s + Docker	15%	weekday 30 min, then the project
Python + GenAI	15%	same 30 min block, phases 1 then 3
Project	5%	threaded: Kafka, Redis, K8s, AWS land here
Certificates	5%	SAA, later CKAD, later GenAI professional

The usual miss

Studying the list top to bottom as twelve courses.

They will ask

Which three skills are you weak at, and which phase owns them?

Before you close the laptop

Write the three weak skills on a card. They are the only 30-minute blocks this week.

One profile, not ten careers

You are a Dell SDE II with 3+ years, Java, Spring Boot, microservices, Kafka, async work, Kubernetes/OpenShift, Docker, and already-shipped agent and RAG work. The switch is interview depth on that base, plus Python and AWS practiced on one project. You are not restarting as an AI fresher.

1. Keep the Dell and TCS backend story
2. Add Python only where the agent project needs it
3. Treat Go as reading ability
4. Apply as a backend engineer who ships AI

Figure



Do it in this order

1. Write one sentence you can say in an interview: Senior backend engineer, Java and Spring, distributed systems, with production-style RAG and agents.
2. Put FDE in a second folder. Do not make it the January application identity.
3. Do not describe Dell proprietary data or internal source in a public repo. Rebuild the shape with synthetic metrics.

Worked example

Primary: Java

Secondary: Python

Tertiary: Go, enough to read a service and change a handler

The usual miss

Throwing away three years to become a LangChain beginner.

They will ask

Walk me through the profile you want in April 2027.

Before you close the laptop

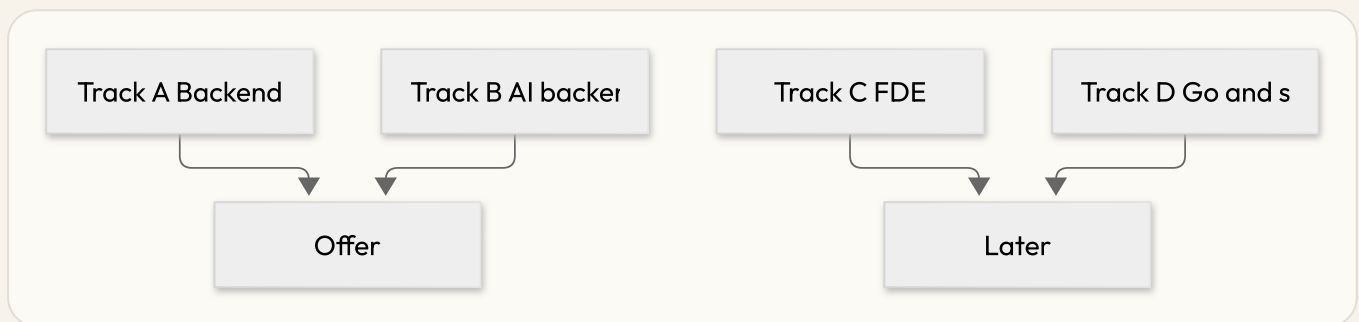
Say the one-sentence profile out loud once. Pin it at the top of the desk.

Four tracks, one weight

Track A is the job you apply for. Track B is the differentiator already started at Dell. Track C is FDE if a posting fits. Track D is Go and storage infrastructure later, which matches object-storage work, not this winter.

1. Backend loop pays the 40-50 target
2. AI work makes the resume specific
3. FDE needs customer discovery
4. Go waits until after the offer

Figure



Do it in this order

1. January applications: backend and AI-platform roles, SDE II / SWE III / senior only where the years match.
2. One FDE application is allowed after the flagship README can explain a safe tool call. It is not the whole pipeline.

Worked example

Atlassian remote-India roles and some India FDE posts exist. US-level pay for India WFH is

The usual miss

Applying only to FDE titles because the tweets are loud.

They will ask

Which track gets 70 percent of this week?

Before you close the laptop

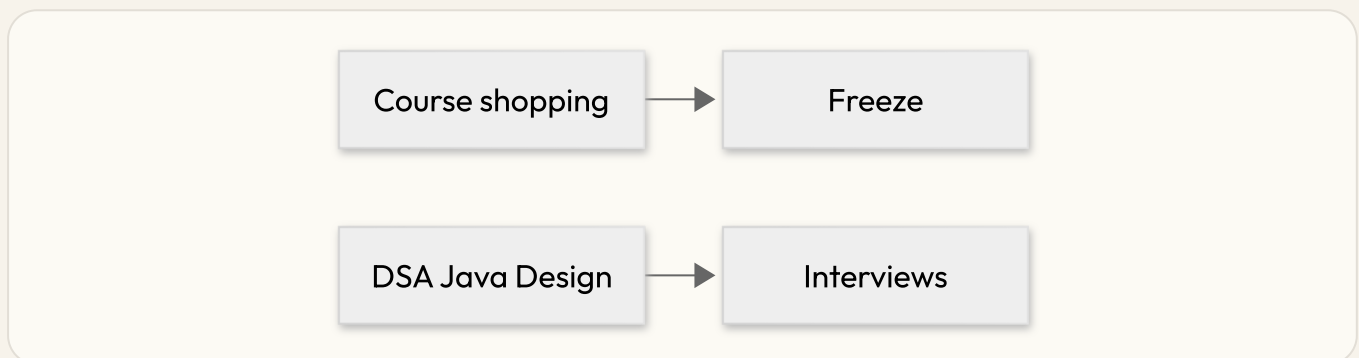
Mark Track A and Track B as the only open tabs this month.

What stays closed until the offer

The failure mode is a stack of courses and no interviews. React, deep learning research, ten frameworks, CKA plus CKAD together, KMP as a lifestyle, and random hards do not move a Java SDE II loop.

1. Close the extra tabs
2. One project
3. Two credentials later
4. January is for applications

Figure



Do it in this order

1. Skip AWS Cloud Practitioner. It is aimed at people new to cloud.
2. Skip AI-900. The current fundamentals exam people mean is AI-901, and even that is optional next to a RAG project.
3. Skip LangChain certificates, Python completion badges, and a Docker beginner badge.

Worked example

Do: AWS Solutions Architect Associate after you can draw a VPC. CKAD after you have deployed

The usual miss

Buying the next Udemy bundle because the last one felt unfinished.

They will ask

Name three things you will not study before January.

Before you close the laptop

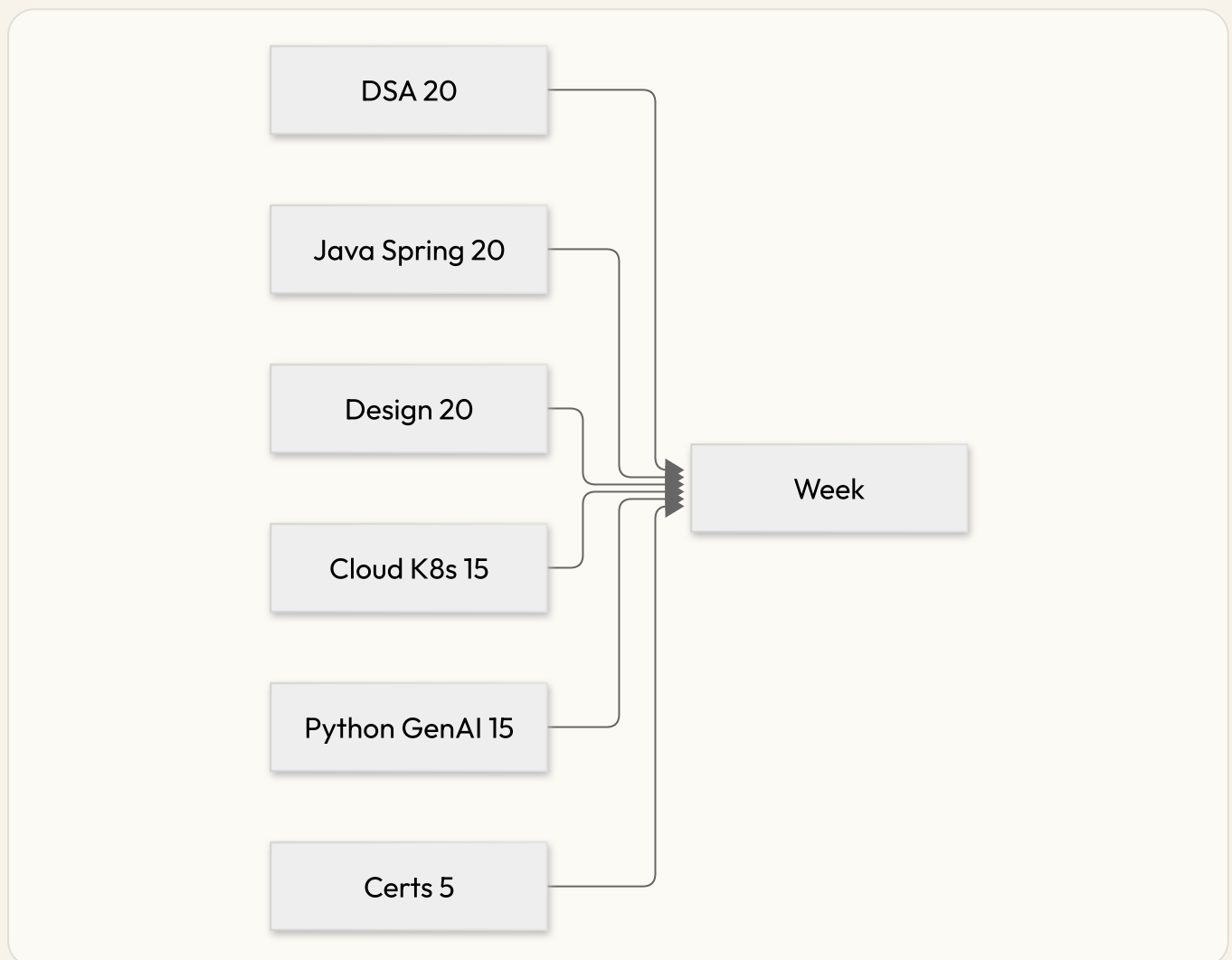
Write the skip list on paper and leave it in the laptop sleeve.

Where the hours go

About 19 to 20 hours a week. DSA, Java/Spring, and system design take 20 percent each. Cloud and Kubernetes 15. Python and GenAI 15. The project is threaded through those hours, not a fifth full-time job. Certificates are 5 percent.

1. 75 minutes DSA on weekdays
2. 45 minutes backend or design
3. 30 minutes Python, AI, or cloud
4. Saturday is the long build

Figure



Do it in this order

1. Weekday total is 2.5 hours, then stop.
2. Saturday: 2 hours DSA, 2 hours design, 2 hours project.
3. Sunday: 1.5 hours retrieval, 2 hours project, 1 hour Java or a mock.

Worked example

A weekday that becomes six hours dies by Wednesday. The cap is the method.

The usual miss

Adding a night session because the morning felt short.

They will ask

What do you stop doing at minute 150 on a Wednesday?

Before you close the laptop

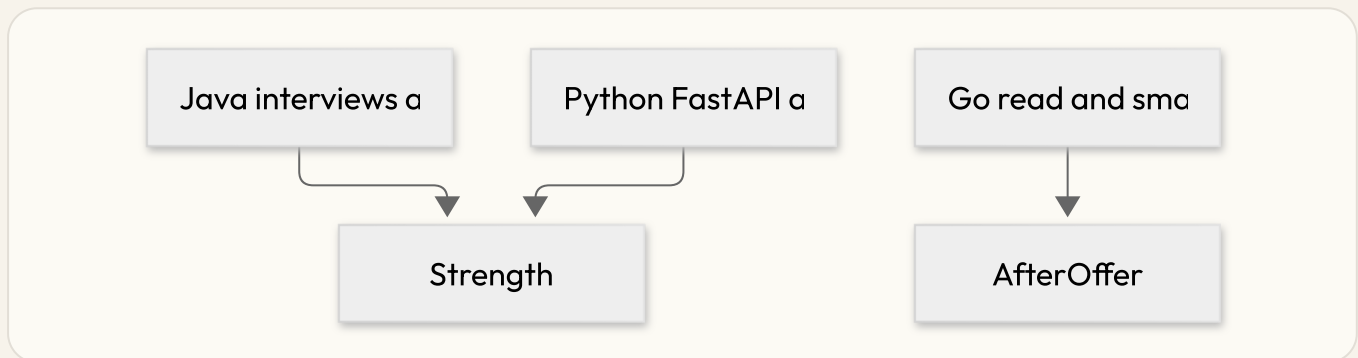
Set a repeating timer for 75, 45, and 30. When it rings, stand up.

Java primary, Python secondary, Go later

Postings are polyglot. Google India lists Java, Go, or Python. Apple Bengaluru pairs Spring with Kubernetes, Kafka, and Go or Python. Salesforce and Cisco do the same. Java is not dying in your target set. You already have the production years.

1. Solve DSA in Java
2. Write the agent API in Python
3. Read one Go file a week after December
4. Do not split practice three ways

Figure



Do it in this order

1. Every blank rewrite this month is Java.
2. Python starts with the types and tests the flagship service needs, not a 40-hour syntax course.
3. Go becomes a weekend reader in 2027: a small HTTP handler, goroutines at a conceptual level, and the storage-service style Apple posts describe.

Worked example

```
// Java stays the interview language
CompletableFuture<Report> report = diagnostics.collect(bucket);
```

The usual miss

Being slightly able to start a main method in three languages and fluent in none.

They will ask

Which language do you code the medium in?

Before you close the laptop

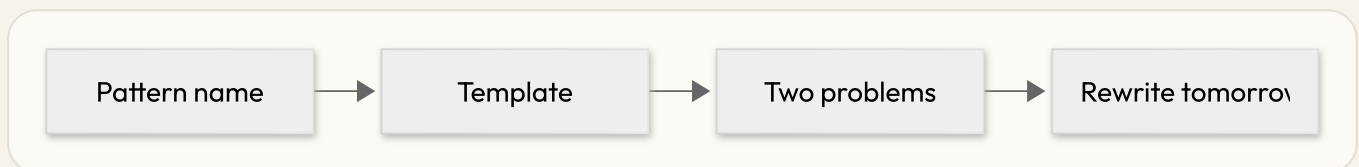
Open one old PR and explain one `CompletableFuture` you wrote, without the IDE.

25 Sep to 25 Oct – retrieval and Python syntax

Phase 1 rebuilds the patterns you forget: arrays, hashing, two pointers, sliding window, binary search, linked lists. Java side is collections, HashMap, equality, streams, and concurrency vocabulary. Python is syntax through requests.

1. Identify the pattern before coding
2. Rewrite yesterday with the file empty
3. One Python function, not a course module
4. Stop at 2.5 hours

Figure



Do it in this order

1. Use the Patterns page. Two or three problems per shape.
2. Sliding window is one machine: expand right, update state, shrink left, record the answer. Fruit baskets and minimum window are variations.
3. HashMap: buckets, equals and hashCode, and what happens on collision. Say it once from memory.

Worked example

A green mark means you rewrote it alone. A red mark comes back. Day 1, 3, 7, 15, 30.

The usual miss

Watching the solution and ticking completed.

They will ask

Is this a new pattern or a variation?

Before you close the laptop

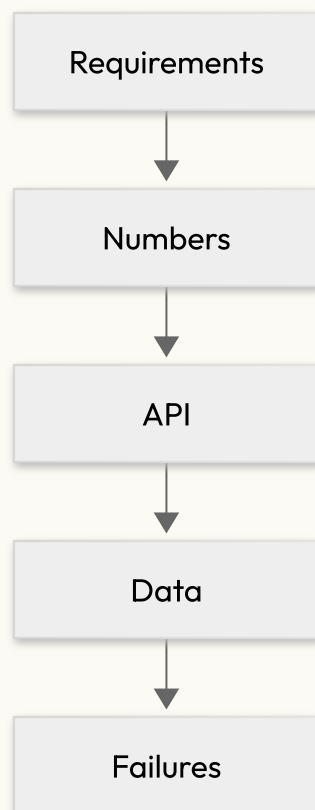
Tonight: Two Sum from a blank file, then say which line updates the map.

26 Oct to 25 Nov – core structures and four designs

Stacks, monotonic stacks, intervals, greedy, trees, BST, heaps. Spring moves from annotations you use to transactions, JPA, Redis, and Kafka failure. Designs: URL shortener, rate limiter, notification service, file storage.

1. One design, 40 minutes, no video first
2. Kafka question is why, not the dependency
3. Tree DFS returns a summary
4. Heap of size K

Figure



Do it in this order

1. Transaction propagation and isolation. Know why a private self-call skips the proxy.
2. N+1: one query becomes one per row. Fix with a fetch join or a batch, and say the cost.
3. Rate limiter: token bucket, where the counters live, what happens when Redis is down.

Worked example

Design out loud before drawing. The picture is the notes, not the thinking.

The usual miss

They will ask

Starting the video at minute zero.

Where does the rate-limit state live with twenty servers?

Before you close the laptop

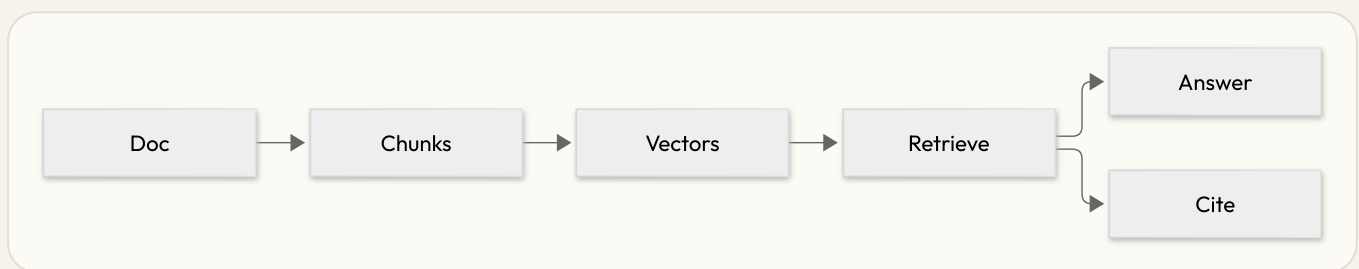
Speak a rate limiter for 10 minutes into a voice note. Listen once.

26 Nov to 20 Dec – graphs, cloud, RAG

Graphs through Dijkstra and basic DP. Cloud is IAM, VPC, a real deploy path, Docker, and Kubernetes objects you can debug. AI is tokens, embeddings, a RAG pipeline, and one tool call.

1. BFS is unweighted distance
2. Dijkstra is non-negative weights
3. VPC before EKS
4. RAG cites or it is a demo

Figure



Do it in this order

1. Basic DP only: climbing stairs, house robber, subset sum, coin change, one grid. Advanced MCM waits.
2. Deploy the order or diagnostics service in Docker. Then the same image behind a Kubernetes Deployment and Service.
3. Python FastAPI endpoint that returns chunks for a question. Keyword overlap is allowed on day one. Embeddings replace it the next week.

Worked example

January interviews do not require Kosaraju, bridges, or a perfect Z-algorithm. Those stay i

The usual miss

Starting EKS before you can explain a subnet.

They will ask

Why is this graph BFS and not Dijkstra?

Before you close the laptop

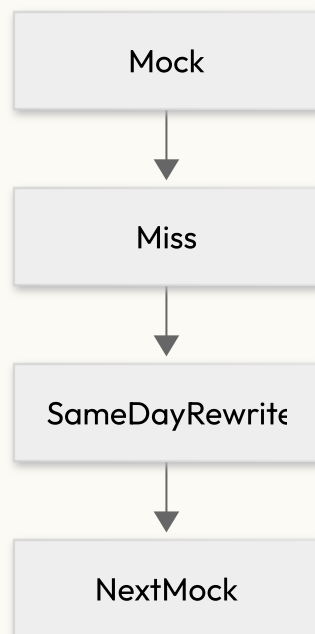
Draw your VPC on paper: two subnets, one load balancer, one database not on the public internet.

21 Dec to 15 Jan – mocks, not new syllabi

Mixed retrieval. Five to eight full designs. The flagship README is the project story. Mocks cover Java, Spring, DSA, LLD, HLD, the project, and a behavioral story with a number.

1. No new pattern family
2. Rewrite the miss the same day
3. Project demo in 8 minutes
4. Resume matches what you can draw

Figure



Do it in this order

1. Resume line: backend, distributed systems, AI, cloud. Each bullet has a number you can defend.
2. Behavioral set: five STAR stories, 90 seconds, including one agent you shipped and one incident.
3. Selective applications start in January. Referrals where you actually know the person.

Worked example

You joined Dell in April 2026. The one-year mark is April 2027. January is for loops that c

The usual miss

Waiting until you feel 100 percent ready.

They will ask

What did yesterday's mock miss, and did you rewrite it?

Before you close the laptop

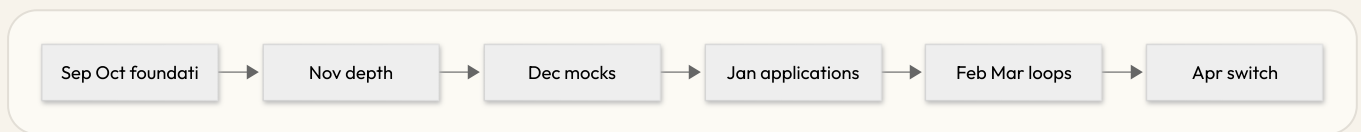
Book one mock with a friend or a recording. Listen for the pattern name arriving late.

January apply, April switch

September through October is foundation. November is depth. December is interview shape. January opens the resume, referrals, and selective applications. February and March are loops. April is the switch window after the Dell year.

1. Eight serious applications, not eighty
2. Referral before a cold form when you can
3. 40-50 is a target, not a promise
4. One year at Dell stays intact

Figure



Do it in this order

1. Target list: Walmart SWE III or senior if the years fit, Apple Java/Spring and storage-infrastructure where Go is a plus not a wall, Microsoft 61/62, Atlassian remote-India backend, Adobe, Salesforce, Intuit, Cisco Java/Go roles.
2. Compensation on the posting is often hidden. One India FDE post has listed about 35-45 LPA base plus bonus. Product-company medians for strong SDE II sit around your target and move. Negotiate with a number, do not assume a US remote salary.

Worked example

Foreign company plus India WFH plus US pay is a special case. Ask which entity employs you

The usual miss

Mass applying in October with no retrieval habit.

They will ask

What is the date you send the first eight applications?

Before you close the laptop

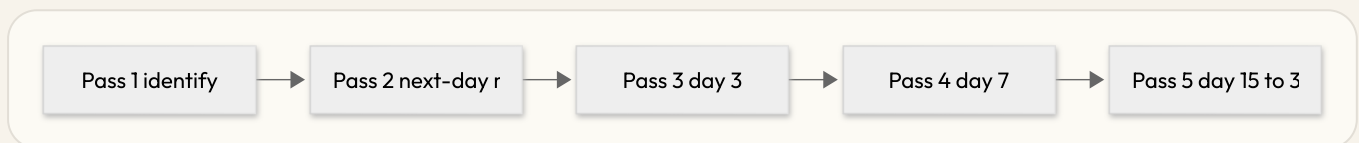
Put 15 January on the calendar as resume freeze, not as a feelings check.

Five passes, or it did not happen

You have forgotten solved problems because you never had to retrieve them. The pass is the study. The checklist's 342 rows are variations of about 40 shapes. Master the shape, then 120 to 150 representatives, not every row.

1. Name the pattern first
2. Close the video
3. Rewrite on an empty file
4. Red comes back sooner

Figure



Do it in this order

1. Pass 1: if you cannot name the pattern, read the lecture, then code.
2. Pass 2 the next day with no video.
3. The Patterns page stores the mark and the next date: day 1, 3, 7, 15, 30.
4. Sliding window's many checklist titles are one loop. Expand, update, shrink, answer.

Worked example

Green: solved alone. Yellow: hint. Red: solution. Blue: pattern yes, code no.

The usual miss

A tick mark the night you watched the editorial.

They will ask

Which pass is this problem on?

Before you close the laptop

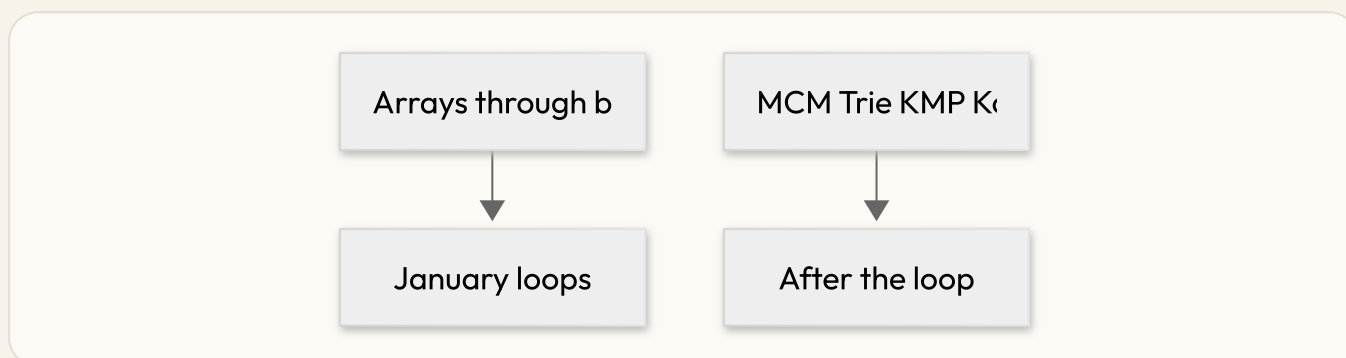
Pick one red problem and schedule only its rewrite tonight.

What January needs, and what waits

By January you can be handed a medium in the core set and start. Advanced string algorithms and exotic graph theorems are not equally important.

1. Core patterns are the job
2. One DP family done deeply
3. MCM is one problem if time
4. KMP is LPS once

Figure



Do it in this order

1. Must: arrays, hash, prefix sums, two pointers, sliding window, binary search, lists, stack, monotonic stack, intervals, greedy, trees, BST, heap, BFS/DFS, topo, Dijkstra, basic DP.
2. Later: MCM, trie depth, KMP and Z as a specialty, Kosaraju, bridges, articulation points.
3. The live pattern pages cover the must-set with Java templates.

Worked example

Basic DP means stairs, robber, grid, subset, coins, one stock shape, one LIS, one LCS. Not

The usual miss

Spending December on Kosaraju while house robber is shaky.

They will ask

Name the pattern for a medium in under a minute.

Before you close the laptop

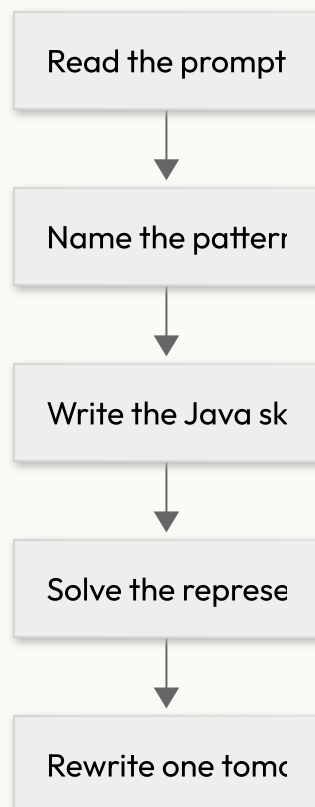
Open the drill and do ten prompts. The misses are the only new DSA.

Hashing / frequency

January. Trade memory for a second pass. Store what you have already seen, then ask the map a yes/no or a count.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Need a pair, count, or membership in $O(1)$ The array is unsorted and a sort would be extra work You care how many times a value appeared Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Trade memory for a second pass. Store what you have already seen, then ask the map a yes/no or a count.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
Map<Integer, Integer> freq = new HashMap<>();
for (int value : nums) {
```

```
freq.merge(value, 1, Integer::sum);  
}
```

Problems that count

Two Sum (Easy). The template. Say the complement out loud before you code.

Group Anagrams (Medium). Key is the sorted word or a 26-count signature.

Longest Consecutive Sequence (Medium). Set lookup. Only start counting at a number with no left neighbor.

Mistakes that make you blank in the interview

Using a list scan inside the loop and calling it hashing Forgetting that `int[]` cannot be a `HashMap` key; use a `String` or a long encoding Updating the map before checking the complement in Two Sum

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Need a pair, count, or membership in $O(1)$ The array is unsorted and a sort would be extra work You care how many times a value appeared
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
Map<Integer, Integer> freq = new HashMap<>();  
for (int value : nums) {  
    freq.merge(value, 1, Integer::sum);  
}
```

The usual miss

Using a list scan inside the loop and calling it hashing

They will ask

Walk Two Sum and say why this pattern fits.

Before you close the laptop

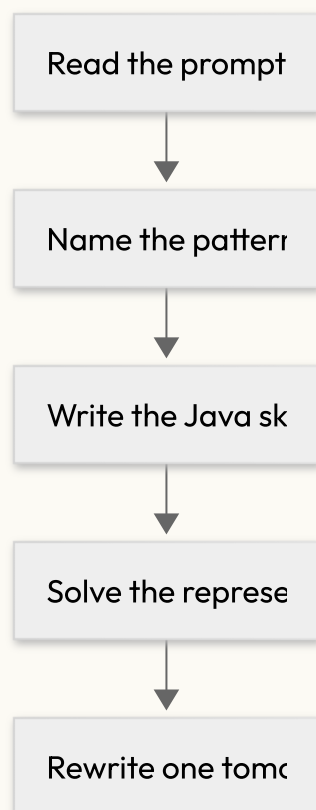
Rewrite Two Sum tomorrow before any new pattern.

Prefix sum + HashMap

January. $\text{prefix}[j] - \text{prefix}[i] = k$ means you have seen $\text{prefix}[j] - k$ before. Store earlier prefixes.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Subarray sum, count of subarrays, or longest subarray with a property The property is about a range, not a single element Brute force is $O(n^2)$ over every $i..j$ Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

$\text{prefix}[j] - \text{prefix}[i] = k$ means you have seen $\text{prefix}[j] - k$ before. Store earlier prefixes.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

Map<Integer, Integer> seen = new HashMap<>();
seen.put(0, 1);
int prefix = 0;
  
```

```
for (int value : nums) {
    prefix += value;
    // query seen.get(prefix - k) before inserting prefix
}
```

Problems that count

Subarray Sum Equals K (Medium). Count version. seen stores frequency.

Contiguous Array (Medium). Treat 0 as -1. Longest subarray with sum 0.

Subarray Sums Divisible by K (Medium). Store prefix mod k. In Java, fix negative mods.

Mistakes that make you blank in the interview

Putting the current prefix into the map before the query Forgetting the empty prefix 0 Using this on an unsorted pair problem that is just Two Sum

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Subarray sum, count of subarrays, or longest subarray with a property The property is about a range, not a single element Brute force is $O(n^2)$ over every $i..j$
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
Map<Integer, Integer> seen = new HashMap<>();
seen.put(0, 1);
int prefix = 0;
for (int value : nums) {
    prefix += value;
    // query seen.get(prefix - k) before inserting prefix
}
```

The usual miss

Putting the current prefix into the map before the query

They will ask

Walk Subarray Sum Equals K and say why this pattern fits.

Before you close the laptop

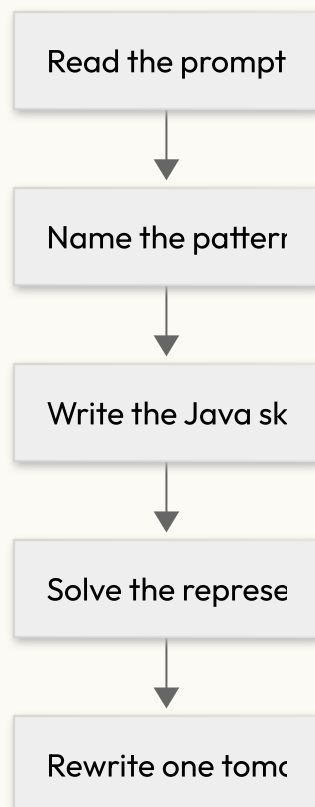
Rewrite Subarray Sum Equals K tomorrow before any new pattern.

Two pointers

January. Sorted order gives you a direction. If the sum is too small, move left. If too big, move right.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Sorted array, or you are allowed to sort. Pair, triplet, or two ends moving toward each other. You can discard one side without missing the answer. Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Sorted order gives you a direction. If the sum is too small, move left. If too big, move right.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
int left = 0;
int right = nums.length - 1;
```

```
while (left < right) {
    int sum = nums[left] + nums[right];
    if (sum == target) return new int[] {left, right};
    if (sum < target) left++;
    else right--;
}
```

Problems that count

Two Sum II (Easy). Sorted input. This is the skeleton.

3Sum (Medium). Sort, fix one index, two-pointer the rest, skip duplicates.

Container With Most Water (Medium). Move the shorter side. Area is width times min height.

Mistakes that make you blank in the interview

Skipping duplicate handling in 3Sum and returning the same triplet
Moving both pointers when only one side is wrong
Using two pointers on an unsorted array without sorting first

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Sorted array, or you are allowed to sort Pair, triplet, or two ends moving toward each other
You can discard one side without missing the answer
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
int left = 0;
int right = nums.length - 1;
while (left < right) {
    int sum = nums[left] + nums[right];
    if (sum == target) return new int[] {left, right};
    if (sum < target) left++;
    else right--;
}
```

The usual miss

Skipping duplicate handling in 3Sum and returning the same triplet

They will ask

Walk Two Sum II and say why this pattern fits.

Before you close the laptop

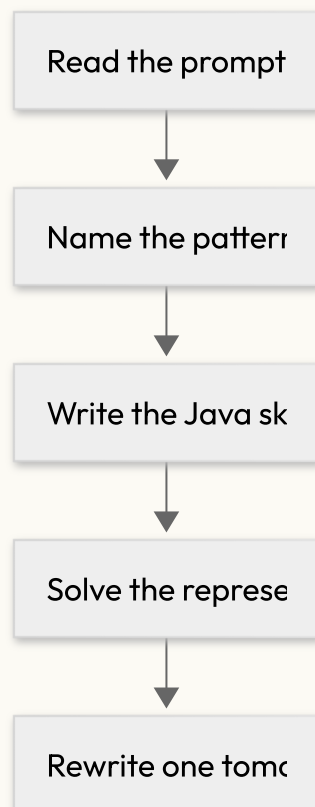
Rewrite Two Sum II tomorrow before any new pattern.

Sliding window

January. Maintain a window that is valid, or the smallest window that just became invalid. The answer hides in how the window changes, not in restarting from every index.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Contiguous subarray or substring Longest, shortest, or count of windows that satisfy a condition Adding the right end and shrinking from the left fixes the condition Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Maintain a window that is valid, or the smallest window that just became invalid. The answer hides in how the window changes, not in restarting from every index.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
int left = 0;
for (int right = 0; right < n; right++) {
    // add s.charAt(right)
    while (/* window breaks the rule */) {
        // remove s.charAt(left++)
    }
    // update answer from [left, right]
}
```

Problems that count

Longest Substring Without Repeating Characters (Medium). Last index of each char. Window of unique chars.

Max Consecutive Ones III (Medium). At most K zeros. The whole family of 'at most K' starts here.

Minimum Window Substring (Hard). Need vs have counts. Shrink while the window still covers the target.

Mistakes that make you blank in the interview

Restarting left from 0 on every right. That is brute force. Shrinking with if instead of while when one removal is not enough Treating at-most-K and exactly-K as the same. Exactly-K is $\text{atMost}(K) - \text{atMost}(K-1)$.

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Contiguous subarray or substring Longest, shortest, or count of windows that satisfy a condition Adding the right end and shrinking from the left fixes the condition
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
int left = 0;
for (int right = 0; right < n; right++) {
    // add s.charAt(right)
    while (/* window breaks the rule */) {
        // remove s.charAt(left++)
    }
    // update answer from [left, right]
}
```

The usual miss

They will ask

Restarting left from 0 on every right. That is brute force.

Walk Longest Substring Without Repeating Characters and say why this pattern fits.

Before you close the laptop

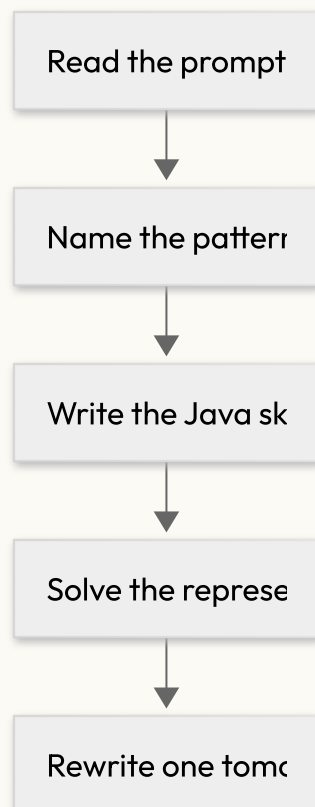
Rewrite Longest Substring Without Repeating Characters tomorrow before any new pattern.

Binary search

January. Keep a range where the answer is still possible. Each step throws away half. Decide which half using the middle, then be strict about inclusive bounds.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Sorted, rotated sorted, or a predicate that flips from false to true once. Find first, last, or any position $O(\log n)$ is required or the array is huge. Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Keep a range where the answer is still possible. Each step throws away half. Decide which half using the middle, then be strict about inclusive bounds.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

int left = 0;
int right = nums.length - 1;
while (left <= right) {
    int mid = left + (right - left) / 2;
    if (nums[mid] == target) return mid;
    if (nums[mid] < target) left = mid + 1;
    else right = mid - 1;
}

```

Problems that count

Binary Search (Easy). Write it closed-interval until you cannot get it wrong.

Search in Rotated Sorted Array (Medium). One half is always sorted. Search there if target lies inside it.

Find First and Last Position (Medium). Two searches: lower bound and upper bound.

Mistakes that make you blank in the interview

mid = (left + right) / 2 overflowing on large bounds. Use left + (right - left) / 2 left < right vs left <= right mixed up, so you loop forever or drop the last index On a rotated array, forgetting to first ask which half is sorted

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Sorted, rotated sorted, or a predicate that flips from false to true once Find first, last, or any position $O(\log n)$ is required or the array is huge
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```

int left = 0;
int right = nums.length - 1;
while (left <= right) {
    int mid = left + (right - left) / 2;
    if (nums[mid] == target) return mid;
    if (nums[mid] < target) left = mid + 1;
    else right = mid - 1;
}

```

The usual miss

They will ask

$\text{mid} = (\text{left} + \text{right}) / 2$ overflowing on large bounds. Use $\text{left} + (\text{right} - \text{left}) / 2$

Walk Binary Search and say why this pattern fits.

Before you close the laptop

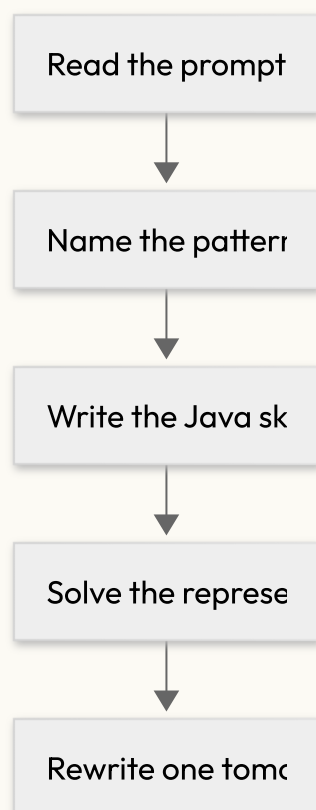
Rewrite Binary Search tomorrow before any new pattern.

Binary search on the answer

January. If X works, every larger X works (or the opposite). Binary search the boundary. The check function is usually a greedy scan.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Minimize the maximum, or maximize the minimum A yes/no check exists: 'can I achieve capacity X ?' The search space is a number range, not an index in the array Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

If X works, every larger X works (or the opposite). Binary search the boundary. The check function is usually a greedy scan.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
int low = 1;
int high = maxPossible;
while (low < high) {
    int mid = low + (high - low) / 2;
    if (can(mid)) high = mid;
    else low = mid + 1;
}
```

Problems that count

Koko Eating Bananas (Medium). Minimum speed. Check is hours needed at speed mid.

Capacity To Ship Packages Within D Days (Medium). Minimum capacity. Same shape as book allocation.

Split Array Largest Sum (Hard). This is book allocation / painter partition. One pattern, three stories.

Mistakes that make you blank in the interview

Binary searching the array index instead of the answer value A check function that is not monotonic Off-by-one on $low < high$ when you want the minimum feasible

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Minimize the maximum, or maximize the minimum A yes/no check exists: 'can I achieve capacity X?' The search space is a number range, not an index in the array
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
int low = 1;
int high = maxPossible;
while (low < high) {
    int mid = low + (high - low) / 2;
    if (can(mid)) high = mid;
    else low = mid + 1;
}
```

The usual miss

Binary searching the array index instead of the answer value

They will ask

Walk Koko Eating Bananas and say why this pattern fits.

Before you close the laptop

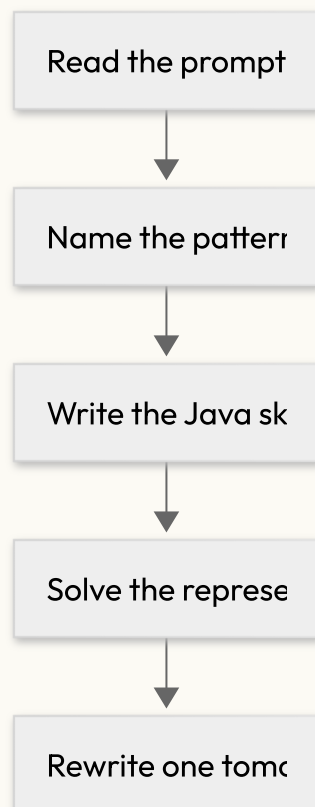
Rewrite Koko Eating Bananas tomorrow before any new pattern.

Matrix traversal

January. Name the boundaries: top, bottom, left, right. Move along one edge, then contract that boundary.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Grid, layers, diagonals, or in-place rotation Boundaries shrink as you walk You must visit every cell once in a defined order Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Name the boundaries: top, bottom, left, right. Move along one edge, then contract that boundary.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

int top = 0, bottom = m - 1, left = 0, right = n - 1;
while (top <= bottom && left <= right) {
    // walk right, down, left, up
}
  
```

```
top++; bottom--; left++; right--;  
}
```

Problems that count

Spiral Matrix (Medium). Four directions, shrink bounds.

Rotate Image (Medium). Transpose, then reverse each row. Or cycle four cells.

Set Matrix Zeroes (Medium). First row and column as markers. Watch the corner cell.

Mistakes that make you blank in the interview

Walking a side after the boundary already collapsed, duplicating cells Rotating by creating a second matrix when the question wants layers of 4-cycles Mixing this with island DFS. Islands are a graph, not a spiral.

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Grid, layers, diagonals, or in-place rotation Boundaries shrink as you walk You must visit every cell once in a defined order
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
int top = 0, bottom = m - 1, left = 0, right = n - 1;  
while (top <= bottom && left <= right) {  
    // walk right, down, left, up  
    top++; bottom--; left++; right--;  
}
```

The usual miss

Walking a side after the boundary already collapsed, duplicating cells

They will ask

Walk Spiral Matrix and say why this pattern fits.

Before you close the laptop

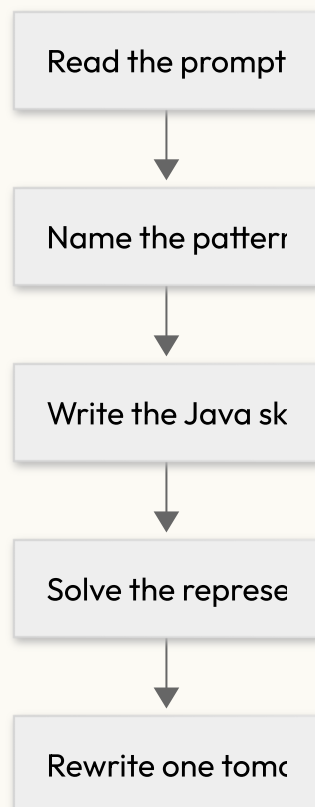
Rewrite Spiral Matrix tomorrow before any new pattern.

Fast / slow pointers

January. In a cycle the fast pointer gains one step per loop and must land on the slow pointer. The meeting point is not always the start; reset one pointer to head to find the start.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Linked list, find the middle, a cycle, or the cycle start You cannot index the list One pointer moving twice as fast meets the other Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

In a cycle the fast pointer gains one step per loop and must land on the slow pointer. The meeting point is not always the start; reset one pointer to head to find the start.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
ListNode slow = head;
ListNode fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
```

Problems that count

Middle of the Linked List (Easy). When fast finishes, slow is the middle.

Linked List Cycle (Easy). Meet means a cycle. No map.

Linked List Cycle II (Medium). After they meet, walk head and slow one step at a time.

Mistakes that make you blank in the interview

Null-checking only fast, then reading fast.next.next Returning the meeting node as the cycle entrance Using an extra HashSet when the question is teaching Floyd

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Linked list, find the middle, a cycle, or the cycle start You cannot index the list One pointer moving twice as fast meets the other
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
ListNode slow = head;
ListNode fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
```

The usual miss

Null-checking only fast, then reading fast.next.next

They will ask

Walk Middle of the Linked List and say why this pattern fits.

Before you close the laptop

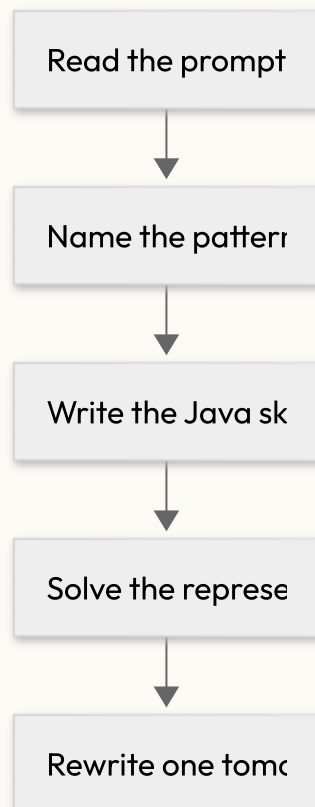
Rewrite Middle of the Linked List tomorrow before any new pattern.

Linked list reversal

January. Three pointers. Point current back to previous before you lose the rest of the list.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Reverse a whole list, k nodes, or the second half Palindrome list, reorder list You need previous, current, next
Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Three pointers. Point current back to previous before you lose the rest of the list.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

ListNode prev = null;
ListNode curr = head;
while (curr != null) {
    ListNode next = curr.next;

```

```
curr.next = prev;
prev = curr;
curr = next;
}
return prev;
```

Problems that count

Reverse Linked List (Easy). Write it until the three lines are automatic.

Reverse Nodes in k-Group (Hard). Count k, reverse that segment, stitch, leave a short tail.

Palindrome Linked List (Easy). Middle, reverse second half, compare, restore if you care.

Mistakes that make you blank in the interview

Losing curr.next before saving it In k-group, reversing a leftover group shorter than k Checking palindrome by copying into an ArrayList and calling it done. Reverse the second half.

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Reverse a whole list, k nodes, or the second half Palindrome list, reorder list You need previous, current, next
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
ListNode prev = null;
ListNode curr = head;
while (curr != null) {
    ListNode next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
}
return prev;
```

The usual miss

Losing curr.next before saving it

They will ask

Walk Reverse Linked List and say why this pattern fits.

Before you close the laptop

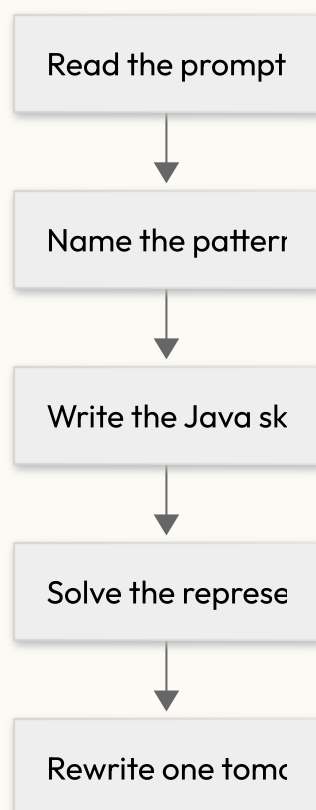
Rewrite Reverse Linked List tomorrow before any new pattern.

Stack and queue

January. A stack remembers unfinished work in reverse. Parentheses, DFS, and monotonic problems all start from that.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Nested structure, last opened must close first Next smaller/greater is a different pattern; plain matching is this one You need undo, a min, or BFS order Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

A stack remembers unfinished work in reverse. Parentheses, DFS, and monotonic problems all start from that.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
Deque<Character> stack = new ArrayDeque<>();
for (char ch : s.toCharArray()) {
    if (isOpen(ch)) stack.push(ch);
    else if (stack.isEmpty() || !matches(stack.pop(), ch)) return false;
}
return stack.isEmpty();
```

Problems that count

Valid Parentheses (Easy). Match pairs. Empty at the end.

Min Stack (Medium). Push the value and the running min together.

Implement Queue using Stacks (Easy). Input stack and output stack. Amortized $O(1)$.

Mistakes that make you blank in the interview

Using Stack instead of ArrayDeque. Stack is synchronized and slower; the idea is the same, the tool is Deque.

Popping an empty stack on a leading close bracket Min Stack storing only the minimum, not the history of minima

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Nested structure, last opened must close first Next smaller/greater is a different pattern; plain matching is this one You need undo, a min, or BFS order
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
Deque<Character> stack = new ArrayDeque<>();
for (char ch : s.toCharArray()) {
    if (isOpen(ch)) stack.push(ch);
    else if (stack.isEmpty() || !matches(stack.pop(), ch)) return false;
}
return stack.isEmpty();
```

The usual miss

Using Stack instead of ArrayDeque. Stack is synchronized and slower; the idea is the same, the tool is Deque.

They will ask

Walk Valid Parentheses and say why this pattern fits.

Before you close the laptop

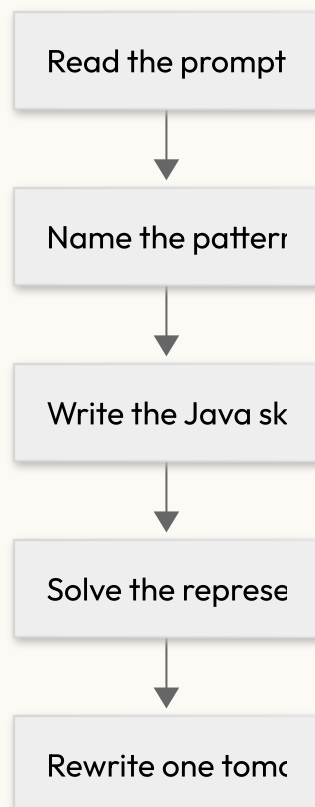
Rewrite Valid Parentheses tomorrow before any new pattern.

Monotonic stack

January. Keep indexes whose values are still waiting for a greater element, in increasing or decreasing order. When the new value beats the top, that top has found its answer.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Next greater, next smaller, previous greater Stock span, daily temperatures, histogram, trapping rain You would otherwise scan left or right for every index Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Keep indexes whose values are still waiting for a greater element, in increasing or decreasing order. When the new value beats the top, that top has found its answer.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
Deque<Integer> stack = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
        int j = stack.pop();
        answer[j] = nums[i];
    }
    stack.push(i);
}
```

Problems that count

Daily Temperatures (Medium). Next warmer day. Store indexes.

Next Greater Element I (Easy). Same loop, then a map from value to answer.

Largest Rectangle in Histogram (Hard). Next and previous smaller. Width is between them.

Mistakes that make you blank in the interview

Storing values when you needed indexes for distance Increasing vs decreasing chosen by habit, not by the question Histogram: forgetting a sentinel 0 so the stack drains

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Next greater, next smaller, previous greater Stock span, daily temperatures, histogram, trapping rain You would otherwise scan left or right for every index
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
Deque<Integer> stack = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
        int j = stack.pop();
        answer[j] = nums[i];
    }
    stack.push(i);
}
```

The usual miss

They will ask

Storing values when you needed indexes for distance

Walk Daily Temperatures and say why this pattern fits.

Before you close the laptop

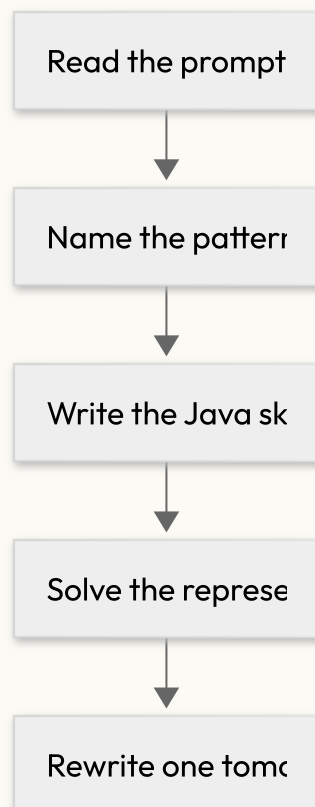
Rewrite Daily Temperatures tomorrow before any new pattern.

Intervals

January. Sort. Then one pass: if the current start is after the previous end, they are disjoint. Otherwise extend the end.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Meetings, ranges, merge, insert, how many overlap Sorting by start or by end changes the question A new interval may swallow several old ones Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Sort. Then one pass: if the current start is after the previous end, they are disjoint. Otherwise extend the end.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));
List<int[]> merged = new ArrayList<>();
```

```

for (int[] interval : intervals) {
    if (merged.isEmpty() || merged.get(merged.size() - 1)[1] < interval[0]) {
        merged.add(interval);
    } else {
        merged.get(merged.size() - 1)[1] =
            Math.max(merged.get(merged.size() - 1)[1], interval[1]);
    }
}

```

Problems that count

Merge Intervals (Medium). Sort by start, extend the last end.

Insert Interval (Medium). Three parts: before, overlapping, after.

Non-overlapping Intervals (Medium). Sort by end. Keep the interval that finishes first.

Mistakes that make you blank in the interview

Sorting by end for merge, and by start for 'remove minimum to avoid overlap' — they are different Using < when the problem treats touching endpoints as overlap Inserting without considering the new interval sitting in a gap

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Meetings, ranges, merge, insert, how many overlap Sorting by start or by end changes the question A new interval may swallow several old ones
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```

Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));
List<int[]> merged = new ArrayList<>();
for (int[] interval : intervals) {
    if (merged.isEmpty() || merged.get(merged.size() - 1)[1] < interval[0]) {
        merged.add(interval);
    } else {
        merged.get(merged.size() - 1)[1] =
            Math.max(merged.get(merged.size() - 1)[1], interval[1]);
    }
}

```

The usual miss

Sorting by end for merge, and by start for
'remove minimum to avoid overlap' — they are
different

They will ask

Walk Merge Intervals and say why this pattern
fits.

Before you close the laptop

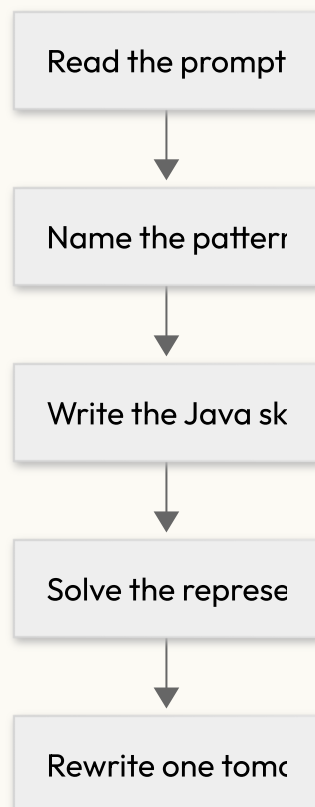
Rewrite Merge Intervals tomorrow before any new pattern.

Greedy

January. Sort by the thing that matters, then take the best available choice. If you cannot explain why a later choice cannot beat it, it is not greedy — it is DP.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

A local choice never needs to be undone Jump as far as you can, schedule the earliest finish, give the smallest cookie that works You can prove the choice with an exchange argument, even informally Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Sort by the thing that matters, then take the best available choice. If you cannot explain why a later choice cannot beat it, it is not greedy — it is DP.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
// Jump Game II: track the farthest reach of this jump window
int jumps = 0, end = 0, far = 0;
for (int i = 0; i < nums.length - 1; i++) {
    far = Math.max(far, i + nums[i]);
    if (i == end) { jumps++; end = far; }
}
```

Problems that count

Jump Game (Medium). Track the farthest index you can reach.

Jump Game II (Medium). Count windows of jumps, not every step.

Gas Station (Medium). If total gas $\geq$ total cost, the unique start is where the tank bottomed out.

Mistakes that make you blank in the interview

Greedy on a problem that needs you to try both choices. Coin change with weird denominations is DP. Sorting the wrong key Jump Game I (can reach) and II (minimum jumps) solved with the same code

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: A local choice never needs to be undone Jump as far as you can, schedule the earliest finish, give the smallest cookie that works You can prove the choice with an exchange argument, even informally
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
// Jump Game II: track the farthest reach of this jump window
int jumps = 0, end = 0, far = 0;
for (int i = 0; i < nums.length - 1; i++) {
    far = Math.max(far, i + nums[i]);
    if (i == end) { jumps++; end = far; }
}
```

The usual miss

Greedy on a problem that needs you to try both choices. Coin change with weird denominations is DP.

They will ask

Walk Jump Game and say why this pattern fits.

Before you close the laptop

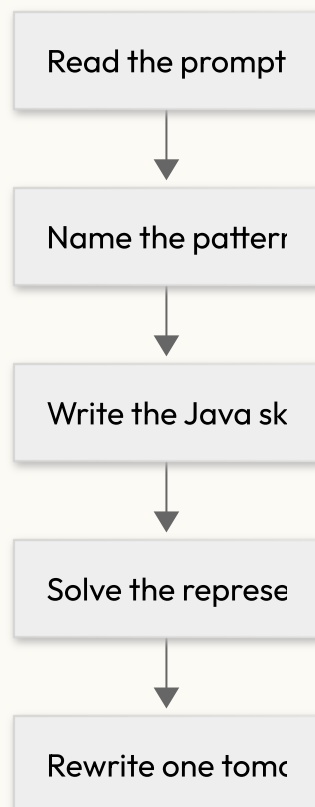
Rewrite Jump Game tomorrow before any new pattern.

Bit and XOR

After the January core is green. XOR cancels pairs: $a \oplus a = 0$ and $a \oplus 0 = a$. For two singles, the lowest set bit of the XOR splits them into two groups.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Every element appears twice except one, or twice except two Find the missing number without extra memory
 The problem mentions bits, parity, or 'constant space' Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

XOR cancels pairs: $a \oplus a = 0$ and $a \oplus 0 = a$. For two singles, the lowest set bit of the XOR splits them into two groups.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
int xor = 0;
for (int value : nums) xor ^= value;
return xor;
```

Problems that count

Single Number (Easy). XOR the array.

Single Number II (Medium). Count each bit mod 3.

Missing Number (Easy). XOR indexes and values, or the gauss formula.

Mistakes that make you blank in the interview

Using XOR for counts of three. Single Number II needs bit counts mod 3. Forgetting $0 \oplus x = x$ and dropping the missing number range Writing a HashSet solution and moving on. Do the bit version once.

When this sits in the calendar

Do this only after the January patterns rewrite cleanly. One evening, one problem. It is not equal in weight to sliding window or basic DP.

Do it in this order

1. Recognition: Every element appears twice except one, or twice except two Find the missing number without extra memory The problem mentions bits, parity, or 'constant space'
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
int xor = 0;
for (int value : nums) xor ^= value;
return xor;
```

The usual miss

Using XOR for counts of three. Single Number II needs bit counts mod 3.

They will ask

Walk Single Number and say why this pattern fits.

Before you close the laptop

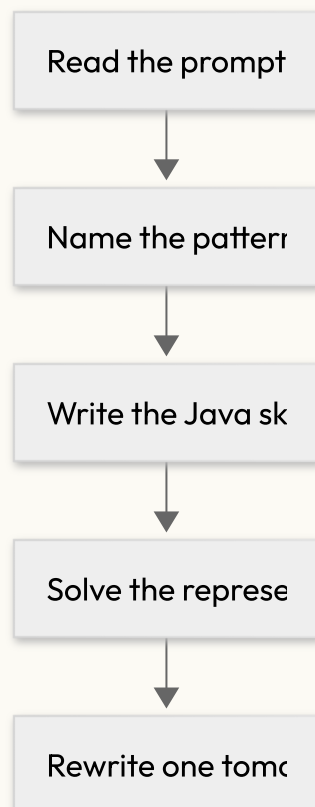
Rewrite Single Number tomorrow before any new pattern.

Backtracking

After the January core is green. The state is the partial decision. Recurse with that decision applied, then revert it so the next sibling sees a clean board.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Generate all combinations, permutations, partitions, or placements You choose, explore, and undo

Constraints prune the tree: queens attack, cells already used, remaining sum Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

The state is the partial decision. Recurse with that decision applied, then revert it so the next sibling sees a clean board.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

void dfs(int start, int remain, List<Integer> path) {
    if (remain == 0) { answer.add(new ArrayList<>(path)); return; }
    for (int i = start; i < candidates.length; i++) {
        if (candidates[i] > remain) break;
        path.add(candidates[i]);
        dfs(i, remain - candidates[i], path);
        path.remove(path.size() - 1);
    }
}

```

Problems that count

Combination Sum (Medium). Reuse the same index if the number can repeat. Copy the path.

Permutations (Medium). Used array, or swap in place and swap back.

N-Queens (Hard). Column, diagonal, anti-diagonal sets. One queen per row.

Mistakes that make you blank in the interview

Adding the path without copying it. The list is mutated later. Forgetting to undo a board cell Starting the next loop at 0 and generating permutations when you wanted combinations

When this sits in the calendar

Do this only after the January patterns rewrite cleanly. One evening, one problem. It is not equal in weight to sliding window or basic DP.

Do it in this order

1. Recognition: Generate all combinations, permutations, partitions, or placements You choose, explore, and undo Constraints prune the tree: queens attack, cells already used, remaining sum
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```

void dfs(int start, int remain, List<Integer> path) {
    if (remain == 0) { answer.add(new ArrayList<>(path)); return; }
    for (int i = start; i < candidates.length; i++) {
        if (candidates[i] > remain) break;
        path.add(candidates[i]);
        dfs(i, remain - candidates[i], path);
        path.remove(path.size() - 1);
    }
}

```

The usual miss

They will ask

Adding the path without copying it. The list is mutated later.

Walk Combination Sum and say why this pattern fits.

Before you close the laptop

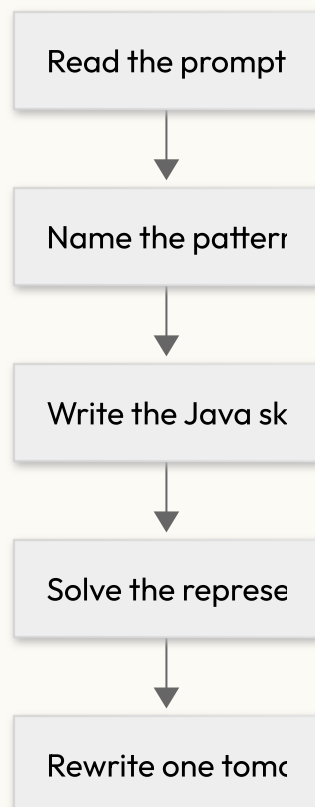
Rewrite Combination Sum tomorrow before any new pattern.

Tree DFS

January. A node asks its children for a small summary, then combines them. Global answers like diameter are updated as a side effect of that summary.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Answer depends on both subtrees: height, diameter, path through a node You can return a value from the child and decide at the parent LCA, path sum, balanced tree Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

A node asks its children for a small summary, then combines them. Global answers like diameter are updated as a side effect of that summary.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
int dfs(TreeNode node) {
    if (node == null) return 0;
    int left = dfs(node.left);
    int right = dfs(node.right);
    best = Math.max(best, left + right);
    return 1 + Math.max(left, right);
}
```

Problems that count

Diameter of Binary Tree (Easy). Height returned, diameter stored outside.

Path Sum III (Medium). Prefix sums on the path down, backtrack the map.

Lowest Common Ancestor of a Binary Tree (Medium). If both sides return a node, this node is the LCA.

Mistakes that make you blank in the interview

Returning the diameter from dfs and also using that return as height Path sum counting paths that must start at the root when any node can start Null pointer on node.left without a base case

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Answer depends on both subtrees: height, diameter, path through a node You can return a value from the child and decide at the parent LCA, path sum, balanced tree
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
int dfs(TreeNode node) {
    if (node == null) return 0;
    int left = dfs(node.left);
    int right = dfs(node.right);
    best = Math.max(best, left + right);
    return 1 + Math.max(left, right);
}
```

The usual miss

Returning the diameter from dfs and also using that return as height

They will ask

Walk Diameter of Binary Tree and say why this pattern fits.

Before you close the laptop

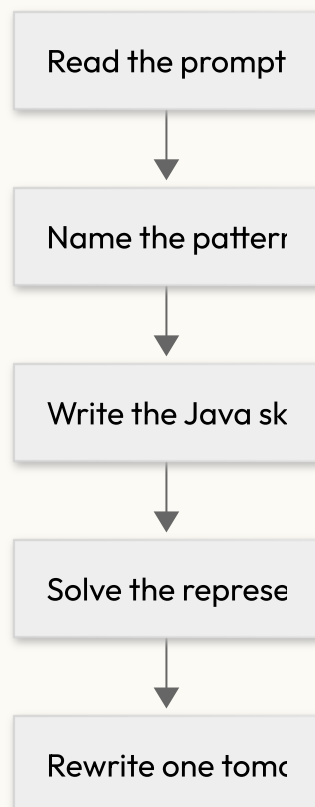
Rewrite Diameter of Binary Tree tomorrow before any new pattern.

Tree BFS

January. Queue plus the size of the current level. Process exactly that many nodes, then the queue holds the next level.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Level order, right view, width, minimum depth The answer is 'the first time I see a level' You need every node at distance d before distance $d+1$ Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Queue plus the size of the current level. Process exactly that many nodes, then the queue holds the next level.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

Queue<TreeNode> queue = new ArrayDeque<>();
queue.add(root);
  
```

```

while (!queue.isEmpty()) {
    int size = queue.size();
    for (int i = 0; i < size; i++) {
        TreeNode node = queue.remove();
        if (node.left != null) queue.add(node.left);
        if (node.right != null) queue.add(node.right);
    }
}

```

Problems that count

Binary Tree Level Order Traversal (Medium). The size snapshot is the whole trick.

Binary Tree Right Side View (Medium). Last node of each level.

Maximum Width of Binary Tree (Medium). Store index with the node. Width is right - left + 1.

Mistakes that make you blank in the interview

Forgetting to snapshot queue.size() and then looping while the queue grows Right view by DFS right-first is fine, but know the BFS version too Maximum width: index nodes as heap indexes, watch overflow with long

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Level order, right view, width, minimum depth The answer is 'the first time I see a level' You need every node at distance d before distance d+1
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```

Queue<TreeNode> queue = new ArrayDeque<>();
queue.add(root);
while (!queue.isEmpty()) {
    int size = queue.size();
    for (int i = 0; i < size; i++) {
        TreeNode node = queue.remove();
        if (node.left != null) queue.add(node.left);
        if (node.right != null) queue.add(node.right);
    }
}

```

The usual miss

They will ask

Forgetting to snapshot `queue.size()` and then looping while the queue grows

Walk Binary Tree Level Order Traversal and say why this pattern fits.

Before you close the laptop

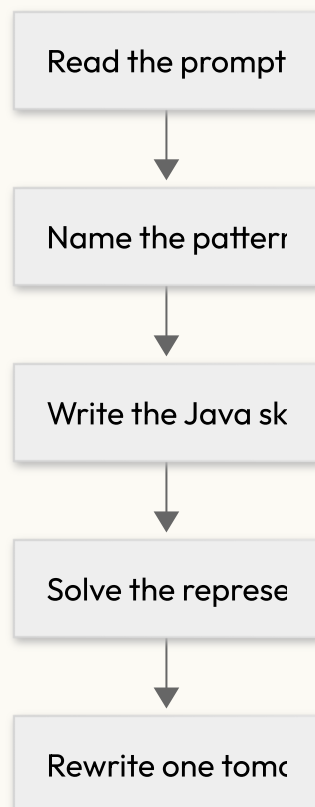
Rewrite Binary Tree Level Order Traversal tomorrow before any new pattern.

BST inorder property

January. Inorder walks values in sorted order. Most BST questions are 'do something on a sorted array' while you walk.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

The tree is a BST, not a general binary tree Kth smallest, validate, successor, two sum Inorder is sorted Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Inorder walks values in sorted order. Most BST questions are 'do something on a sorted array' while you walk.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

void inorder(TreeNode node) {
    if (node == null) return;
    inorder(node.left);

```

```
// node.val is the next sorted value
inorder(node.right);
}
```

Problems that count

Validate Binary Search Tree (Medium). Pass a low and high bound, not just the parent.

Kth Smallest Element in a BST (Medium). Inorder count. Stop at k.

Lowest Common Ancestor of a BST (Medium). Split: one target on each side. Else go left or right.

Mistakes that make you blank in the interview

Running a general-tree LCA when the BST lets you compare with the target and drop a side
Validating only against the parent, not against the whole ancestor range
Collecting the entire inorder list when a counter would do

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: The tree is a BST, not a general binary tree Kth smallest, validate, successor, two sum Inorder is sorted
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
void inorder(TreeNode node) {
    if (node == null) return;
    inorder(node.left);
    // node.val is the next sorted value
    inorder(node.right);
}
```

The usual miss

Running a general-tree LCA when the BST lets you compare with the target and drop a side

They will ask

Walk Validate Binary Search Tree and say why this pattern fits.

Before you close the laptop

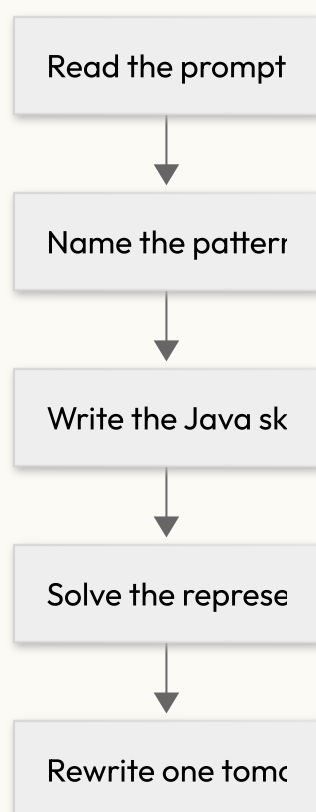
Rewrite Validate Binary Search Tree tomorrow before any new pattern.

Heap / priority queue

January. A heap of size K keeps only the candidates that can still win. Java's PriorityQueue is a min-heap; for max-heap pass a reverse comparator.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Kth largest, top K, merge K sorted, running median You repeatedly need the current min or max Sorting fully is wasted work Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

A heap of size K keeps only the candidates that can still win. Java's PriorityQueue is a min-heap; for max-heap pass a reverse comparator.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

PriorityQueue<Integer> heap = new PriorityQueue<>();
for (int value : nums) {

```

```
heap.offer(value);
if (heap.size() > k) heap.poll();
}
return heap.peek();
```

Problems that count

Kth Largest Element in an Array (Medium). Min-heap of size k. Also know QuickSelect exists.

Top K Frequent Elements (Medium). Frequency map, then a heap of size k. Bucket sort is the follow-up.

Find Median from Data Stream (Hard). Max-heap left, min-heap right.

Mistakes that make you blank in the interview

Default PriorityQueue treated as a max-heap Kth largest built with a heap of size n. Size K is the point.

Median stream: the two heaps must stay balanced, and the max-heap holds the smaller half

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Kth largest, top K, merge K sorted, running median You repeatedly need the current min or max Sorting fully is wasted work
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
PriorityQueue<Integer> heap = new PriorityQueue<>();
for (int value : nums) {
    heap.offer(value);
    if (heap.size() > k) heap.poll();
}
return heap.peek();
```

The usual miss

Default PriorityQueue treated as a max-heap

They will ask

Walk Kth Largest Element in an Array and say why this pattern fits.

Before you close the laptop

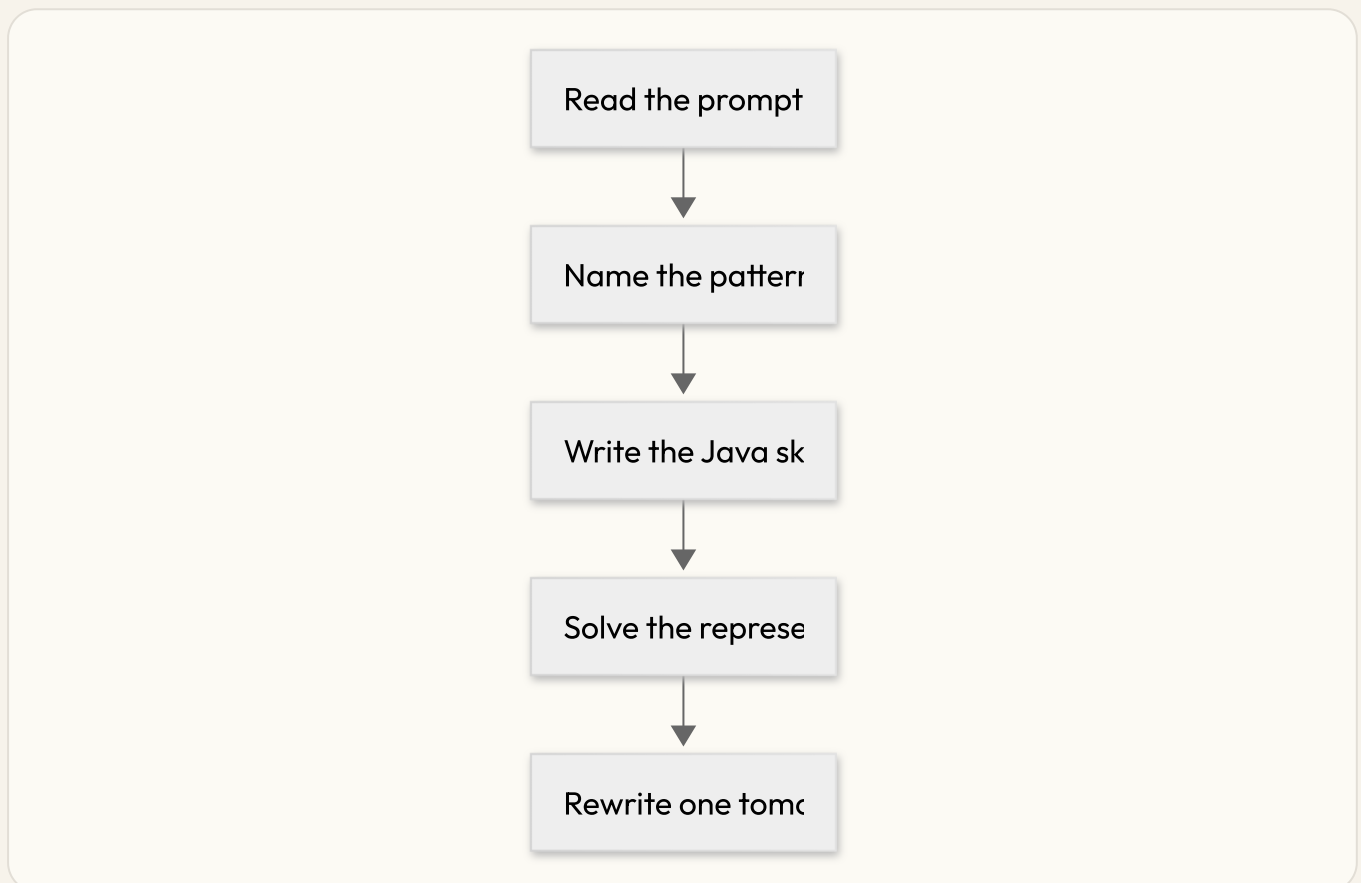
Rewrite Kth Largest Element in an Array tomorrow before any new pattern.

Graph BFS / DFS

January. DFS explores a component. BFS explores by distance. Mark visited when you enqueue or enter, not when you leave, or you will process the same cell many times.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Grid of land and water, rooms, or connected components Shortest path in an unweighted graph Visit each node or cell once Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

DFS explores a component. BFS explores by distance. Mark visited when you enqueue or enter, not when you leave, or you will process the same cell many times.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
void dfs(int r, int c) {
    if (r < 0 || c < 0 || r >= m || c >= n || grid[r][c] == '0') return;
```

```
grid[r][c] = '0';
dfs(r + 1, c); dfs(r - 1, c); dfs(r, c + 1); dfs(r, c - 1);
}
```

Problems that count

Number of Islands (Medium). Each DFS sinks one island.

Rotting Oranges (Medium). Multi-source BFS. Minute equals a layer.

Clone Graph (Medium). Map from original node to the copy. DFS or BFS.

Mistakes that make you blank in the interview

Marking visited on exit, so the queue fills with duplicates Using DFS for shortest path in an unweighted grid

Forgetting the 4-direction vs 8-direction the problem asked for

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Grid of land and water, rooms, or connected components Shortest path in an unweighted graph Visit each node or cell once
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
void dfs(int r, int c) {
    if (r < 0 || c < 0 || r >= m || c >= n || grid[r][c] == '0') return;
    grid[r][c] = '0';
    dfs(r + 1, c); dfs(r - 1, c); dfs(r, c + 1); dfs(r, c - 1);
}
```

The usual miss

Marking visited on exit, so the queue fills with duplicates

They will ask

Walk Number of Islands and say why this pattern fits.

Before you close the laptop

Rewrite Number of Islands tomorrow before any new pattern.

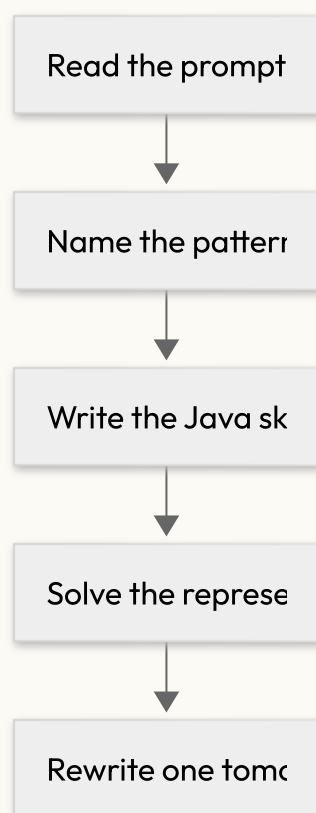
Cycle and bipartite

January. Directed: three colors, visiting and visited. A back edge into visiting is a cycle.

Undirected: a visited neighbor that is not the parent is a cycle. Bipartite: BFS colors, and a neighbor with your color breaks it.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Detect a loop in directed or undirected graph Can you 2-color the nodes Prerequisites that might be circular Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Directed: three colors, visiting and visited. A back edge into visiting is a cycle. Undirected: a visited neighbor that is not the parent is a cycle. Bipartite: BFS colors, and a neighbor with your color breaks it.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
// 0 unseen, 1 visiting, 2 done
boolean dfs(int node) {
    state[node] = 1;
    for (int next : graph[node]) {
        if (state[next] == 1) return true;
        if (state[next] == 0 && dfs(next)) return true;
    }
    state[node] = 2;
    return false;
}
```

Problems that count

Course Schedule (Medium). Directed cycle. If there is a cycle, you cannot finish.

Is Graph Bipartite (Medium). Odd cycle means not bipartite. Color with BFS.

Redundant Connection (Medium). Undirected cycle. DSU fits this one even better.

Mistakes that make you blank in the interview

Using the undirected parent rule on a directed graph
Two colors stored as visited boolean, so you cannot tell a back edge from a cross edge
Building the adjacency list in the wrong direction for prerequisites

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Detect a loop in directed or undirected graph Can you 2-color the nodes Prerequisites that might be circular
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
// 0 unseen, 1 visiting, 2 done
boolean dfs(int node) {
    state[node] = 1;
    for (int next : graph[node]) {
        if (state[next] == 1) return true;
        if (state[next] == 0 && dfs(next)) return true;
    }
    state[node] = 2;
    return false;
}
```

The usual miss

Using the undirected parent rule on a directed graph

They will ask

Walk Course Schedule and say why this pattern fits.

Before you close the laptop

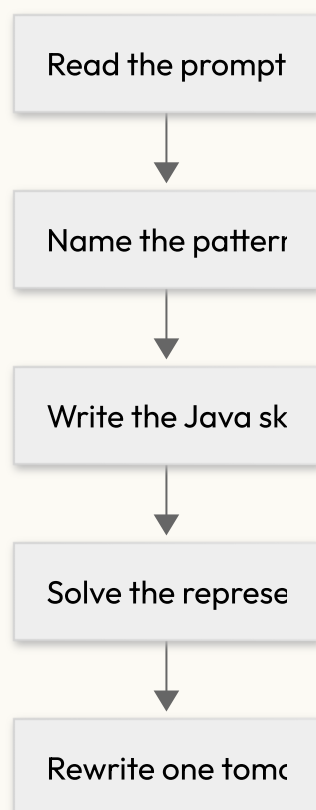
Rewrite Course Schedule tomorrow before any new pattern.

Topological sort

January. Kahn: repeatedly take nodes with indegree 0. If you cannot take every node, there was a cycle. DFS postorder reversed is the other version.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Ordering with prerequisites Alien dictionary, build order, course schedule II Directed acyclic graph Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Kahn: repeatedly take nodes with indegree 0. If you cannot take every node, there was a cycle. DFS postorder reversed is the other version.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

Queue<Integer> queue = new ArrayDeque<>();
for (int i = 0; i < n; i++) if (indegree[i] == 0) queue.add(i);

```

```

List<Integer> order = new ArrayList<>();
while (!queue.isEmpty()) {
    int node = queue.remove();
    order.add(node);
    for (int next : graph[node]) {
        if (--indegree[next] == 0) queue.add(next);
    }
}

```

Problems that count

Course Schedule II (Medium). Kahn's algorithm. Return any valid order.

Course Schedule (Medium). Same graph. Here you only return whether the order is complete.

Alien Dictionary (Hard). Premium on LeetCode. Build edges from adjacent words, then topo. If you lack premium, redo Course Schedule II from a blank file.

Mistakes that make you blank in the interview

Edge direction reversed, so indegree means the wrong thing
 Returning a partial order when a cycle left nodes behind
 Alien dictionary: comparing whole words instead of the first differing character

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Ordering with prerequisites Alien dictionary, build order, course schedule II Directed acyclic graph
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```

Queue<Integer> queue = new ArrayDeque<>();
for (int i = 0; i < n; i++) if (indegree[i] == 0) queue.add(i);
List<Integer> order = new ArrayList<>();
while (!queue.isEmpty()) {
    int node = queue.remove();
    order.add(node);
    for (int next : graph[node]) {
        if (--indegree[next] == 0) queue.add(next);
    }
}

```

The usual miss

Edge direction reversed, so indegree means the wrong thing

They will ask

Walk Course Schedule II and say why this pattern fits.

Before you close the laptop

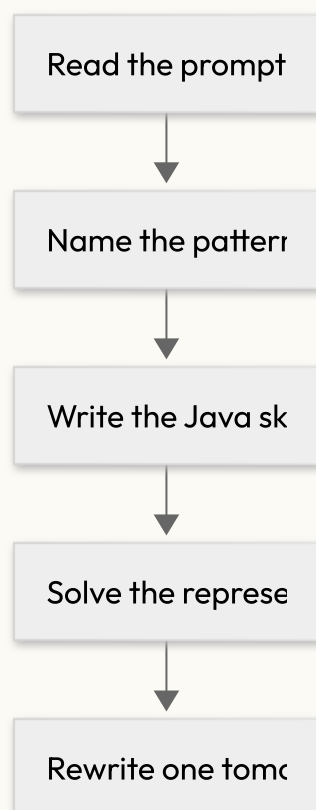
Rewrite Course Schedule II tomorrow before any new pattern.

Shortest path

January. Do not Dijkstra an unweighted maze. The algorithm follows the constraint. Dijkstra's heap pops the next closest settled node.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Unweighted: BFS. Non-negative weights: Dijkstra. Negative edges: Bellman-Ford. All pairs: Floyd-Warshall A grid with cost, a network delay, flights with at most K stops Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Do not Dijkstra an unweighted maze. The algorithm follows the constraint. Dijkstra's heap pops the next closest settled node.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

PriorityQueue<int[]> heap = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
int[] dist = new int[n];
Arrays.fill(dist, Integer.MAX_VALUE);
dist[src] = 0;
heap.offer(new int[] {0, src});
while (!heap.isEmpty()) {
    int[] cur = heap.poll();
    if (cur[0] != dist[cur[1]]) continue;
    // relax edges
}

```

Problems that count

Shortest Path in Binary Matrix (Medium). Unweighted. BFS, 8 directions.

Network Delay Time (Medium). Dijkstra. Answer is the max distance if every node is reached.

Cheapest Flights Within K Stops (Medium). Bellman-Ford limited to K+1 edges, or Dijkstra state (node, stops).

Mistakes that make you blank in the interview

Missing the stale-heap check, so you relax outdated distances Bellman-Ford run $|V|$ times and forgetting the extra pass only detects a negative cycle Floyd-Warshall k loop outside i, j. The order is k, then i, then j.

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Unweighted: BFS. Non-negative weights: Dijkstra. Negative edges: Bellman-Ford. All pairs: Floyd-Warshall A grid with cost, a network delay, flights with at most K stops
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```

PriorityQueue<int[]> heap = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
int[] dist = new int[n];
Arrays.fill(dist, Integer.MAX_VALUE);
dist[src] = 0;
heap.offer(new int[] {0, src});
while (!heap.isEmpty()) {
    int[] cur = heap.poll();
    if (cur[0] != dist[cur[1]]) continue;
    // relax edges
}

```

The usual miss

Missing the stale-heap check, so you relax outdated distances

They will ask

Walk Shortest Path in Binary Matrix and say why this pattern fits.

Before you close the laptop

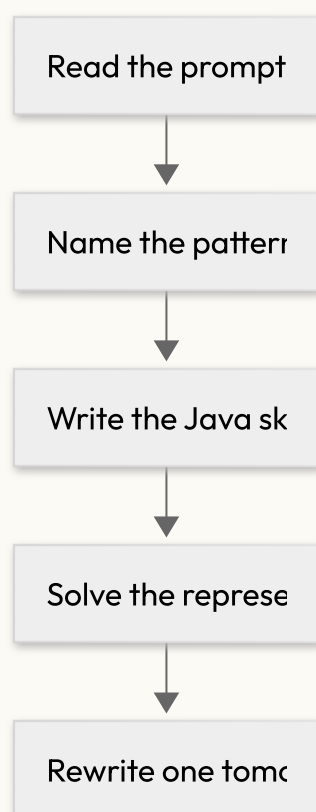
Rewrite Shortest Path in Binary Matrix tomorrow before any new pattern.

Disjoint set union

After the January core is green. Parent array plus rank or size. Find compresses the path. Union attaches the smaller tree under the larger.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Merge groups, connectivity, 'are these two in the same component' online Accounts merge, number of provinces, redundant edge You do not need the actual path, only the group Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Parent array plus rank or size. Find compresses the path. Union attaches the smaller tree under the larger.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
}
  
```

```

    return parent[x];
}
boolean union(int a, int b) {
    int pa = find(a), pb = find(b);
    if (pa == pb) return false;
    parent[pb] = pa;
    return true;
}

```

Problems that count

Number of Provinces (Medium). Each successful union reduces the component count.

Redundant Connection (Medium). The edge that connects two nodes already in the same set.

Accounts Merge (Medium). Union emails that share an account. Group by root.

Mistakes that make you blank in the interview

Forgetting path compression and then timing out Union by comparing raw indexes instead of roots Accounts merge: union emails, then forget to map the root email back to a name

When this sits in the calendar

Do this only after the January patterns rewrite cleanly. One evening, one problem. It is not equal in weight to sliding window or basic DP.

Do it in this order

1. Recognition: Merge groups, connectivity, 'are these two in the same component' online Accounts merge, number of provinces, redundant edge You do not need the actual path, only the group
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}
boolean union(int a, int b) {
    int pa = find(a), pb = find(b);
    if (pa == pb) return false;
    parent[pb] = pa;
    return true;
}

```

The usual miss

They will ask

Forgetting path compression and then timing out

Walk Number of Provinces and say why this pattern fits.

Before you close the laptop

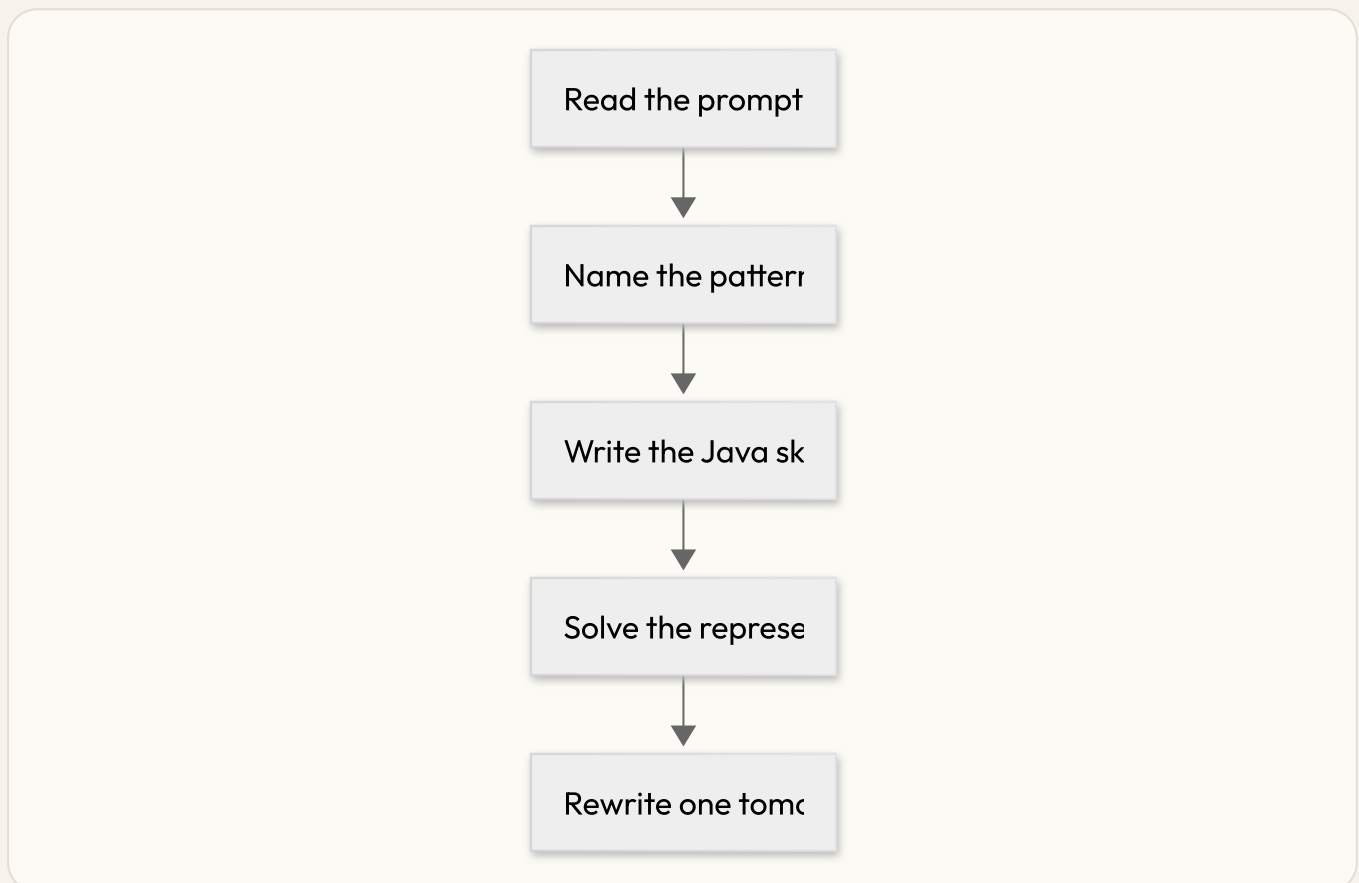
Rewrite Number of Provinces tomorrow before any new pattern.

Minimum spanning tree

After the January core is green. Kruskal is 'sort edges, add if DSU says the endpoints are still disconnected'. Stop after $n-1$ edges.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Connect all points at minimum cost No cycles in the chosen edges, and every node ends up connected Kruskal sorts edges. Prim grows a frontier. Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Kruskal is 'sort edges, add if DSU says the endpoints are still disconnected'. Stop after $n-1$ edges.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
Arrays.sort(edges, Comparator.comparingInt(e -> e[2]));
int cost = 0, used = 0;
```

```
for (int[] edge : edges) {
    if (union(edge[0], edge[1])) {
        cost += edge[2];
        if (++used == n - 1) break;
    }
}
```

Problems that count

Min Cost to Connect All Points (Medium). Manhattan distance edges. Kruskal or Prim.

Connecting Cities With Minimum Cost (Medium). Premium. If locked, re-implement Kruskal on a handwritten edge list until the DSU check is boring.

Optimize Water Distribution (Hard). Add a virtual well node. Still Kruskal.

Mistakes that make you blank in the interview

Adding an edge inside a component and calling it a tree Prim's decrease-key faked badly and visiting a node twice without a visited set Confusing MST with shortest path. Dijkstra does not build an MST in general.

When this sits in the calendar

Do this only after the January patterns rewrite cleanly. One evening, one problem. It is not equal in weight to sliding window or basic DP.

Do it in this order

1. Recognition: Connect all points at minimum cost No cycles in the chosen edges, and every node ends up connected Kruskal sorts edges. Prim grows a frontier.
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
Arrays.sort(edges, Comparator.comparingInt(e -> e[2]));
int cost = 0, used = 0;
for (int[] edge : edges) {
    if (union(edge[0], edge[1])) {
        cost += edge[2];
        if (++used == n - 1) break;
    }
}
```

The usual miss

Adding an edge inside a component and calling it a tree

They will ask

Walk Min Cost to Connect All Points and say why this pattern fits.

Before you close the laptop

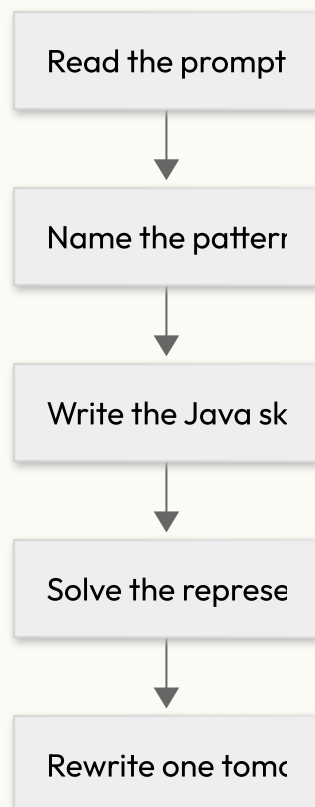
Rewrite Min Cost to Connect All Points tomorrow before any new pattern.

1D DP

January. Write the recurrence in words first. $dp[i]$ is the answer for the prefix ending at i . Then see if you only need the last two values.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Climb stairs, rob houses, frog jumps, decode ways The answer at i depends on a fixed small set of earlier answers Recursion with the same arguments repeats Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Write the recurrence in words first. $dp[i]$ is the answer for the prefix ending at i . Then see if you only need the last two values.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
int prev2 = 1; // ways to stand before the first step
int prev1 = 1;
for (int i = 2; i <= n; i++) {
    int cur = prev1 + prev2;
    prev2 = prev1;
    prev1 = cur;
}
```

Problems that count

Climbing Stairs (Easy). Fibonacci. Say why $dp[i] = dp[i-1] + dp[i-2]$.

House Robber (Medium). Rob this house plus $dp[i-2]$, or skip and take $dp[i-1]$.

House Robber II (Medium). Houses form a circle. Run the linear solution twice, without first or without last.

Mistakes that make you blank in the interview

Coding a 2D table for a recurrence that is one array House Robber including both neighbors Jumping to memo code before you can say the state in one sentence

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Climb stairs, rob houses, frog jumps, decode ways The answer at i depends on a fixed small set of earlier answers Recursion with the same arguments repeats
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
int prev2 = 1; // ways to stand before the first step
int prev1 = 1;
for (int i = 2; i <= n; i++) {
    int cur = prev1 + prev2;
    prev2 = prev1;
    prev1 = cur;
}
```

The usual miss

Coding a 2D table for a recurrence that is one array

They will ask

Walk Climbing Stairs and say why this pattern fits.

Before you close the laptop

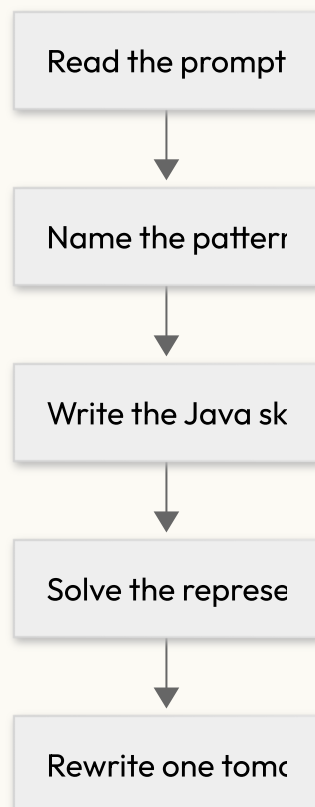
Rewrite Climbing Stairs tomorrow before any new pattern.

Grid DP

January. $dp[r][c]$ comes from the cells that can move into it. Fill in an order that already knows those cells.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Paths across a grid, minimum path sum, falling paths, triangle You only move right/down, or to adjacent cells in the next row Overlapping subproblems on cells Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

$dp[r][c]$ comes from the cells that can move into it. Fill in an order that already knows those cells.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

for (int r = 0; r < m; r++) {
    for (int c = 0; c < n; c++) {

```

```

    if (r == 0 && c == 0) continue;
    int fromTop = r > 0 ? dp[r - 1][c] : Integer.MAX_VALUE;
    int fromLeft = c > 0 ? dp[r][c - 1] : Integer.MAX_VALUE;
    dp[r][c] = grid[r][c] + Math.min(fromTop, fromLeft);
}
}

```

Problems that count

Unique Paths (Medium). Combinatorics works too. Still write the DP once.

Minimum Path Sum (Medium). Add the cell to the min of top and left.

Triangle (Medium). Bottom-up from the last row saves you the boundary pain.

Mistakes that make you blank in the interview

Integer overflow when seeding unreachable cells with Integer.MAX_VALUE and then adding. Walking the grid with DFS and no memo, then wondering why it TLEs Unique Paths obstacles: a blocked cell is 0 ways, and it must not add into neighbors.

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Paths across a grid, minimum path sum, falling paths, triangle. You only move right/down, or to adjacent cells in the next row. Overlapping subproblems on cells.
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```

for (int r = 0; r < m; r++) {
    for (int c = 0; c < n; c++) {
        if (r == 0 && c == 0) continue;
        int fromTop = r > 0 ? dp[r - 1][c] : Integer.MAX_VALUE;
        int fromLeft = c > 0 ? dp[r][c - 1] : Integer.MAX_VALUE;
        dp[r][c] = grid[r][c] + Math.min(fromTop, fromLeft);
    }
}

```

The usual miss

Integer overflow when seeding unreachable cells with Integer.MAX_VALUE and then adding

They will ask

Walk Unique Paths and say why this pattern fits.

Before you close the laptop

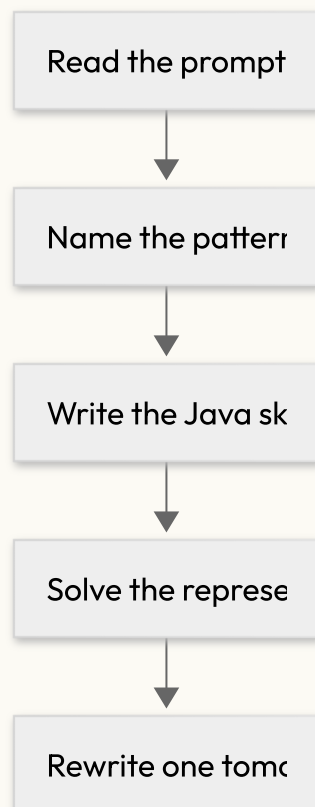
Rewrite Unique Paths tomorrow before any new pattern.

Knapsack / subset DP

January. 0/1: iterate capacity downward so each item is used once. Unbounded coins: iterate capacity upward so the same coin can be reused.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Pick or skip each item once Subset sum, partition, coin change, target sum Capacity is the second dimension
Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

0/1: iterate capacity downward so each item is used once. Unbounded coins: iterate capacity upward so the same coin can be reused.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
boolean[] can = new boolean[target + 1];
can[0] = true;
```

```
for (int num : nums) {
    for (int sum = target; sum >= num; sum--) {
        can[sum] = can[sum] || can[sum - num];
    }
}
```

Problems that count

Partition Equal Subset Sum (Medium). 0/1 subset sum to total/2.

Coin Change (Medium). Unbounded. Minimum count. Seed with a large number, not the combination count.

Target Sum (Medium). Same as count of subsets with a derived sum.

Mistakes that make you blank in the interview

Looping capacity upward on a 0/1 problem and using an item twice Coin Change I (fewest coins) confused with Coin Change II (number of combinations) Partition: forgetting the total must be even, and the target is total/2

When this sits in the calendar

This is in the January must-set. It belongs in the 75-minute DSA block on weekdays. Do not add a new pattern on a day the previous rewrite failed.

Do it in this order

1. Recognition: Pick or skip each item once Subset sum, partition, coin change, target sum Capacity is the second dimension
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
boolean[] can = new boolean[target + 1];
can[0] = true;
for (int num : nums) {
    for (int sum = target; sum >= num; sum--) {
        can[sum] = can[sum] || can[sum - num];
    }
}
```

The usual miss

Looping capacity upward on a 0/1 problem and using an item twice

They will ask

Walk Partition Equal Subset Sum and say why this pattern fits.

Before you close the laptop

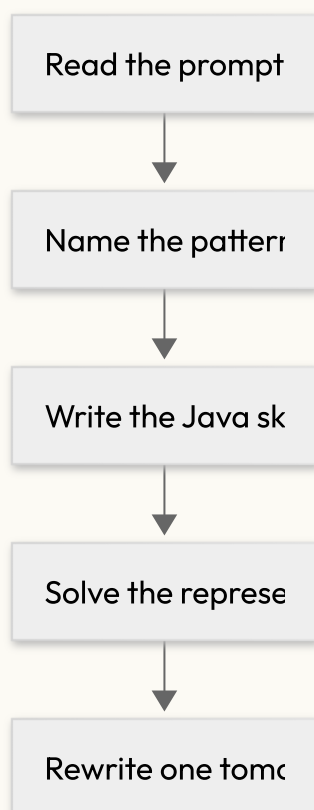
Rewrite Partition Equal Subset Sum tomorrow before any new pattern.

Stock DP

After the January core is green. At each day you hold or you do not. Transitions are buy, sell, rest. Fees and cooldowns only change one transition.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Buy and sell with a limit on transactions, a cooldown, or a fee State is day plus whether you hold a share, plus transactions left Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

At each day you hold or you do not. Transitions are buy, sell, rest. Fees and cooldowns only change one transition.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
int hold = -prices[0];
int cash = 0;
```

```
for (int i = 1; i < prices.length; i++) {  
    hold = Math.max(hold, cash - prices[i]);  
    cash = Math.max(cash, hold + prices[i]);  
}
```

Problems that count

Best Time to Buy and Sell Stock (Easy). Min price so far, max profit. This is the one-transaction base.

Best Time to Buy and Sell Stock II (Medium). Unlimited trades. Add every upward difference, or the hold/cash pair.

Best Time to Buy and Sell Stock with Cooldown (Medium). Sold state cannot buy the next day.

Mistakes that make you blank in the interview

One variable trying to remember a cooldown. Cooldown needs a third state. At most K transactions exploding into a new formula instead of a loop over k Selling and buying on the same day when the problem forbids it

When this sits in the calendar

Do this only after the January patterns rewrite cleanly. One evening, one problem. It is not equal in weight to sliding window or basic DP.

Do it in this order

1. Recognition: Buy and sell with a limit on transactions, a cooldown, or a fee State is day plus whether you hold a share, plus transactions left
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
int hold = -prices[0];  
int cash = 0;  
for (int i = 1; i < prices.length; i++) {  
    hold = Math.max(hold, cash - prices[i]);  
    cash = Math.max(cash, hold + prices[i]);  
}
```

The usual miss

One variable trying to remember a cooldown.
Cooldown needs a third state.

They will ask

Walk Best Time to Buy and Sell Stock and say why this pattern fits.

Before you close the laptop

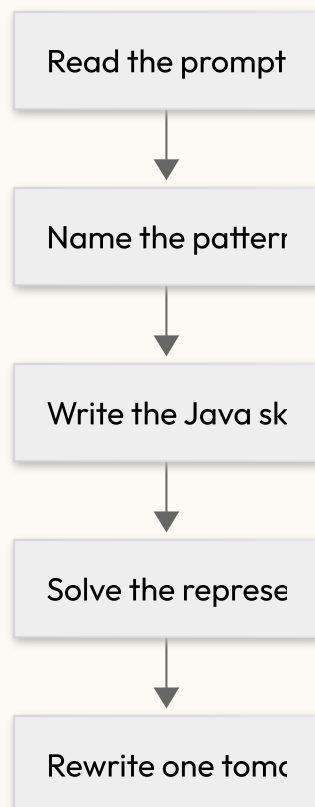
Rewrite Best Time to Buy and Sell Stock tomorrow before any new pattern.

LIS pattern

After the January core is green. $O(n^2)$: $dp[i]$ is the best chain ending at i . $O(n \log n)$: $tails[len]$ is the smallest tail of an increasing subsequence of that length.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Longest increasing subsequence, not subarray Russian dolls, divisible subset, maximum height stack of boxes
 Patience sorting / binary search on tails for $O(n \log n)$ Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

$O(n^2)$: $dp[i]$ is the best chain ending at i . $O(n \log n)$: $tails[len]$ is the smallest tail of an increasing subsequence of that length.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
int[] tails = new int[n];
int len = 0;
for (int value : nums) {
    int i = Arrays.binarySearch(tails, 0, len, value);
    if (i < 0) i = -(i + 1);
    tails[i] = value;
    if (i == len) len++;
}
```

Problems that count

Longest Increasing Subsequence (Medium). Write $O(n^2)$ first. Then the tails array.

Largest Divisible Subset (Medium). Sort, then predecessor links so you can rebuild the subset.

Number of Longest Increasing Subsequence (Medium). Store length and count ending at i .

Mistakes that make you blank in the interview

Sorting when the order of the original array matters. LIS is not 'sort then count'. `binarySearch` insertion point handled wrong on negative results Divisible subset: sort first, then it becomes LIS with a divisibility check

When this sits in the calendar

Do this only after the January patterns rewrite cleanly. One evening, one problem. It is not equal in weight to sliding window or basic DP.

Do it in this order

1. Recognition: Longest increasing subsequence, not subarray Russian dolls, divisible subset, maximum height stack of boxes Patience sorting / binary search on tails for $O(n \log n)$
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
int[] tails = new int[n];
int len = 0;
for (int value : nums) {
    int i = Arrays.binarySearch(tails, 0, len, value);
    if (i < 0) i = -(i + 1);
    tails[i] = value;
    if (i == len) len++;
}
```

The usual miss

They will ask

Sorting when the order of the original array matters. LIS is not 'sort then count'.

Walk Longest Increasing Subsequence and say why this pattern fits.

Before you close the laptop

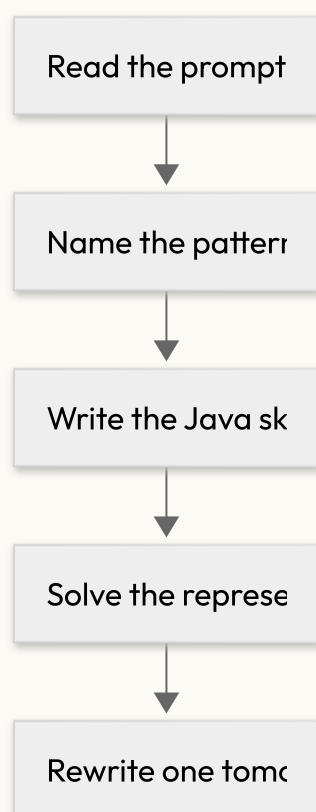
Rewrite Longest Increasing Subsequence tomorrow before any new pattern.

LCS pattern

After the January core is green. If the characters match, take the diagonal plus one. If not, take the better of skipping i or skipping j.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Two strings, subsequence or substring in common Delete both to equal, shortest common supersequence, distinct subsequences A grid of i versus j Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

If the characters match, take the diagonal plus one. If not, take the better of skipping i or skipping j.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
int[][] dp = new int[m + 1][n + 1];
for (int i = 1; i <= m; i++) {
```

```

for (int j = 1; j <= n; j++) {
    if (a.charAt(i - 1) == b.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1] + 1;
    else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
}
}

```

Problems that count

Longest Common Subsequence (Medium). The table above. Rebuild one string on paper.

Delete Operation for Two Strings (Medium). $m + n - 2 * \text{LCS}$.

Shortest Common Supersequence (Hard). Walk the LCS table and append the characters you skipped.

Mistakes that make you blank in the interview

Subsequence vs substring. Substring cannot skip in the middle; the streak resets. Indexing `dp[i][j]` against `charAt(i)` instead of `charAt(i - 1)` Shortest common supersequence is $m + n - \text{LCS}$, but rebuilding the string needs the table

When this sits in the calendar

Do this only after the January patterns rewrite cleanly. One evening, one problem. It is not equal in weight to sliding window or basic DP.

Do it in this order

1. Recognition: Two strings, subsequence or substring in common Delete both to equal, shortest common supersequence, distinct subsequences A grid of i versus j
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```

int[][] dp = new int[m + 1][n + 1];
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (a.charAt(i - 1) == b.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1] + 1;
        else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
    }
}

```

The usual miss

Subsequence vs substring. Substring cannot skip in the middle; the streak resets.

They will ask

Walk Longest Common Subsequence and say why this pattern fits.

Before you close the laptop

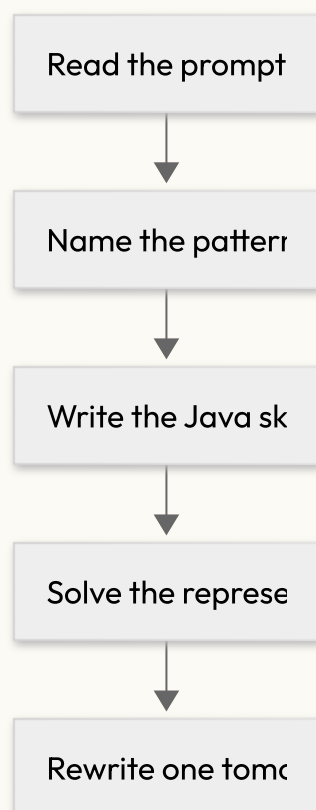
Rewrite Longest Common Subsequence tomorrow before any new pattern.

String DP

After January. Edit distance is insert, delete, replace. A match or replace comes from the diagonal. Insert and delete come from the neighbors.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Edit distance, palindrome cuts, wildcard, regex, distinct subsequences. Two indexes, or a substring $i..j$. Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Edit distance is insert, delete, replace. A match or replace comes from the diagonal. Insert and delete come from the neighbors.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
    dp[i][j] = dp[i - 1][j - 1];
}

```

```

} else {
    dp[i][j] = 1 + Math.min(dp[i - 1][j - 1], Math.min(dp[i - 1][j], dp[i][j - 1]));
}

```

Problems that count

Longest Palindromic Subsequence (Medium). LCS of the string with its reverse.

Edit Distance (Medium). The three operations. Base cases are the gaps.

Distinct Subsequences (Hard). Count ways. Match uses diagonal plus the skip.

Mistakes that make you blank in the interview

Wildcard star consuming the wrong row. Draw a 4-character example before coding. Palindrome subsequence confused with palindrome substring Base row and column left as 0, so turning an empty string into 'abc' costs nothing

When this sits in the calendar

Leave this until after January interviews are underway. MCM, trie depth, KMP, Z, and the exotic graph algorithms are real, and they are the wrong use of December.

Do it in this order

1. Recognition: Edit distance, palindrome cuts, wildcard, regex, distinct subsequences Two indexes, or a substring i..j
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```

if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
    dp[i][j] = dp[i - 1][j - 1];
} else {
    dp[i][j] = 1 + Math.min(dp[i - 1][j - 1], Math.min(dp[i - 1][j], dp[i][j - 1]));
}

```

The usual miss

Wildcard star consuming the wrong row. Draw a 4-character example before coding.

They will ask

Walk Longest Palindromic Subsequence and say why this pattern fits.

Before you close the laptop

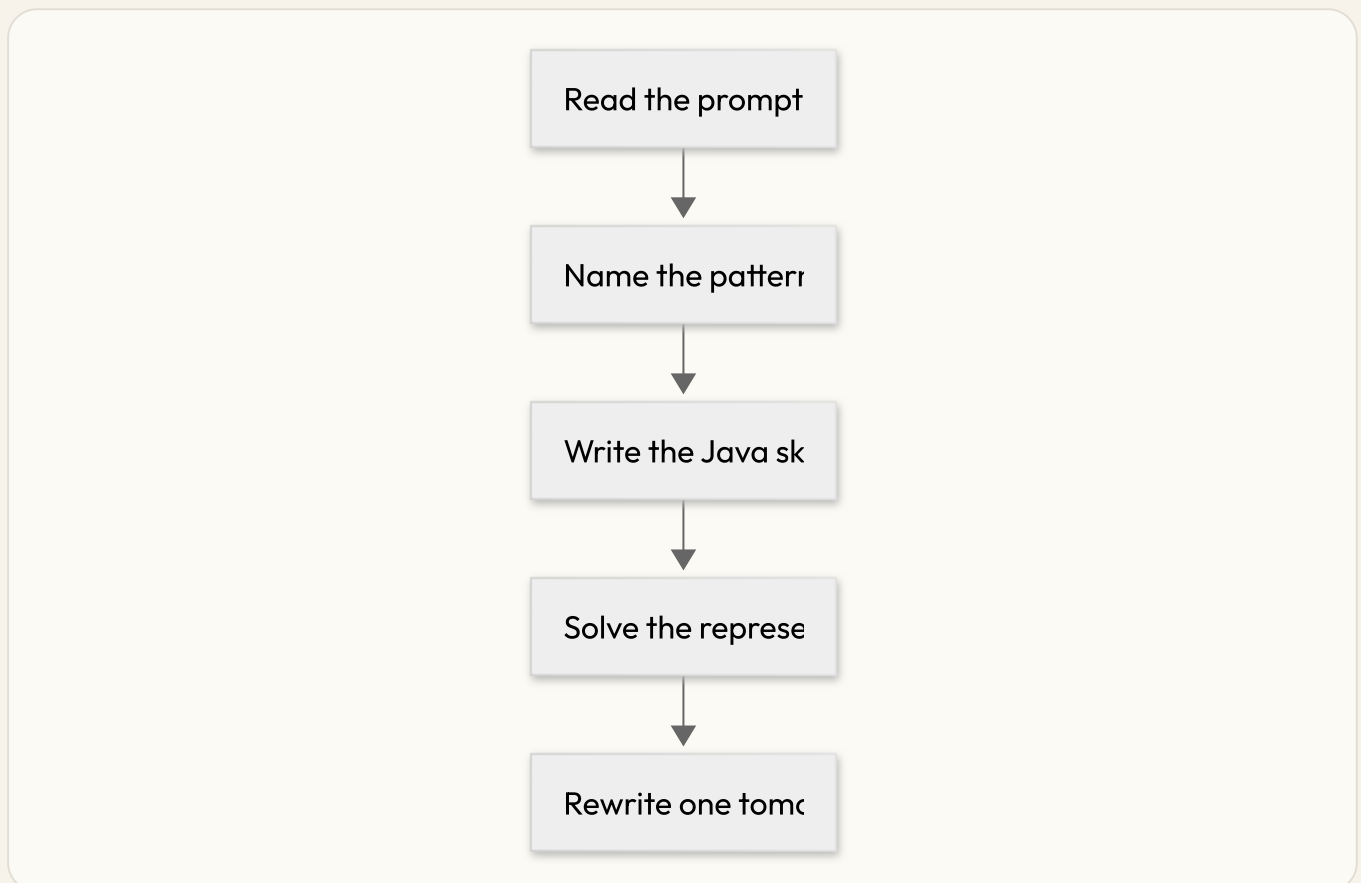
Rewrite Longest Palindromic Subsequence tomorrow before any new pattern.

MCM / partition DP

After January. $dp[i][j]$ is the best way to solve the slice $i..j$. Try every split. Fill short slices before long ones.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Burst balloons, matrix chain, palindrome partitioning cost You split an array at every k between i and j and combine the two sides The interval length grows from small to large Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

$dp[i][j]$ is the best way to solve the slice $i..j$. Try every split. Fill short slices before long ones.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

for (int len = 2; len <= n; len++) {
    for (int i = 0; i + len - 1 < n; i++) {

```

```

    int j = i + len - 1;
    for (int k = i; k < j; k++) {
        dp[i][j] = Math.min(dp[i][j], dp[i][k] + dp[k + 1][j] + cost(i, k, j));
    }
}
}

```

Problems that count

Burst Balloons (Hard). The one to blank-rewrite. Coins on the ends are 1.

Palindrome Partitioning II (Hard). Minimum cuts. Precompute which slices are palindromes.

Matrix Chain Multiplication (Hard). Not a LeetCode staple. If time is gone, skip and keep Burst Balloons sharp.

Mistakes that make you blank in the interview

Filling the table by row before shorter lengths exist Burst balloons: forgetting the padded 1s on both ends, so the last balloon has no neighbors Trying to master five MCM problems. One blank rewrite of Burst Balloons is the goal.

When this sits in the calendar

Leave this until after January interviews are underway. MCM, trie depth, KMP, Z, and the exotic graph algorithms are real, and they are the wrong use of December.

Do it in this order

1. Recognition: Burst balloons, matrix chain, palindrome partitioning cost You split an array at every k between i and j and combine the two sides The interval length grows from small to large
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```

for (int len = 2; len <= n; len++) {
    for (int i = 0; i + len - 1 < n; i++) {
        int j = i + len - 1;
        for (int k = i; k < j; k++) {
            dp[i][j] = Math.min(dp[i][j], dp[i][k] + dp[k + 1][j] + cost(i, k, j));
        }
    }
}

```

The usual miss

Filling the table by row before shorter lengths exist

They will ask

Walk Burst Balloons and say why this pattern fits.

Before you close the laptop

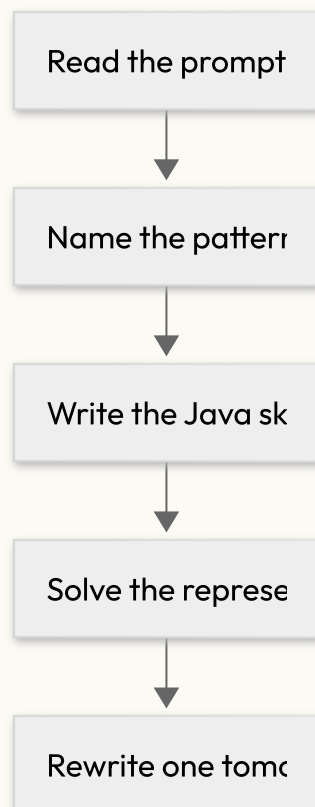
Rewrite Burst Balloons tomorrow before any new pattern.

Trie

After January. Each edge is a character. A word is a path. Sharing prefixes is the memory win. XOR trie walks the opposite bit when it exists.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Many prefix queries Word search on a board with a dictionary XOR maximize: a binary trie of bits Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Each edge is a character. A word is a path. Sharing prefixes is the memory win. XOR trie walks the opposite bit when it exists.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```

class Node {
    Node[] next = new Node[26];
  }

```

```

        boolean word;
    }
    void insert(String word) {
        Node cur = root;
        for (char ch : word.toCharArray()) {
            int i = ch - 'a';
            if (cur.next[i] == null) cur.next[i] = new Node();
            cur = cur.next[i];
        }
        cur.word = true;
    }
}

```

Problems that count

Implement Trie (Medium). Insert, search, startsWith.

Word Search II (Hard). Trie plus board DFS. Remove a word after you find it.

Maximum XOR of Two Numbers (Medium). Binary trie. Prefer the opposite bit from bit 31 downward.

Mistakes that make you blank in the interview

HashSet of every prefix when a trie was the point. A set is fine for one problem; still build the node version.

Word Search II without pruning the trie, so you rescan dead branches Binary trie using characters instead of bits 0 and 1 from the high end

When this sits in the calendar

Leave this until after January interviews are underway. MCM, trie depth, KMP, Z, and the exotic graph algorithms are real, and they are the wrong use of December.

Do it in this order

1. Recognition: Many prefix queries Word search on a board with a dictionary XOR maximize: a binary trie of bits
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```

class Node {
    Node[] next = new Node[26];
    boolean word;
}
void insert(String word) {
    Node cur = root;
    for (char ch : word.toCharArray()) {
        int i = ch - 'a';
        if (cur.next[i] == null) cur.next[i] = new Node();
        cur = cur.next[i];
    }
}

```

```
}  
    cur.word = true;  
}
```

The usual miss

HashSet of every prefix when a trie was the point. A set is fine for one problem; still build the node version.

They will ask

Walk Implement Trie and say why this pattern fits.

Before you close the laptop

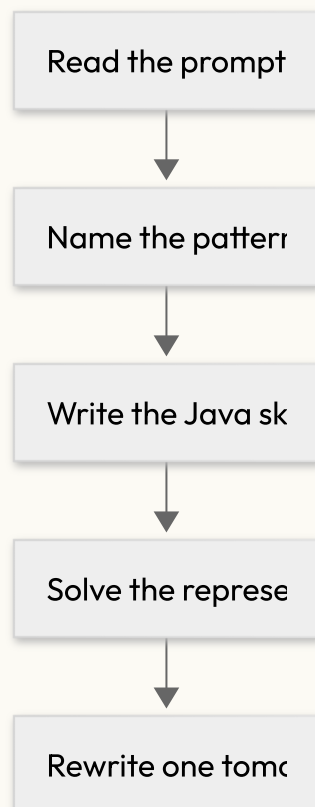
Rewrite Implement Trie tomorrow before any new pattern.

KMP, Z, Rabin-Karp

After January. LPS says how far to jump after a mismatch, because that prefix is already known to match. You never restart the text index.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Find a pattern in a text faster than checking every shift Longest prefix that is also a suffix Repeated string match, shortest palindrome Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

LPS says how far to jump after a mismatch, because that prefix is already known to match. You never restart the text index.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
int[] lps(String pattern) {
    int[] pi = new int[pattern.length()];
    int len = 0;
    for (int i = 1; i < pattern.length(); i++) {
        if (pattern.charAt(i) == pattern.charAt(len)) pi[i++] = ++len;
        else if (len > 0) len = pi[len - 1];
        else pi[i++] = 0;
    }
    return pi;
}
```

Problems that count

Find the Index of the First Occurrence (Easy). Write KMP, not indexOf. The LPS array is the skill.

Shortest Palindrome (Hard). KMP on $s + \# + \text{reverse}(s)$. The last LPS value is the palindromic prefix.

Repeated String Match (Medium). Rabin-Karp or KMP. You need at most $\text{len}(b)/\text{len}(a) + 2$ copies.

Mistakes that make you blank in the interview

Incrementing i on a mismatch when $\text{len} > 0$, which skips a character Learning Z, KMP, and Rabin-Karp as three lifestyles. LPS plus one use is enough for interviews. Rabin-Karp without a double hash or a verify step, so a collision becomes a wrong match

When this sits in the calendar

Leave this until after January interviews are underway. MCM, trie depth, KMP, Z, and the exotic graph algorithms are real, and they are the wrong use of December.

Do it in this order

1. Recognition: Find a pattern in a text faster than checking every shift Longest prefix that is also a suffix
Repeated string match, shortest palindrome
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
int[] lps(String pattern) {
    int[] pi = new int[pattern.length()];
    int len = 0;
    for (int i = 1; i < pattern.length(); i++) {
        if (pattern.charAt(i) == pattern.charAt(len)) pi[i++] = ++len;
        else if (len > 0) len = pi[len - 1];
        else pi[i++] = 0;
    }
}
```

```
return pi;  
}
```

The usual miss

Incrementing i on a mismatch when $len > 0$,
which skips a character

They will ask

Walk Find the Index of the First Occurrence and
say why this pattern fits.

Before you close the laptop

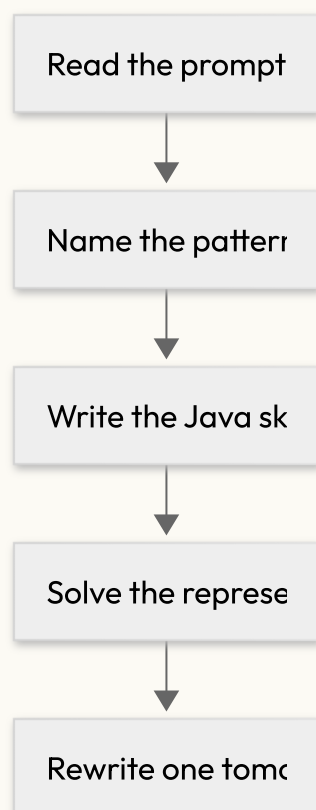
Rewrite Find the Index of the First Occurrence tomorrow before any new pattern.

Sieve and factors

After January. Cross off multiples. Start marking from $p \cdot p$, step by p . Smallest prime factor lets you factor in log time afterwards.

1. Name it before coding
2. Write the skeleton from memory
3. Solve the listed problems
4. Blank rewrite the next day

Figure



How to recognize it

Many primality checks, not one Count primes under n , smallest prime factor, factorization of every number in a range Two of these in the prompt is enough. Say the pattern name out loud, then open the editor.

The idea

Cross off multiples. Start marking from $p \cdot p$, step by p . Smallest prime factor lets you factor in log time afterwards.

Java template

Type this from memory. Only the condition inside the loop changes between problems in this family.

```
boolean[] composite = new boolean[n];
for (int p = 2; p * p < n; p++) {
```

```
if (composite[p]) continue;
for (int m = p * p; m < n; m += p) composite[m] = true;
}
```

Problems that count

Count Primes (Medium). Sieve of Eratosthenes.

Smallest Prime Factor practice (Easy). Not one famous prompt. Build `spf[1..n]` and factor 12 numbers by hand with it.

Factorial Trailing Zeroes (Medium). Count factors of 5. This is the math question that actually shows up.

Mistakes that make you blank in the interview

Trial dividing every number up to n when one sieve would do $p * p$ overflowing int. Cast to long. Spending a week on math. Two problems, then back to DP or the project.

When this sits in the calendar

Leave this until after January interviews are underway. MCM, trie depth, KMP, Z, and the exotic graph algorithms are real, and they are the wrong use of December.

Do it in this order

1. Recognition: Many primality checks, not one Count primes under n , smallest prime factor, factorization of every number in a range
2. If you cannot name it in a minute, this is pass 1: read one solution, then code it yourself.
3. Pass 2 is the next day, empty file, no video. Pass 3 is day 3, pass 4 is day 7, pass 5 is day 15 to 30.
4. Mark the attempt on the Patterns tab. Green means the rewrite worked alone.

Worked example

```
boolean[] composite = new boolean[n];
for (int p = 2; p * p < n; p++) {
    if (composite[p]) continue;
    for (int m = p * p; m < n; m += p) composite[m] = true;
}
```

The usual miss

Trial dividing every number up to n when one sieve would do

They will ask

Walk Count Primes and say why this pattern fits.

Before you close the laptop

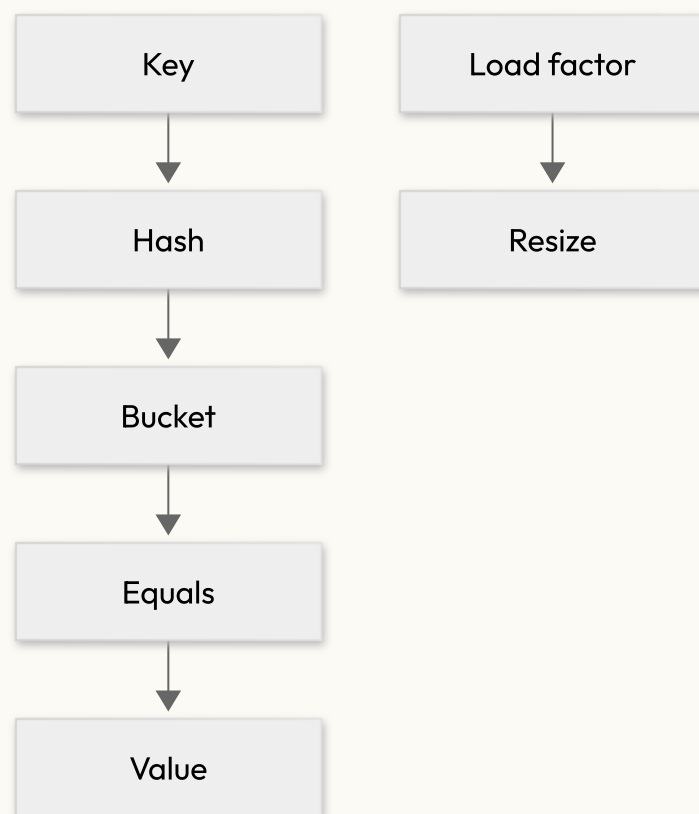
Rewrite Count Primes tomorrow before any new pattern.

HashMap internals

Interviewers use HashMap to see if you know the structure you call every day. Buckets, hash spreading, equals versus hashCode, and resize are the whole answer.

1. Compute hash
2. Pick a bucket
3. Walk equals
4. Resize when full

Figure



What you say

A HashMap is an array of buckets. The key's hash selects the bucket. Inside the bucket, equals finds the entry. A bad hashCode puts everything in one bucket and the map becomes a list. When the load grows, the table resizes and rehashes. ConcurrentHashMap does not lock the whole map for a read. A plain HashMap is not safe for two writers.

```
public final class Key {  
    private final String id;  
    public boolean equals(Object other) {  
        return other instanceof Key key && id.equals(key.id);  
    }  
    public int hashCode() { return id.hashCode(); }  
}
```

equals and the bucket

Equal keys must share a hashCode, or the lookup looks in the wrong bucket and returns null. Mutable keys move buckets after insertion. Use an immutable key. TreeMap is ordered and logarithmic. HashMap is expected constant time and unordered. Say which one you picked and the cost.

Do it in this order

1. hashCode chooses the bucket. equals decides if the key is the one you want.
2. If you override equals you override hashCode. Otherwise a HashSet silently keeps duplicates.
3. Java 8 turns a long collision chain into a tree. You should know that the chain is not always a linked list.

Worked example

```
class Key {  
    final String id;  
    @Override public int hashCode() { return id.hashCode(); }  
    @Override public boolean equals(Object o) {  
        return o instanceof Key k && id.equals(k.id);  
    }  
}
```

The usual miss

Using the array index as identity and skipping equals.

They will ask

What breaks if hashCode changes while the key sits in the map?

Before you close the laptop

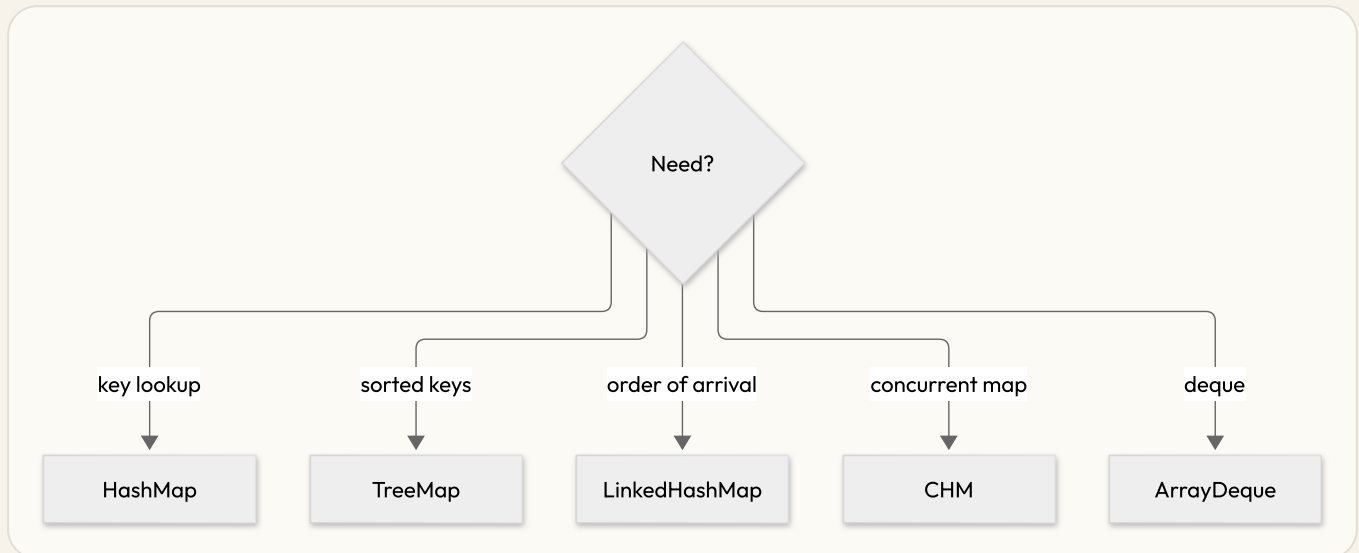
Break a HashSet on purpose by deleting hashCode. Put the class back.

Collections you must choose out loud

The question is rarely the syntax. It is why this collection, and what it costs.

1. Name the operation
2. Name the cost
3. Reject `java.util.Stack`
4. `ArrayDeque` for stack and queue

Figure



The choice

`ArrayList` for indexed reads. `LinkedList` almost never in interview code. `HashSet` for membership. `TreeSet` when you need order. `ArrayDeque` as a stack or queue. `PriorityQueue` as a heap. `ConcurrentHashMap` when two threads share the map. Name the operation you do most, then the structure.

Do it in this order

1. `HashMap` is expected $O(1)$ and unordered. `TreeMap` is $O(\log n)$ and sorted.
2. `ConcurrentHashMap` allows concurrent readers and segmented updates. It is not a lock around the whole map.
3. Optional is for a return value that might be absent. It is not a field type on an entity.

Worked example

Stack is a legacy synchronized `Vector`. Say `ArrayDeque`.

The usual miss

Picking a List and scanning it when a map was the point.

They will ask

When would you refuse a `HashMap`?

Before you close the laptop

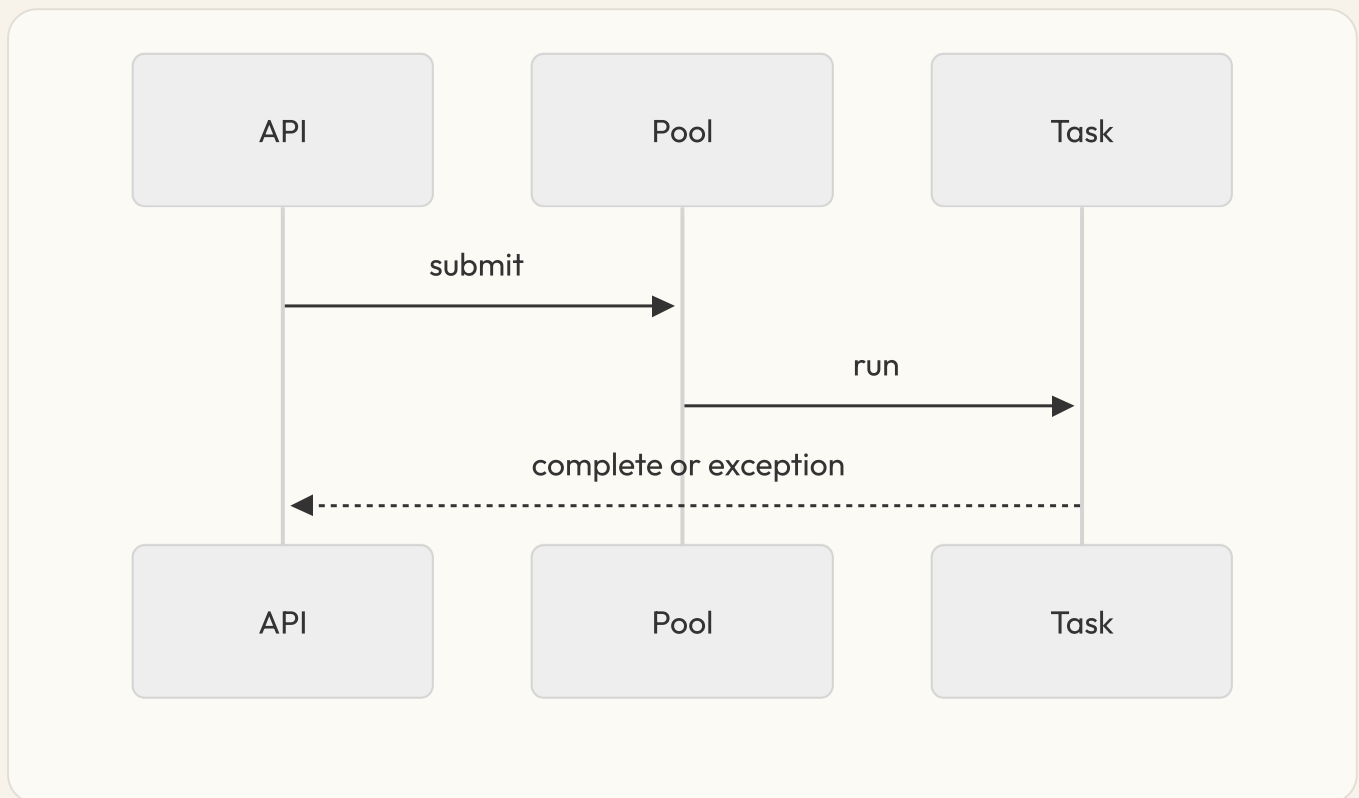
Write three method signatures and name the collection in the margin before any code.

Executors, futures, and CompletableFuture

Your resume already mentions asynchronous work. The interview will ask what runs where, what happens on failure, and how you avoid blocking the common pool.

1. Do not new Thread in a request
2. Name the pool
3. Handle the exception
4. Do not block on the common pool

Figure



Where the work runs

A thread pool has a bound. An unbounded queue in front of a pool will eat the heap.

`CompletableFuture.supplyAsync()` uses the common pool unless you pass an executor. Do not block that pool on JDBC. Compose with `thenCompose`, handle the exception with `exceptionally`, and time out. A Future you never cancel is a leak of attention. Your Dell async work is this paragraph with a real pool size and a real failure.

```
CompletableFuture
    .supplyAsync(() -> load(bucket), ioPool)
    .orTimeout(2, TimeUnit.SECONDS)
    .exceptionally(error -> fallback(bucket));
```

Do it in this order

1. `ExecutorService` owns the threads. Future is the handle. `CompletableFuture` composes them.

2. `supplyAsync` on the common pool is fine for CPU work that is short. Blocking IO needs your own pool.
3. `exceptionally` or `handle`, or the exception disappears until someone joins.

Worked example

```
CompletableFuture<String> body = CompletableFuture
    .supplyAsync(this::readLogs, ioPool)
    .exceptionally(error -> "logs unavailable");
```

The usual miss

Calling `get()` on the thread that is supposed to stay free.

They will ask

What happens if the callable throws?

Before you close the laptop

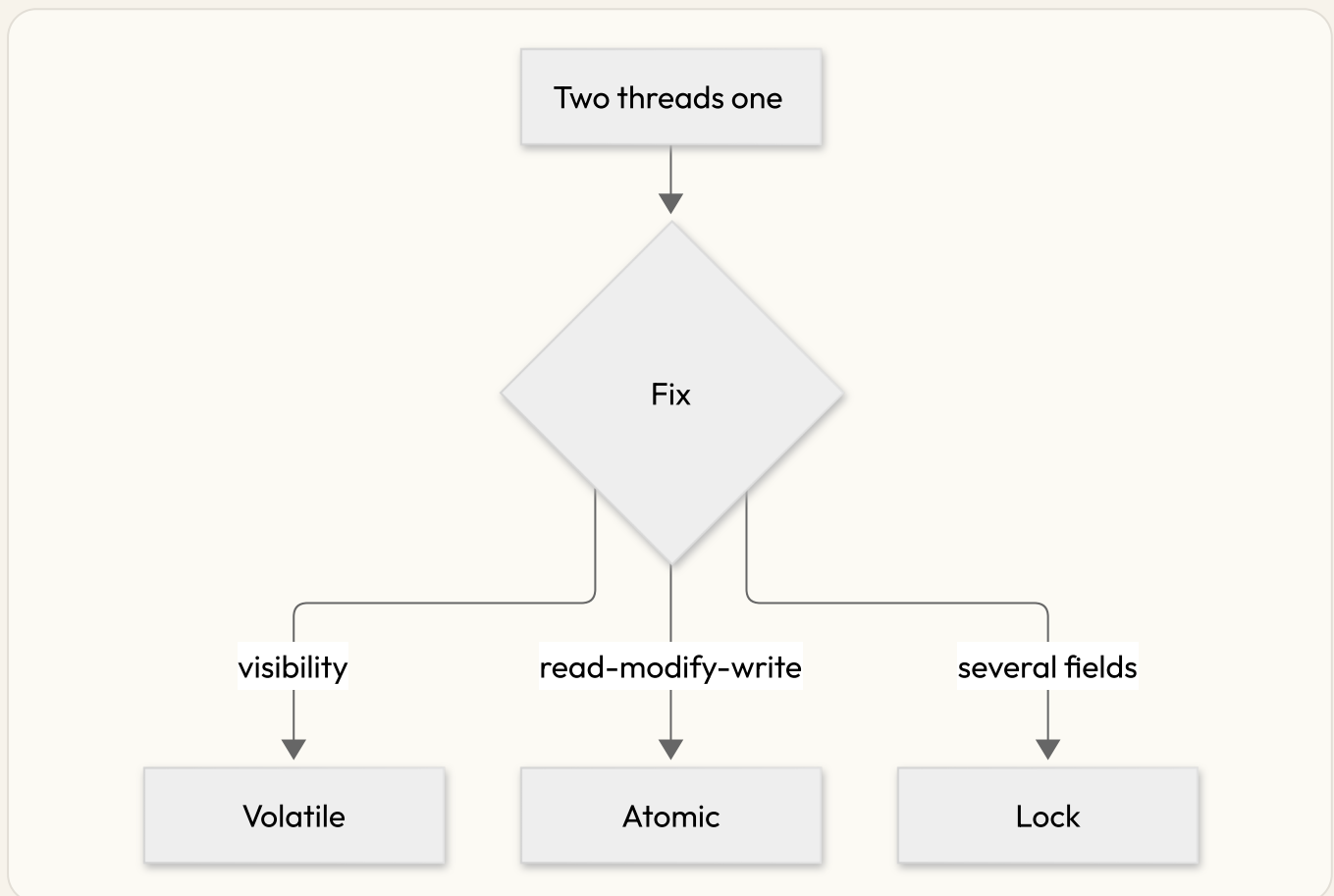
Draw the pool you would use for log collection versus the pool for a CPU score.

volatile, locks, atomics, deadlock

You need the vocabulary of visibility and exclusion. You do not need to recite the Java Memory Model chapter.

1. volatile is visibility
2. Atomic is a single variable update
3. Lock guards a block
4. Deadlock is lock order

Figure



Visibility and exclusion

volatile makes a write visible to a later read. It does not make a check-then-act atomic. synchronized or a Lock gives exclusion. AtomicInteger gives an atomic increment. A deadlock needs two locks and two orders. Take locks in one global order, or do not take two. Say the race in a sentence: two threads read the tokens, both decrement, the bucket goes negative.

Do it in this order

1. volatile does not make count++ safe. The increment is three steps.
2. synchronized or ReentrantLock gives exclusion. Always take locks in the same order.
3. A deadlock story: thread A holds lock 1 and wants lock 2, thread B holds lock 2 and wants lock 1.

Worked example

AtomicLong for a counter. A lock when two fields must change together.

The usual miss

Sprinkling volatile on a compound action and calling it thread-safe.

They will ask

Give a race that volatile does not fix.

Before you close the laptop

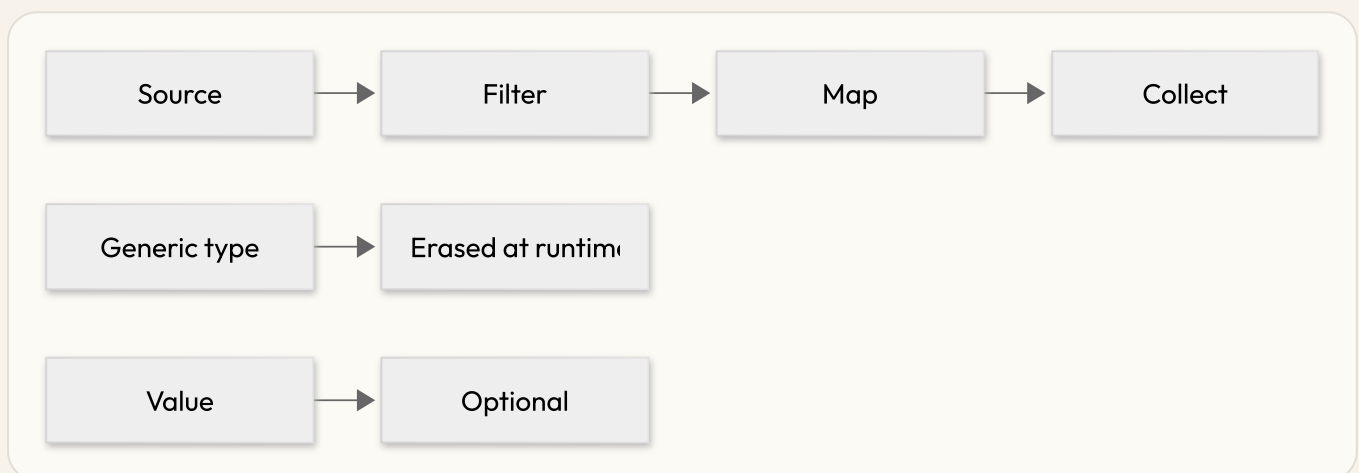
Write a 6-line deadlock on paper, then the lock-order fix.

Streams, generics, Optional, immutability

This is the Java you use and then fail to explain. The questions are small and they sort people who only remember syntax.

1. Streams are pipelines, not loops with style
2. Type erasure is real
3. Optional is a return type
4. A record is the immutable carrier

Figure



Streams

Intermediate operations build a pipeline. The terminal operation pulls values. `forEach` is a terminal that gives you nothing back. `Collect` or `toList` is the one you want. Parallel streams on a small list, or on a stream that touches JDBC, are a performance bug. Say: I keep streams sequential unless I have measured a CPU-bound pure function.

Generics and Optional

`List<String>` is `List` at runtime. That is why a raw list can be polluted. PECS: producer extends, consumer super. `Optional` expresses a missing return. `orElse` throws away a costly default because it evaluates the argument. `orElseGet` takes a supplier. Empty optionals in a field hide the real model. Use a nullable only at the edge, and `Optional` as the return.

equals, hashCode, immutability

If `equals` says two keys are the same, `hashCode` must match, or `HashMap` loses the entry. Mutable keys change the bucket after insertion. Prefer a record for a key or a request. Defensive copy a list you store. Immutability is how you stop a caller from changing the event you already published.

Do it in this order

1. A stream does not run until a terminal operation. `map` and `filter` are lazy. A stream is used once.
2. Generics erase to `Object`. You cannot `new T()` or `check instanceof T`. A wildcard `?` extends is for reading, `?` super is for writing.

3. Optional.ofNullable at a boundary. Do not pass Optional into fields and collections.

4. equals and hashCode stay a pair. A record gives you both, and an immutable value.

Worked example

```
List<String> names = items.stream()
    .map(Item::name)
    .filter(name -> !name.isBlank())
    .toList();
```

The usual miss

Using Optional.get() because the IDE suggested it.

They will ask

Why can two objects be equal and land in different HashMap buckets?

Before you close the laptop

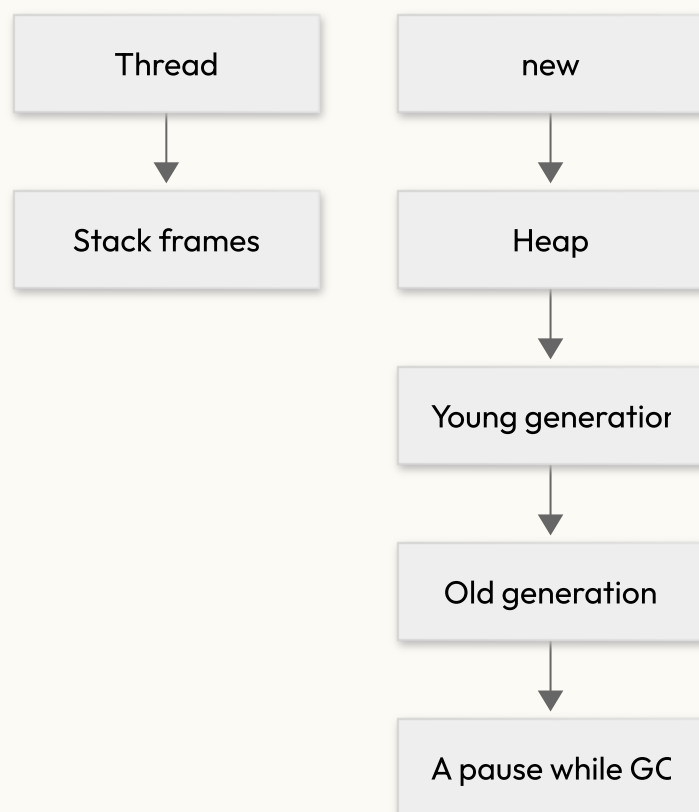
Rewrite one loop in the project as a stream, then say what the terminal operation is.

JVM, stack, heap, and GC

You need a clear story, not a collector catalog. Interviewers ask where an object lives and what a pause is.

1. Stack is the frame
2. Heap is the objects
3. GC reclaims unreachable objects
4. A pause is latency

Figure



The story in four sentences

Each thread has a stack. new allocates on the heap. When no thread can reach an object, GC can reclaim it. A stop-the-world pause freezes application threads so the collector can move or mark memory, and that pause shows up as latency. Modern collectors shorten the pause. They do not make an unbounded cache correct.

What you refuse to pretend

You do not tune G1 versus ZGC from memory in an interview. You say what you would measure: allocation rate, pause time, and the dominator in a heap dump. For this project the bounded structures are the rate-limit map, the agent session in Redis, and the Kafka consumer's in-flight count.

Do it in this order

1. Local variables and calls live on the stack. Objects and arrays live on the heap.
2. A leak in a server is usually a collection that grows: a static map, a listener, an unbounded queue.
3. Generational GC collects young garbage often. A long-lived cache belongs in a bounded structure with eviction, not in the hope that GC will save you.
4. You look at a heap dump or at least at live set and pause time before you change flags.

Worked example

```
OutOfMemoryError: Java heap space
First question: which collection grew?
Second: is the cache bounded?
```

The usual miss

Adding -Xmx until the box dies, without finding the collection.

They will ask

Where does a HashMap entry live, and what makes it unreachable?

Before you close the laptop

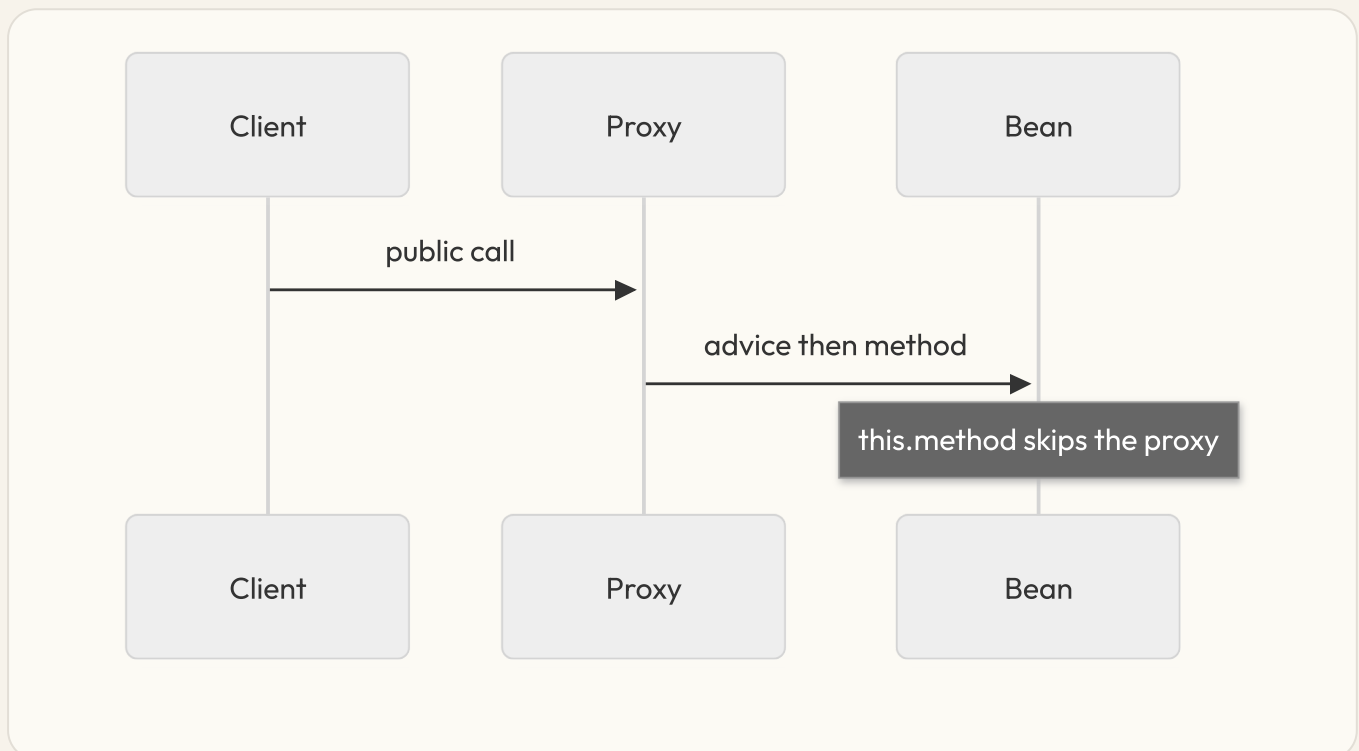
Name one structure in the project that must have a max size.

IoC, beans, and the proxy

Spring creates the objects and injects them. Transactions, security, and async work through proxies. If you call a proxied method from inside the same class, the proxy is skipped.

1. Container builds the bean
2. Proxy wraps cross-cutting work
3. Self-invocation skips it
4. Constructor injection is the default

Figure



The proxy

Spring builds the objects and injects them. `@Transactional` works because a proxy wraps the bean. A call from inside the same class does not pass the proxy, so the transaction annotation is ignored. The bean lifecycle is construction, dependency injection, init, use, destroy. Auto-configuration creates beans when a class is on the classpath and you did not define your own.

Do it in this order

1. Prefer constructor injection. The bean is immutable and tests can pass fakes.
2. Bean lifecycle in one breath: construct, inject, aware callbacks, init, use, destroy.
3. Auto-configuration is conditional beans. You override them by declaring your own.

Worked example

```
@Transactional on a private method or a self-call does nothing useful.
```

The usual miss

Field injection because it is fewer lines.

They will ask

Why did this @Transactional not roll back?

Before you close the laptop

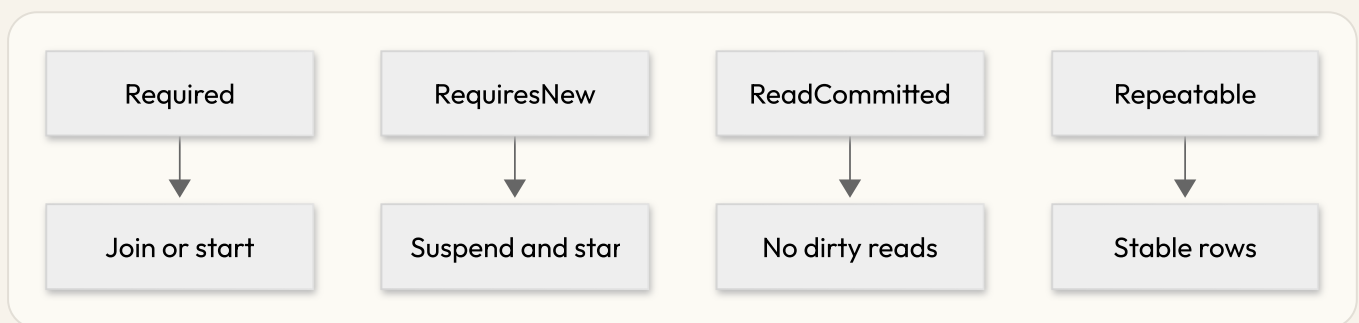
Point at one service in your Dell work and say which calls go through the proxy.

Transactions, propagation, isolation

This is SDE II territory. You should say what is atomic, what isolation you need, and what a nested call does.

1. Name the boundary
2. Name propagation
3. Name isolation
4. Say what rolls back

Figure



Propagation and isolation

REQUIRED joins the current transaction or starts one. REQUIRES_NEW suspends the current one. A failure in the inner new transaction does not, by itself, roll back the outer one. Default isolation on Postgres is read committed. A lost update needs a version or a lock. Rollback happens on a runtime exception. A checked exception needs rollbackFor. Say the boundary: the business row and the outbox row share one transaction.

Do it in this order

1. REQUIRED joins the current transaction. REQUIRES_NEW commits the inner work even if the outer rolls back. That is how you persist an audit row.
2. Default rollback is runtime exceptions. Checked exceptions need rollbackFor.
3. Isolation: read committed is the usual Postgres setting. Repeatable read stops a row changing under you. Phantom rows are a different problem.

Worked example

The outbox insert and the order insert share one transaction. The Kafka send does not.

The usual miss

A long transaction that calls an external API.

They will ask

When do you want REQUIRES_NEW?

Before you close the laptop

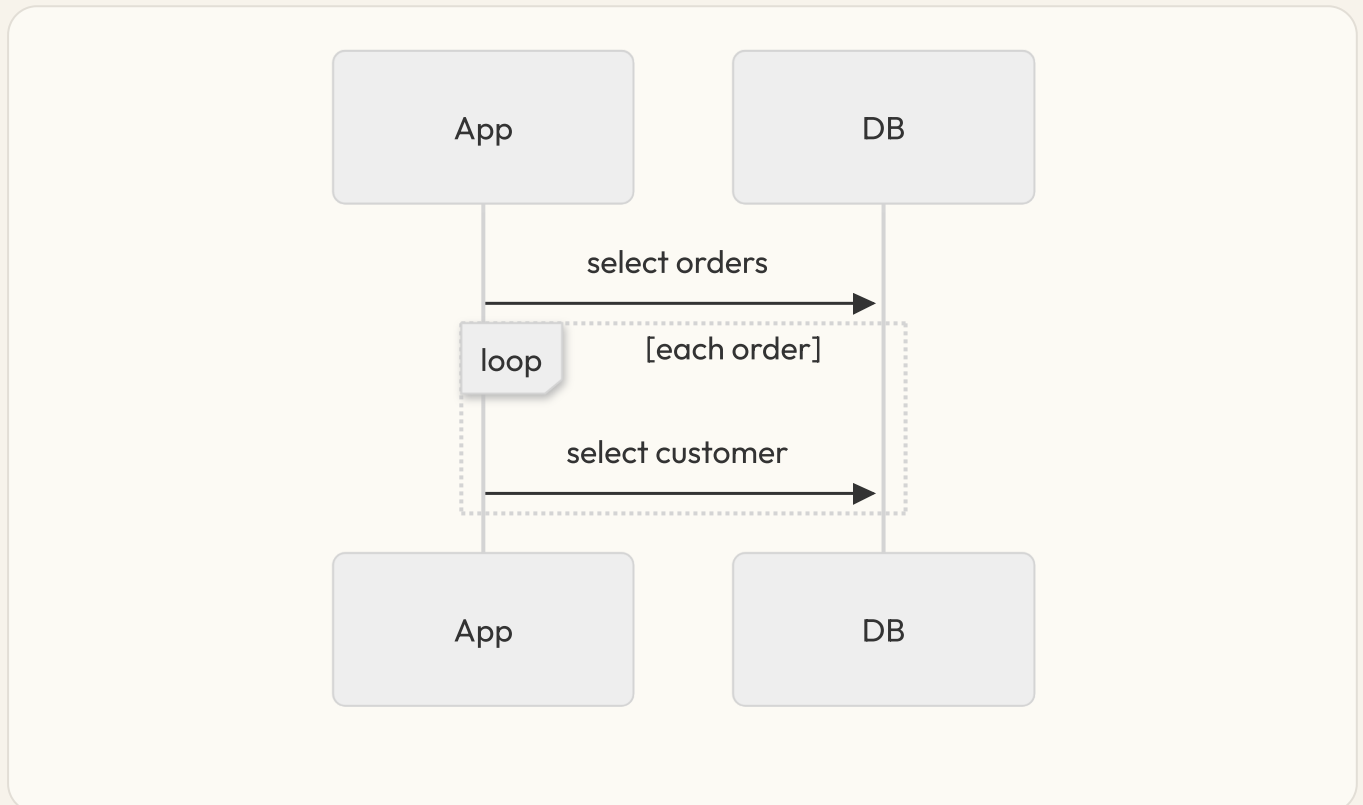
Write the two methods for order plus outbox and mark which transaction they share.

JPA, N+1, locking

Hibernate will happily issue a query per row. You should notice it before production does.

1. One query becomes N
2. Fetch join or batch
3. Lazy is a loaded graph
4. Version column for optimistic lock

Figure



The queries you did not write

A lazy list inside a loop issues one select per parent. That is N+1. Fix it with a fetch join or a batch. Eager on everything loads the graph. Optimistic locking throws on a version mismatch. Pessimistic locking holds the row. Open-session-in-view hides the N+1 until serialization. Turn SQL logging on for the test and count the selects.

Do it in this order

1. N+1 shows up when you touch a lazy association in a loop.
2. Optimistic lock: a version column, conflict becomes an exception, the client retries. Pessimistic lock holds a database lock. Use it rarely.
3. Eager everything is not the fix. You load the graph you need for this request.

Worked example

@Version on the aggregate that two agents might update.

The usual miss

Open-session-in-view hiding the N+1 until a loader disappears.

They will ask

How do you detect N+1 locally?

Before you close the laptop

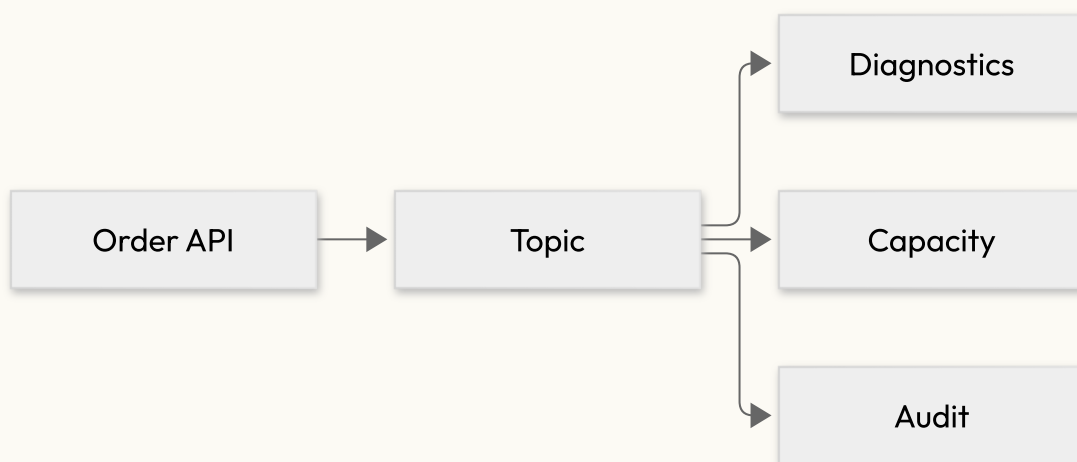
Turn on SQL logging for one repository test and count the selects.

Why Kafka, not another REST call

REST couples the caller to the callee being up. A log of events lets the diagnostics agent, the capacity agent, and the audit trail each consume at their own speed.

1. Producer commits the fact
2. Consumers are independent
3. Replay is possible
4. REST was the wrong coupling

Figure



The decision

Use Kafka when several consumers need the same fact, when a slow consumer must not stop the writer, or when you may replay. Use REST when the caller needs the answer in this request. Ordering is per key. The key is bucket id if one bucket's events must stay ordered. Retention lets the RCA agent read yesterday. A topic is not a command bus for a synchronous user click.

Do it in this order

1. Use Kafka when more than one downstream cares, when you need a buffer, or when you must replay.
2. Do not use Kafka for a request that needs the answer in the same HTTP call.
3. Ordering is per partition key. Choose the key on purpose, such as bucket id.

Worked example

Key = bucketId so one bucket's events stay ordered.

The usual miss

A topic with a random key and a later complaint that events arrived out of order.

They will ask

Why not just call the agent over HTTP?

Before you close the laptop

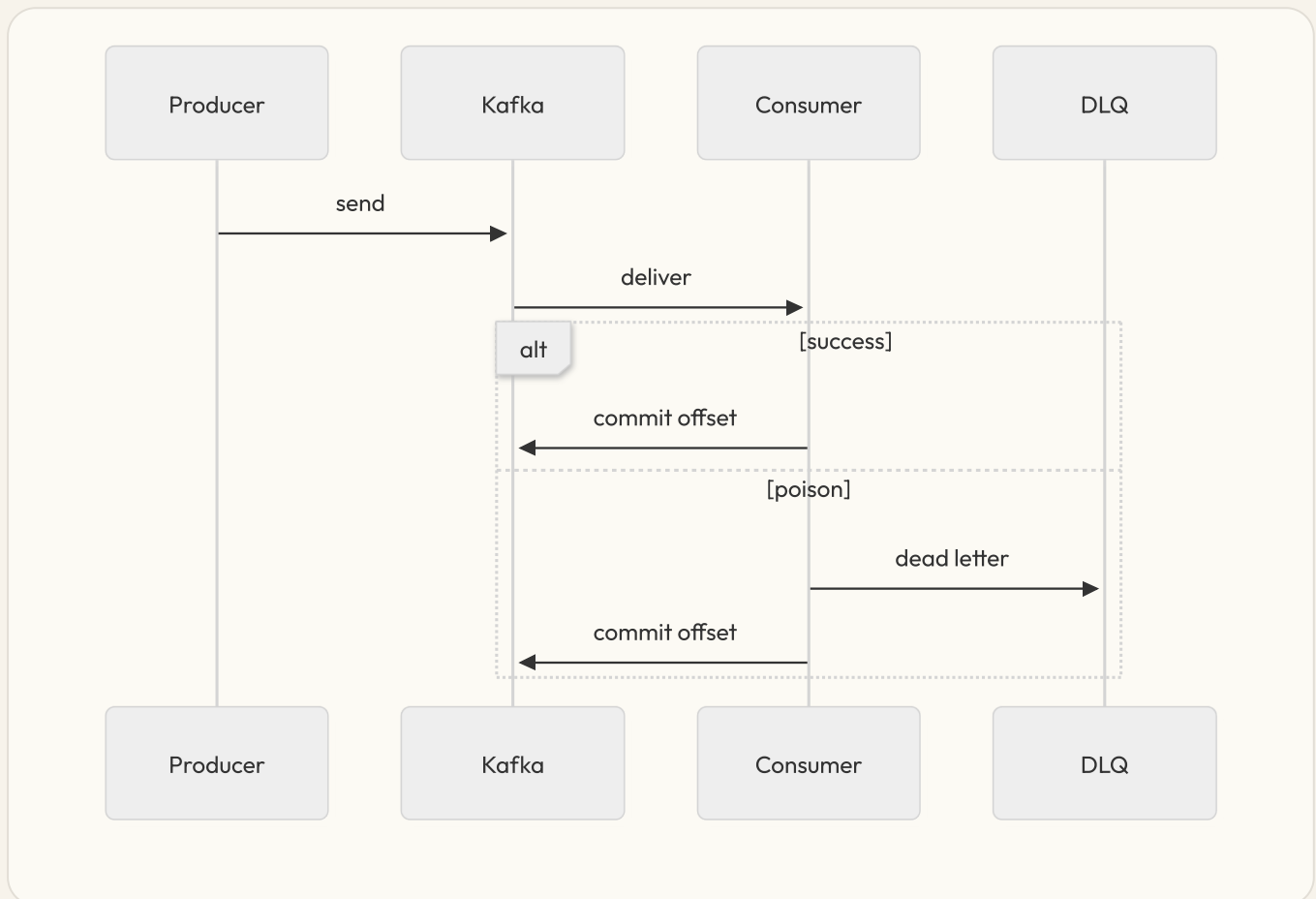
Name one Dell flow that is a fact other systems observe, and one that must stay a request.

Consumer crash, duplicates, retries, DLQ

This is the senior part of your Kafka story. Delivery, crashes, and duplicates are one design.

1. At least once is the default
2. Crash before commit redelivers
3. Handler is idempotent
4. Poison goes to DLQ

Figure



The crash

The consumer writes the database, then crashes before commit. The message is delivered again. The handler sees the idempotency key and skips. That is at-least-once. Committing before the work loses the message. A poison payload goes to a dead-letter topic after a small number of tries. A down dependency uses backoff, not an infinite loop. Offset order is the order you commit, so do not commit a later offset and skip a failed earlier one unless you have recorded the skip.

Do it in this order

1. If the process dies after the side effect and before the offset commit, the message comes back. Your handler must tolerate that.
2. Idempotency key on the event. The second delivery finds the row and returns.
3. Retries with backoff for a down dependency. A DLQ when the payload itself is bad. Do not retry forever.

Worked example

Store processed event ids. On duplicate, ack and skip.

The usual miss

Committing the offset before the work, then losing the crash window's events.

They will ask

The consumer dies after writing the database and before commit. What do you do?

Before you close the laptop

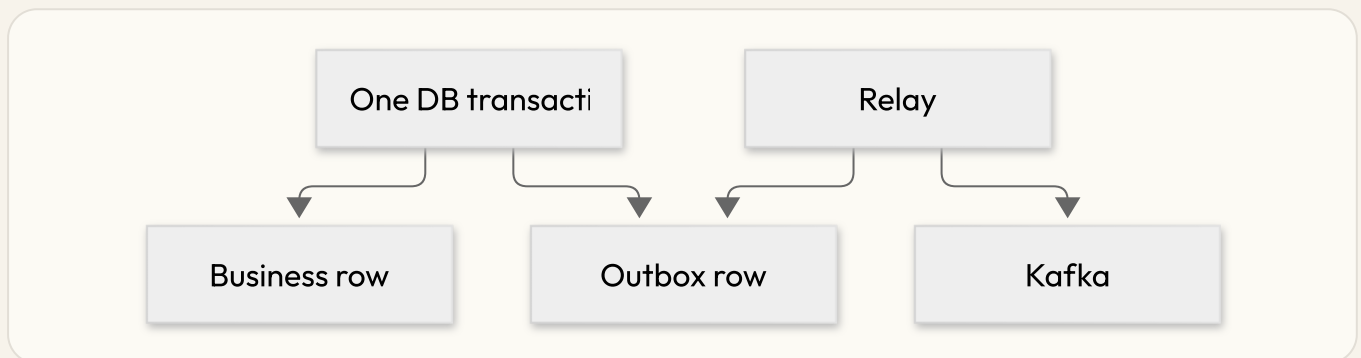
Add an idempotency key to the project event on paper. Three lines.

Outbox

You cannot commit a database row and a Kafka send as one transaction without a pattern. The outbox is that pattern.

1. Same transaction
2. Relay reads the outbox
3. Mark sent
4. Kafka is outside the transaction

Figure



One transaction

The service inserts the business row and an outbox row in one database transaction. A relay reads unpublished outbox rows and sends them to Kafka, then marks them sent. If the process dies after commit and before send, the row is still there and the relay retries. Kafka can still duplicate, so the consumer stays idempotent. This is the project milestone for 25 October.

Do it in this order

1. Insert the business row and the outbox row together.
2. A worker publishes and then marks the outbox sent. If it crashes, it publishes again. Consumers stay idempotent.
3. This is the story that connects your order API and your agents.

Worked example

```
outbox(event_id, type, payload, sent_at)
```

The usual miss

Publishing to Kafka inside the transaction and hoping both commit.

They will ask

What is atomic in the outbox pattern, and what is not?

Before you close the laptop

Sketch the table and the worker loop in the flagship README.

Redis as cache, lock, and agent memory

Redis is the fast state: rate-limit counters, a cache in front of bucket metadata, and a short-lived agent session. It is not your source of truth.

1. Cache has a TTL
2. Source of truth stays in Postgres
3. Stampede is a real bug
4. Session state expires

Figure



Three uses, one rule

Cache: bucket metadata with a TTL, cache-aside, delete on write. Rate limit: a counter or a token field with a TTL. Agent session: a short-lived JSON blob keyed by the conversation, not the system of record. Eviction can drop any of these. Postgres still has the bucket. A lock in Redis needs a token and an expiry so a dead owner does not hold it forever. Persistence is optional. Do not describe Redis as the ledger.

Do it in this order

1. Cache aside: read cache, on miss read the database, fill the cache.
2. Set a TTL. Invalidate on write when stale data would be wrong, such as a quota.
3. A single-flight lock or a short lease stops a hundred misses from hammering Postgres.

Worked example

```
SET bucket:42 meta EX 30
```

The usual miss

Caching a permission decision with no TTL.

They will ask

What is wrong if Redis is empty after a restart?

Before you close the laptop

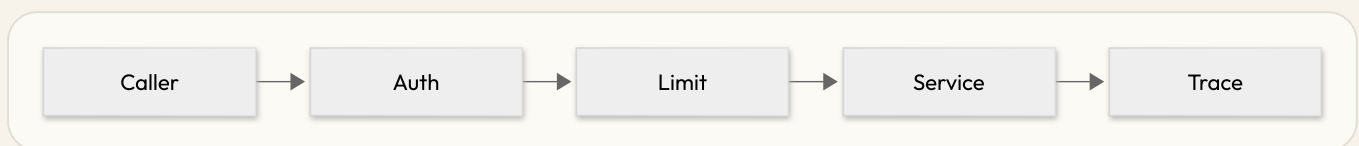
Name three keys the flagship project would store, and the TTL for each.

Security, resilience, actuator

JWT and OAuth answer who is calling. Resilience answers what you do when the next service is sick. Actuator and traces answer what happened.

1. Authenticate then authorize
2. Timeout every remote call
3. Retry only idempotent calls
4. A trace id crosses the agent

Figure



Who calls, and what you do when the next hop is sick

JWT is a signed claim. Validate the signature, the expiry, and the audience. OAuth is how the token was issued. The resource server checks the token. It does not ask the user for a password. Resilience: a timeout, a small retry on idempotent calls, a circuit breaker when the dependency is failing fast, and a fallback you can explain. Actuator opens health and metrics. The correlation id ties the log to the trace.

Do it in this order

1. Spring Security: filter authenticates, method or route rules authorize.
2. Resilience4j or a simple policy: timeout, small retry with jitter, circuit breaker when the dependency is down.
3. Micrometer plus actuator. A correlation id from the HTTP request lands in the Kafka header and the agent log.

Worked example

Do not retry a payment POST unless the idempotency key makes the retry safe.

The usual miss

Logging the full prompt and the customer's object listing.

They will ask

Which calls are safe to retry?

Before you close the laptop

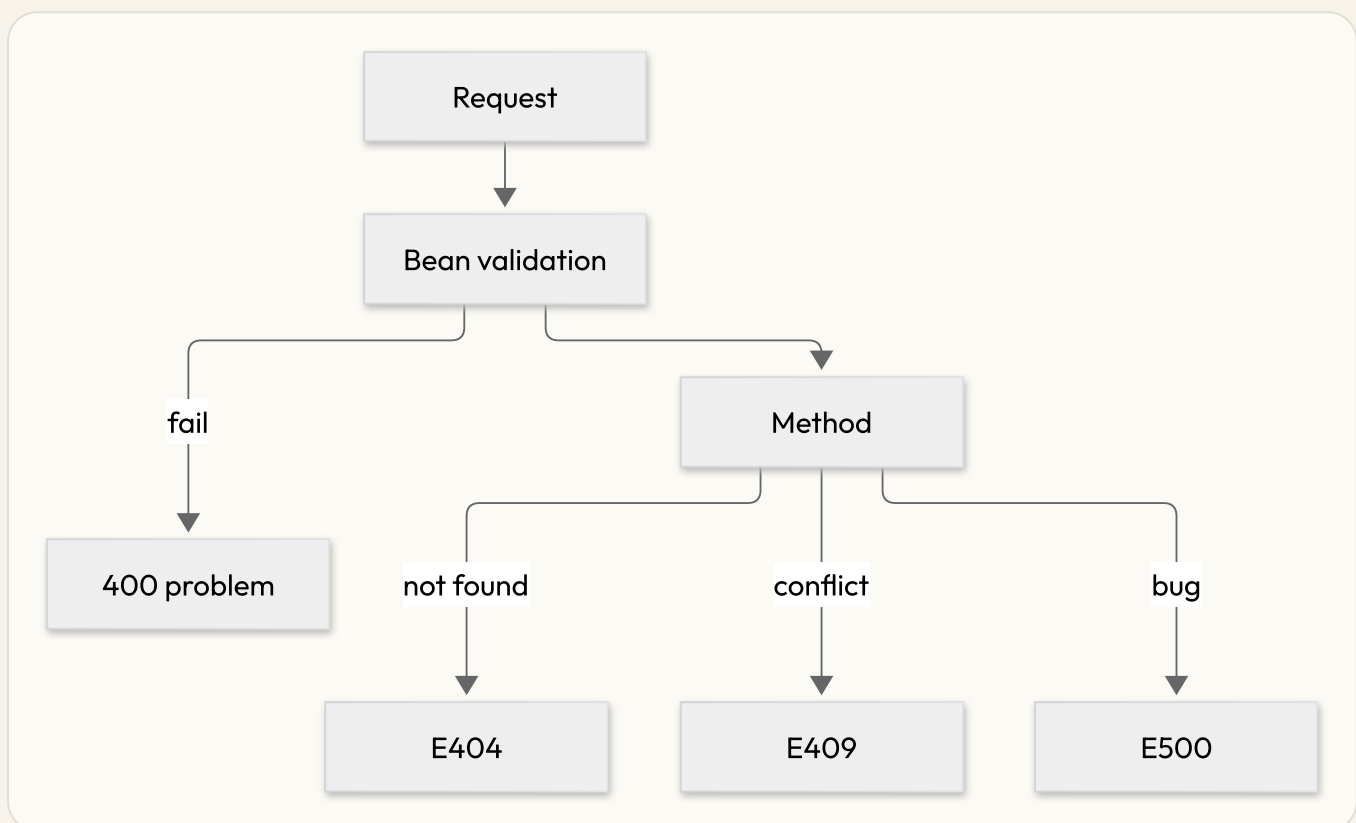
Add one correlation id to the story you will tell about DefectIQ-style RAG. No internal data.

REST, validation, and one error shape

A senior backend answer includes the status code, the body, and what the client can retry. Annotations are the implementation.

1. Validate at the boundary
2. One error JSON
3. 409 for an idempotency conflict
4. Do not leak stack traces

Figure



The contract

Resources are nouns. GET is safe. PUT replaces. POST creates or starts a command. A command that must not run twice takes an Idempotency-Key header. Validation errors are 400 with the field name.

Authentication is 401. Authorization is 403. You already know the words. The interview is whether the API you design uses them consistently.

Exceptions

The service throws a domain exception. The advice translates it. A catch that logs and returns null hides the failure from the client and from the metric. Checked exceptions in a Spring service are usually noise. A transaction rolls back on a runtime exception. Say that when they ask what happens if the handler throws.

Do it in this order

1. Controller methods take a DTO. Bean validation annotations reject a blank bucket id before the service runs.

2. A `@ControllerAdvice` maps exceptions to a problem body: code, message, request id. The client never sees a stack.
3. POST that creates is 201 with a location, or 200 on an idempotent replay. A bad id is 404. A version clash is 409.
4. Pagination is limit and cursor, not an unbounded list.

Worked example

```
{  
  "code": "bucket_not_found",  
  "message": "No bucket b1",  
  "requestId": "9f2c"  
}
```

The usual miss

Returning 200 with `{success:false}` for every failure.

They will ask

The same idempotency key arrives with a different body. What status?

Before you close the laptop

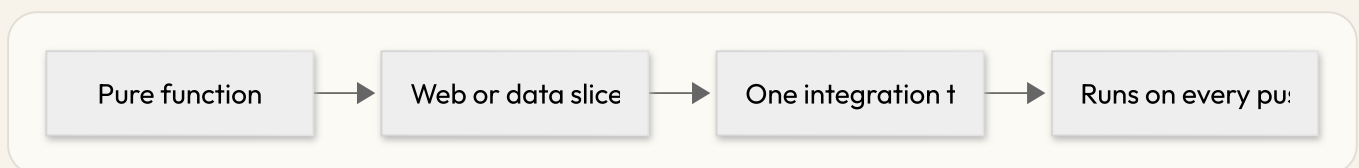
Add one `@ControllerAdvice` to the project and one test that expects 400.

Tests that protect the interview story

The tests worth writing are the ones you will mention: idempotency, the outbox commit, and the N+1 you already fixed.

1. Test the behavior you claim
2. One database test for the outbox
3. Mock the agent, not the transaction
4. A red test before the fix

Figure



The pyramid for this project

Most tests are plain JUnit on pure code: refill math, partition-key choice, prompt assembly. A few tests use the database because the bug lives in the transaction. One test calls the agent with a fake model so the eval set runs without a bill. CI runs them. A test you cannot run on the laptop is a demo, not a test.

What interviewers listen for

They ask how you knew the fix worked. The answer is a failing test that reproduced the duplicate, then the constraint or the idempotency lookup that made it pass. That story beats a coverage percentage.

Do it in this order

1. A unit test covers the token bucket math with a fake clock.
2. A data test starts Postgres, writes the business row and the outbox row, and asserts both exist or neither does.
3. A web test sends the same Idempotency-Key twice and asserts one row.
4. Do not mock the repository and then claim you tested the transaction.

Worked example

```

@Test
void replayDoesNotCreateASecondBucket() {
    api.create(key, body);
    api.create(key, body);
    assertEquals(1, buckets.count());
}
  
```

The usual miss

A hundred controller tests that only check 200.

They will ask

Which test would fail if the outbox write left the transaction?

Before you close the laptop

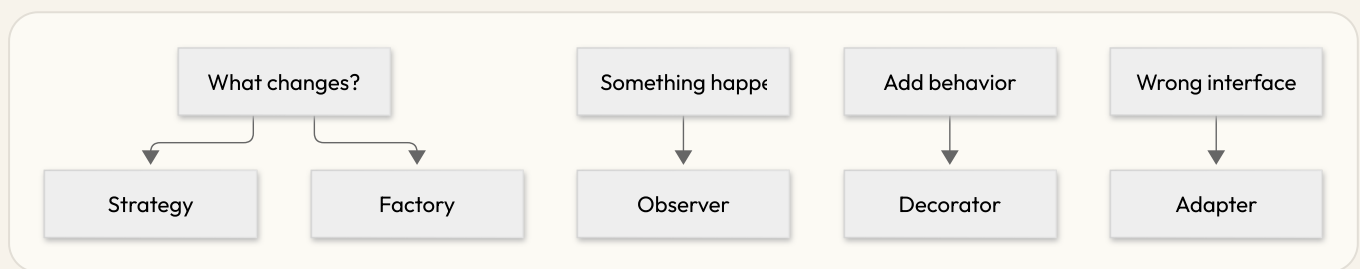
Add that one test name to the README, even before it passes.

LLD: classes, SOLID, and five patterns

HLD picks the boxes. LLD picks the classes inside one box. SDE II loops ask for both. You already use these patterns inside Spring. Name them.

1. One reason to change per class
2. Strategy when the algorithm changes
3. Factory when construction is messy
4. Observer when others must hear an event

Figure



How to run the 45 minutes

Clarify the operations, not every future feature. Parking lot: park, unpark, find a spot. Rate limiter: allow or deny for a key. Name the entities, the relationships, and the one method that can go wrong under two threads. Then write that method. Mention the lock or the atomic only where the shared state lives.

The five patterns you will actually use

Strategy: the limit algorithm is TokenBucket or SlidingWindow, chosen at construction. Factory: building a notifier for email or push without a switch in the caller. Observer: an object event notifies the audit log and the agent. Decorator: timing or retries wrap a repository. Adapter: the Python agent speaks HTTP, the Java client speaks your interface. Singleton is usually the wrong answer. Spring already scopes beans.

A shape that interviews accept

RateLimiter depends on a Clock and a Store. Store is in-memory for the interview and Redis in production. Clock is injectable so a test can move time. allow(key) reads the bucket, refills from elapsed time, and decrements. The store update is atomic. Say what happens when Redis is down: fail open or fail closed, and why you picked it.

Do it in this order

1. SOLID in one line each. Single responsibility: one reason to change. Open/closed: new behavior by new code. Liskov: a subtype can stand in. Interface segregation: small interfaces. Dependency inversion: depend on the interface the caller owns.
2. In a 45-minute LLD, spend 10 minutes on the nouns and the operations, 25 on classes and the one tricky method, 10 on concurrency or failure.
3. Prefer composition. A class that implements five interfaces is usually three classes.

Worked example

```
interface RateLimit {
    boolean allow(String client);
}
class TokenBucket implements RateLimit {
    public boolean allow(String client) { return true; }
}
class Limiter {
    private final RateLimit policy;
    Limiter(RateLimit policy) { this.policy = policy; }
}
```

The usual miss

Drawing a UML poster and never writing the method that races.

They will ask

Design a rate limiter. Which class owns the counters?

Before you close the laptop

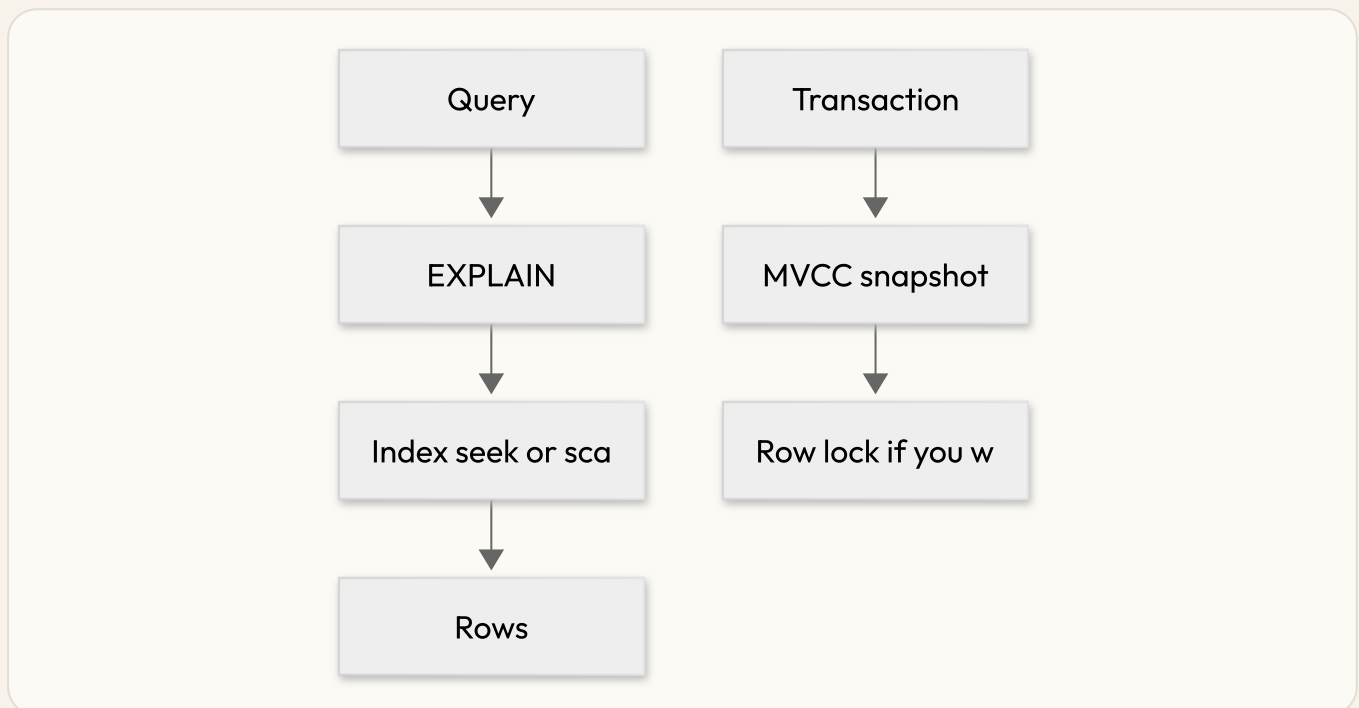
Write the four class names for a token bucket and the one method that must be atomic.

Indexes, plans, and isolation

JPA hides the database until a page gets slow. The interview asks what the database did, not which annotation you used.

1. Filter columns want an index
2. Leftmost prefix of a composite index
3. Read committed is the usual start
4. A deadlock is a lock cycle

Figure



What an index is

The heap stores rows. An index stores ordered keys plus pointers. A seek is cheap. A scan reads the table. A covering index holds every column the query needs, so the heap is not visited. Cardinality matters: an index on a boolean is rarely useful. Write the predicate first, then the index that matches its left prefix.

Transactions you must be able to say

A transaction is atomic on commit or rollback. Lost update: two transactions read a balance and both write. Fix it with a version column (optimistic) or SELECT FOR UPDATE (pessimistic). Deadlock: A locks row 1 and wants row 2, B does the opposite. The database kills one. Retry the killed transaction. Phantom: a second read sees new rows. Isolation level or a constraint decides whether you care.

How this meets JPA

Lazy associations plus a loop are the N+1. Fetch join or a batch size fixes it. An open session in the view hides the cost until the serializer runs. Optimistic lock is @Version. A failed version check is a retry, not a 500 you swallow. The SQL lesson is the reason the annotation exists.

Do it in this order

1. A B+ tree index supports equality and range. A column in a function often cannot use it.
2. Composite index (bucket_id, created_at) serves WHERE bucket_id = ? ORDER BY created_at. It does not serve a predicate on created_at alone.
3. EXPLAIN the slow repository method. Look for Seq Scan on a large table and for rows removed after the index.
4. Isolation: read committed stops dirty reads. Repeatable read keeps a snapshot. Serializable refuses anomalies. Postgres uses MVCC, so readers do not block writers.

Worked example

```
EXPLAIN SELECT * FROM object_event
WHERE bucket_id = 'b1'
ORDER BY created_at DESC
LIMIT 20;
```

The usual miss

Indexing every column and calling it tuned.

They will ask

Why is this query a sequential scan?

Before you close the laptop

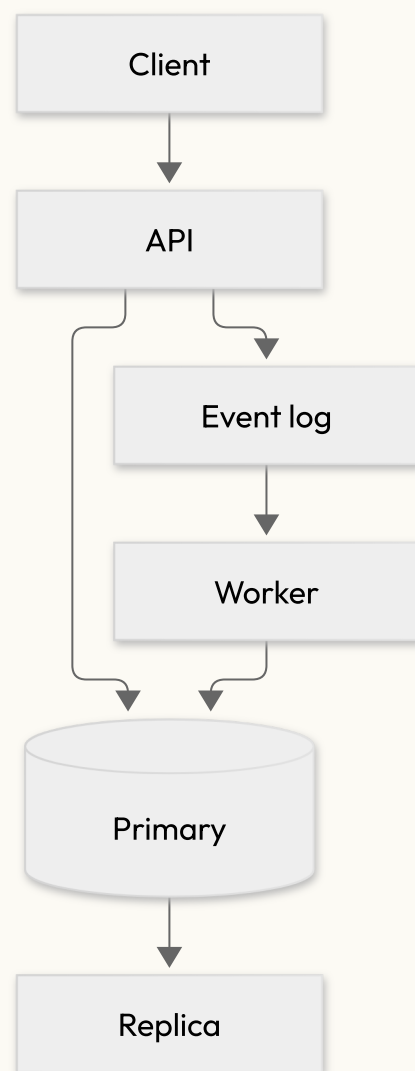
Run EXPLAIN on the hottest query in the project and write one sentence about the plan.

Replication, partitions, and delivery

Distributed systems is the reason Kafka, retries, and idempotency exist. You already run pieces of this at Dell. The interview wants the failure story.

1. A timeout is not a no
2. A retry repeats the side effect
3. Idempotency makes the retry safe
4. A partition key is an ordering choice

Figure



The failure you should narrate

The client timed out. The server may have committed. The client retries. Without a key, you have two orders. With a key stored in the same transaction as the order, the second call finds the first and returns it. That is the whole distributed-systems answer for a write API. Say the timeout value and what you do when the dependency is down: a bounded retry, then an error the caller can see.

Ordering and partitions

Kafka orders inside a partition. The key decides the partition. Bucket id keeps one bucket's events ordered. A random key spreads load and loses order. Consumers in a group split partitions. You do not get global order unless you accept one partition and its throughput ceiling. Say that trade out loud.

Clocks and leases

Machine clocks drift. A lease with a timeout is how a scheduler gives one worker a job. If the worker dies, the lease expires and another worker takes it. If the worker is slow and the lease expires while it is still working, two workers can run the job. The job handler must be idempotent too. This is the job-scheduler design in one paragraph.

Do it in this order

1. Replication: a primary takes writes, replicas serve reads or stand by. Lag means a read-your-write can miss the write. Route that read to the primary.
2. A network partition means a node cannot see another. Timeouts, retries, and a decision about availability follow. CAP is a reminder, not a design.
3. At-least-once delivery plus an idempotent handler is the practical choice. Exactly-once is a careful composition of the log and the database, which is what the outbox approaches.
4. Backpressure: if the consumer is slower than the producer, the lag grows. Scale consumers, or slow the producer. Do not let the heap be the queue.

Worked example

```
Idempotency-Key: 9f2c
First call: create the row, publish, return 201.
Second call: find the row, return 200.
No second object.
```

The usual miss

Retrying a non-idempotent POST and hoping the cloud is kind.

They will ask

The consumer crashes after the database write and before the offset commit. What happens?

Before you close the laptop

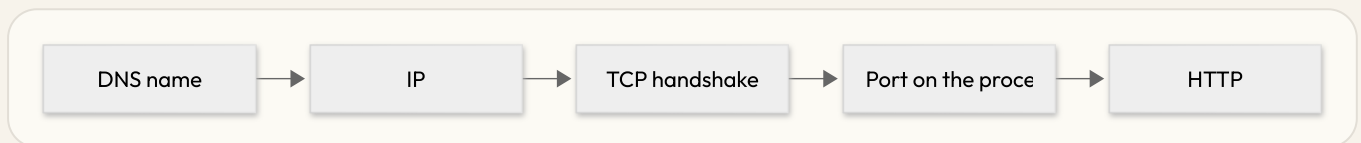
Add the idempotency key to one project endpoint and the test that calls it twice.

Processes, DNS, and TCP

Cloud and Kubernetes debugging is Linux with extra YAML. You need to tell a refused connection from a timeout, and a process from a container.

1. Connection refused: nothing listens
2. Timeout: a filter or a route dropped you
3. DNS failure: the name never became an IP
4. A container has its own network namespace

Figure



The three errors

Connection refused means the SYN reached a host and no socket accepted it. The process is down, or you used the wrong port. Timeout means packets vanished: security group, NACL, network policy, a route, or a firewall. Name or service not known means DNS. Read the error before you change a manifest. In Kubernetes, the Service's targetPort must match the container port, and the pod's labels must match the Service selector.

What you run

Inside a debug container or on the laptop: curl, dig or getent hosts, ss, and the application log. On AWS, the security group is a stateful firewall on the instance. The VPC route table must have a path. Say those two names when a cloud deploy times out. You do not need to become a kernel engineer before the switch.

Do it in this order

1. A process has a pid, file descriptors, and ports. A container is a process with namespaces. A pod is one or more containers that share a network namespace.
2. curl -v shows DNS, the TCP connect, and the HTTP status. ss -lntp shows who listens.
3. HTTP keep-alive reuses the TCP connection. A load balancer idle timeout that is shorter than the client's is a classic stuck-connection bug.

Worked example

```

curl -v http://127.0.0.1:8080/health
# refused: the process is down
# timeout: security group, network policy, or the wrong subnet
# 500: the process is up and the bug is in the app
  
```

The usual miss

They will ask

Restarting the pod before you know whether the port is open.

The service exists and the pod is Running. The client times out. Where do you look?

Before you close the laptop

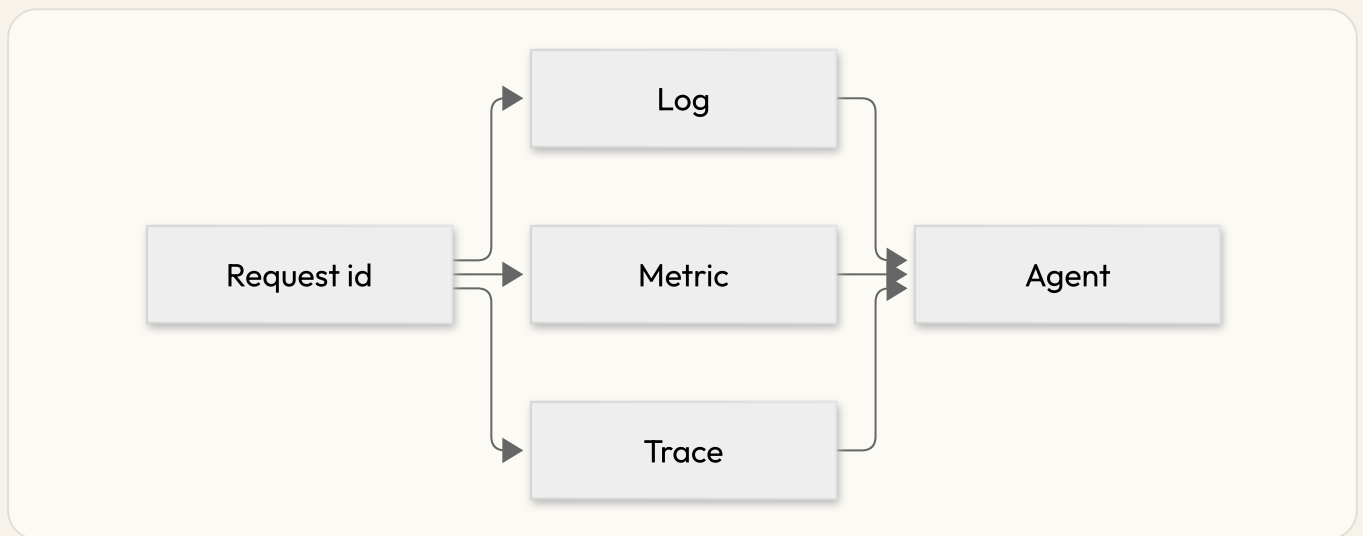
Break the project on purpose: stop Postgres and write down the exact client error.

Logs, metrics, traces

Reliability work is how you notice the bucket latency before the user writes the ticket. The diagnostics agent reads the same three signals.

1. One correlation id
2. RED: rate, errors, duration
3. A log is an event, not a novel
4. An alert needs an action

Figure



How you debug with the three

Start with the metric: is error rate or latency up, and since when? Pick one slow request id from the log. Open its trace. The widest span is the suspect. Then read the log lines for that id only. This is also the diagnostics agent's procedure. If you can do it by hand, you can tell the agent to do it and then check the citation.

What you page a human for

Page on user-visible failure and on a disk or error budget that will run out. Do not page on a single log line. The capacity agent forecasts the 80 percent mark so the page happens before the cluster is full. Actuator on Spring exposes health and metrics. CloudWatch or Prometheus scrapes them. The tool is secondary to the three signals.

Do it in this order

1. Logs: structured, with the request id and the bucket id. High-cardinality data stays out of metric labels.
2. Metrics: request rate, error rate, latency histogram. A gauge for disk used. Alert on the symptom the user feels, and on saturation before the disk fills.
3. Traces: spans across the API, the database, and the agent tool call. The trace is how you prove the latency is in the database and not in the model.

Worked example

```
requestId=9f2c bucketId=b1 latencyMs=840 dbMs=790
```

The story is the database, not the model.

The usual miss

Logging the entire object on every request and calling it observability.

They will ask

Latency is high. Which signal do you open first, and what would change your mind?

Before you close the laptop

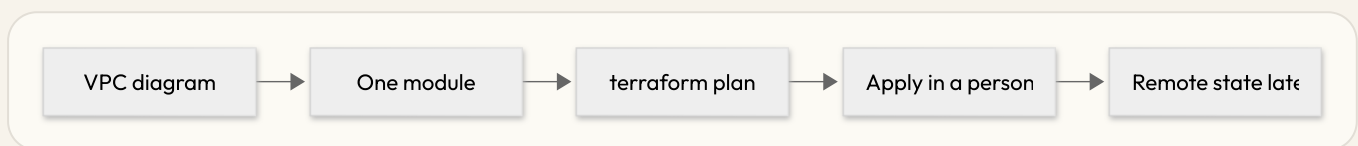
Put a request id on the project API log line and on the Kafka event.

One Terraform module after the diagram

Infrastructure as code is how the VPC you drew stays the VPC you deployed. It is useful. It is not a certification and not phase 1.

1. Resources match the diagram
2. Plan before apply
3. State is the source of what exists
4. Secrets stay out of the file

Figure



What you are declaring

The module for this project is small: a VPC public and private subnet story you can explain, a security group that allows the app port from the load balancer only, an S3 bucket for synthetic objects, and an RDS subnet group. Names match the diagram in the cloud lessons. If a resource is in the file and you cannot say why, delete it from the file.

State and drift

Terraform state is a JSON map from your names to AWS ids. Two people applying without a lock will fight. A console click that Terraform does not know about is drift. Import or abandon it. None of this is required to pass a Java loop in January. It is required the week you deploy the project for real and want to talk about it.

Do it in this order

1. A resource block declares an VPC, a subnet, a security group, or an S3 bucket.
2. terraform plan is the diff. Read it. Apply only what you meant.
3. State maps the config to real ids. Lose the state and Terraform will try to create duplicates. For a real account, state lives in a remote bucket with a lock.
4. Do this in phase 3, after IAM and VPC make sense. One module for the project network is enough.

Worked example

```

resource "aws_s3_bucket" "objects" {
  bucket = "prep-object-events-demo"
}
  
```

The usual miss

They will ask

Starting a 40-hour Terraform course before the first VPC is drawn.

What does terraform plan tell you that the AWS console does not?

Before you close the laptop

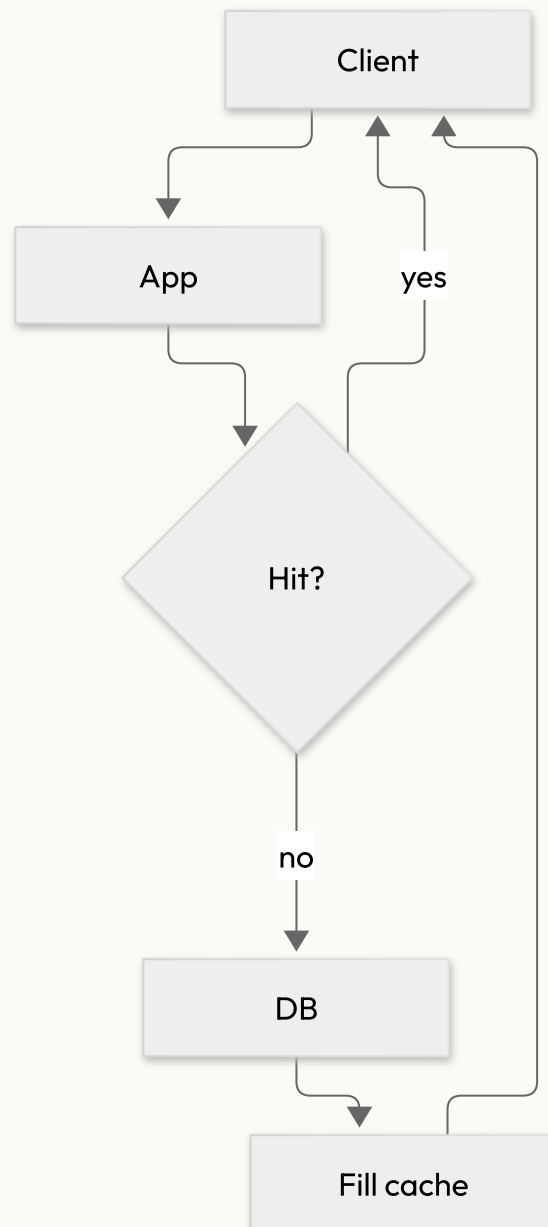
Write the three resources the project needs. Do not apply them until the IAM lesson is done.

Distributed cache

Cache is a design on its own because freshness, stampedes, and failure are the whole question.

1. Cache aside
2. TTL matches the tolerance
3. Single-flight on a miss
4. Redis down has a decision

Figure



Walk the checklist

Requirements and the staleness you will accept come first. Scale is the hot key, not the average key. The API does not change. The data model is the key and the TTL. Architecture is cache-aside in front of Postgres. Consistency is eventual within the TTL, plus an explicit delete on update. Observability is hit ratio and fill

latency. Security is a cache that does not hold raw credentials. The trade-off is freshness against database load.

Do it in this order

1. Requirements: read-heavy bucket metadata, 5 minutes of staleness is acceptable, writes are rare.
2. Scale: state the read QPS and the row size. The cache holds the hot set, not the whole table.
3. API stays the same. The cache is invisible to the caller.
4. Data: key is bucket id. Value is the metadata JSON. TTL 60 seconds.
5. Failure: on Redis timeout, read the database. Latency rises. Correctness holds. A stampede uses a lock or single-flight so one request fills the key.

Worked example

```
Key: bucket:{id}  
TTL: 60s  
Miss: one filler, others wait or hit the database  
Invalidate on write, TTL as the backstop
```

The usual miss

Caching the write path and serving a stale capacity number as if it were live.

They will ask

A hot key expires and a thousand requests miss together. What do you do?

Before you close the laptop

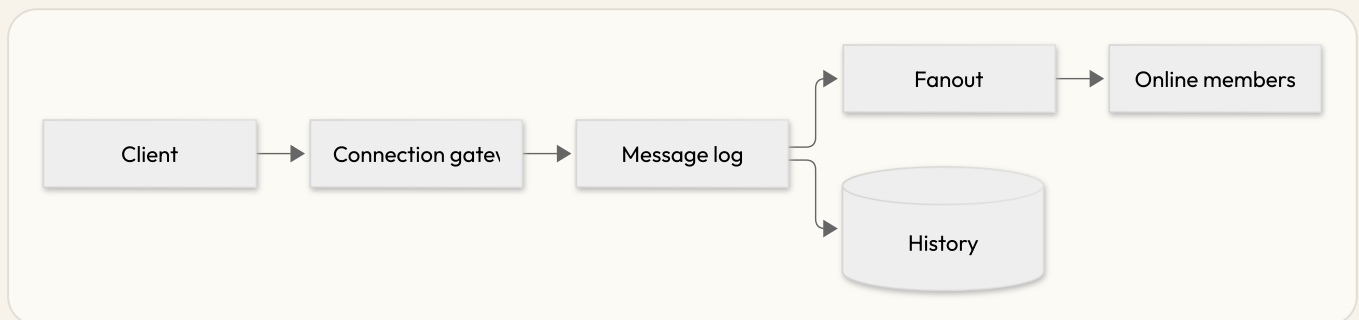
Say the TTL for bucket metadata and why that number.

Chat

Chat is a log plus a connection tier. The hard part is order inside a conversation and a client that reconnects.

1. A channel is a partition
2. Store then fanout
3. Presence is ephemeral
4. Reconnect sends the last seen id

Figure



What makes it different from a cache

A cache can drop data. A chat log cannot. Presence and typing are the cache-like parts: they expire.

Messages are the database. Fanout to online users is best-effort on top of the log. Read receipts can wait. Say what you cut.

Do it in this order

1. Send, receive, history, typing if you have time. One conversation, then groups.
2. Scale: messages per second, not concurrent sockets alone. A socket is a connection. The log is the product.
3. API: `send(conversationId, clientMsgId, body)`. History by cursor.
4. The conversation id is the partition key so one conversation stays ordered.
5. On reconnect the client sends the last message id. The server replays after that id.

Worked example

```

clientMsgId makes send idempotent.
The server stores the message, then fans it out.
A crash after store and before fanout is repaired by the client catch-up.
  
```

The usual miss

Keeping history only in the memory of the gateway.

They will ask

Two devices send, one socket drops. What does the client send on reconnect?

Before you close the laptop

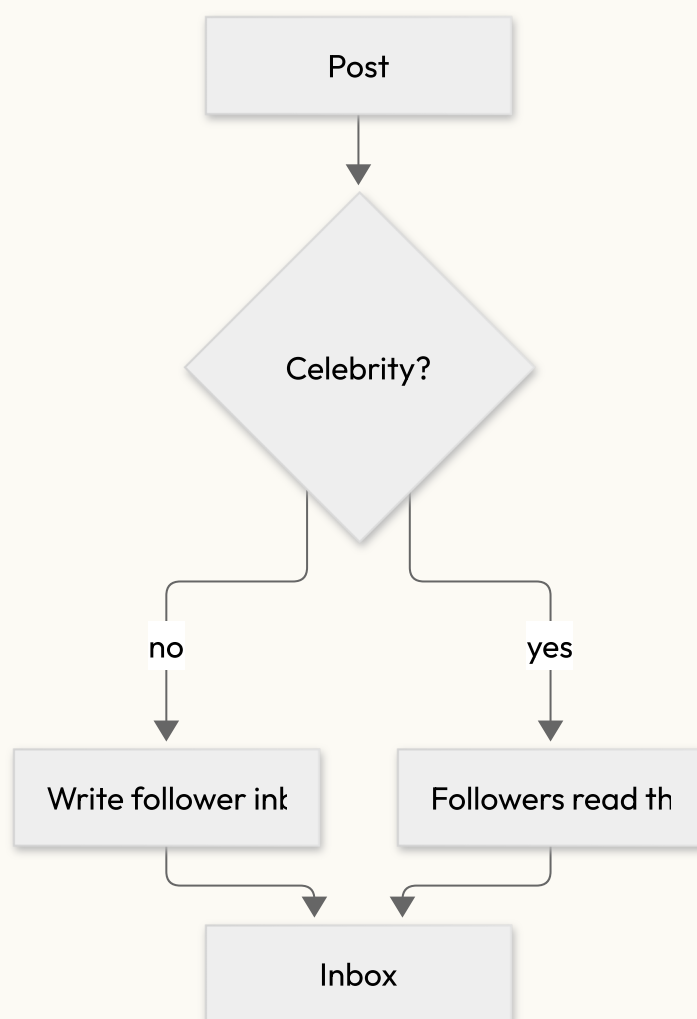
Draw the log and the last-seen id. Stop there.

News feed

The trick is fanout. Push the post to followers when followers are few. Pull, or mix, when one account has millions.

1. Fanout on write for normal users
2. Fanout on read for celebrities
3. The feed is a ranked inbox
4. The post store is the source

Figure



Checklist in short

The queue is the fanout work. The cache is the rendered page of the inbox. Consistency is eventual: a post appears within seconds. Failure of a fanout worker retries. A poison post goes to a dead letter so one bad body does not block the worker. Observability is fanout lag.

Do it in this order

1. Requirements: home feed, follow, post. Ranking can be recency in version one.
2. Scale: follows are uneven. Design for the heavy followee.

3. Post is stored once. Inbox rows are post ids, not copies of the text.

4. A hybrid: push to the first N followers, pull the rest at read time.

Worked example

Normal user: on post, enqueue fanout, workers append the id to follower inboxes.

Celebrity: skip the fanout. At read, merge the celebrity's recent posts into the inbox.

The usual miss

Copying the full post into every follower's row.

They will ask

When does fanout on write fall over?

Before you close the laptop

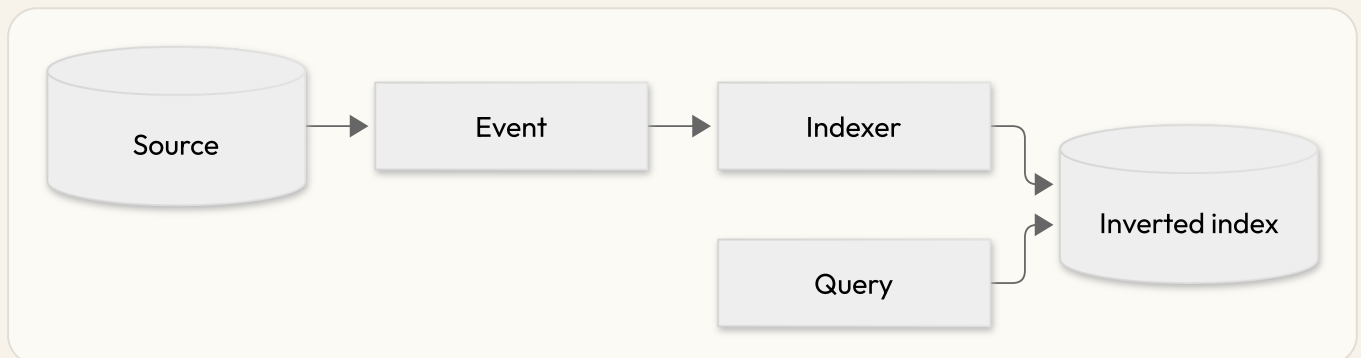
Say hybrid in one sentence with the threshold you invented, and call it an assumption.

Search

The database is the source of truth. The index is a derived store built from a stream.

1. Index is not the database
2. Build from the event stream
3. Queries hit the index
4. Rebuild is possible

Figure



The trick to say first

Search is a second store. It can be wrong for a while. The source cannot. Tokens, inverted lists, and a rank function are enough. You do not need to implement a search engine. You need to say what is authoritative.

Do it in this order

1. Requirements: keyword search over object names and synthetic incident notes.
2. The write path updates Postgres and emits an event. The indexer updates the inverted index.
3. Search can be stale by the lag of the indexer. Say the lag.
4. A full rebuild from the database exists, because indexes corrupt.

Worked example

Document: incident 41, tokens [disk, latency, bucket].
 Query "latency" returns the document id, then the API loads the row.

The usual miss

Searching with LIKE on the primary table at millions of rows.

They will ask

The indexer is an hour behind. What does the user see?

Before you close the laptop

Name the event that updates the index in the project.

Payments

A payment is a state machine and a ledger. The idempotency key is the part you already understand from orders.

1. Idempotency key
2. Ledger entries that sum
3. Illegal state jumps fail
4. The provider call is at-least-once

Figure



Why this design shows up

It tests idempotency, money, and honesty about failure. You do not need to have worked at a bank. You need the state machine and the key. Connect it to the safe executor: a tool call that changes capacity is a state machine with a human approval edge.

Do it in this order

1. Requirements: charge once, refund once, show a history.
2. Scale: write QPS is smaller than read. Correctness dominates.
3. API: POST /charges with Idempotency-Key. Same key and same body returns the same charge. Same key and a different body returns 409.
4. Data: a charge row and ledger lines. The balance is the sum, or a cached sum you can rebuild.
5. The provider timeout uses the same key on the retry so the provider does not capture twice.

Worked example

States: created, authorized, captured, failed, refunded.
 captured to created is rejected.
 Two ledger lines: debit the payer, credit the merchant.

The usual miss

They will ask

Updating a single balance column with no history.

The provider timed out. You retry. How many captures occur?

Before you close the laptop

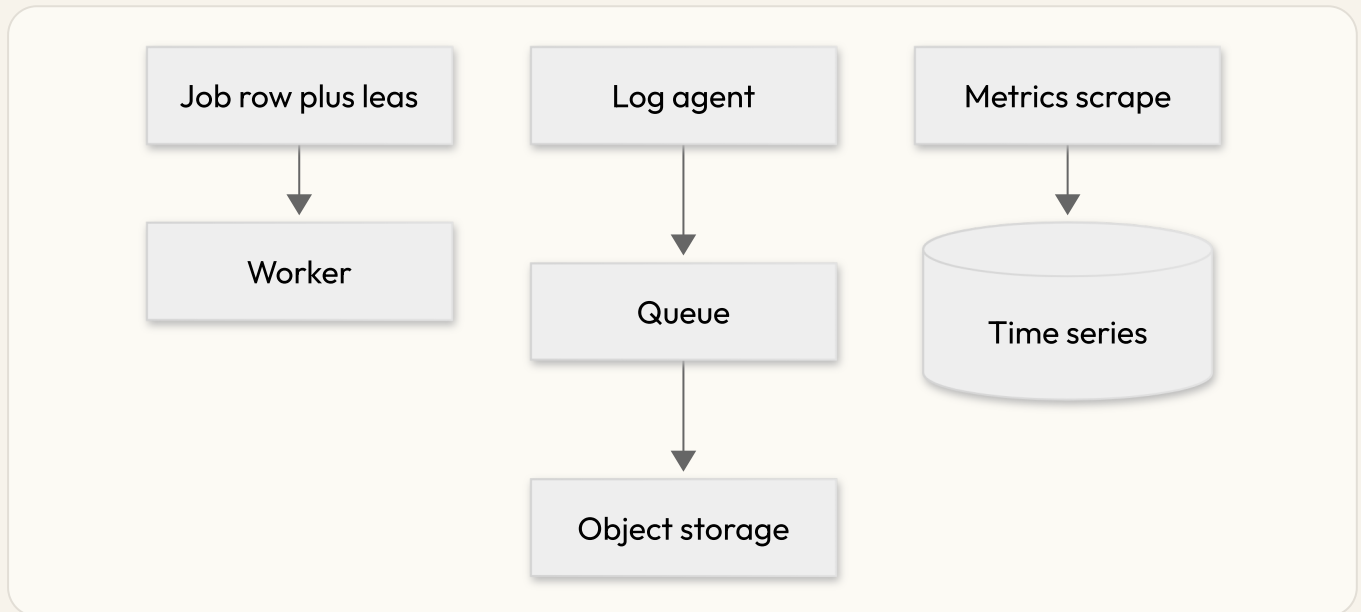
Draw the states. This is also the shape of a safe tool call.

Job scheduler, logs, and metrics

Three operational designs. Each has one trick: a lease, a buffer, and a time series.

1. One owner via a lease
2. Logs are buffered
3. Metrics are aggregated
4. All three tolerate a dead worker

Figure



Logs versus metrics

A log line is an event with a high amount of detail and a request id. A metric is a number you can alert on. You do not grep logs to page someone, and you do not put the request id into a metric label. The diagnostics agent is allowed to read both. The page is a metric.

Do it in this order

1. Scheduler: a job has a `run_at` and a `lease_until`. A worker updates the lease in a transaction. If it dies, the lease expires. The handler is idempotent because the lease can expire early.
2. Logs: agents batch lines, a queue absorbs a burst, object storage keeps the bulk, an index keeps the recent window for the RCA agent.
3. Metrics: counters and histograms, scraped or pushed. Retention is short at high resolution and long at low resolution. The capacity agent reads this store.

Worked example

```
UPDATE job SET lease_until = now() + interval '30 seconds'
WHERE id = ? AND lease_until < now()
```

The usual miss

A cron on every pod.

They will ask

Two workers run the same job. What prevents a double side effect?

Before you close the laptop

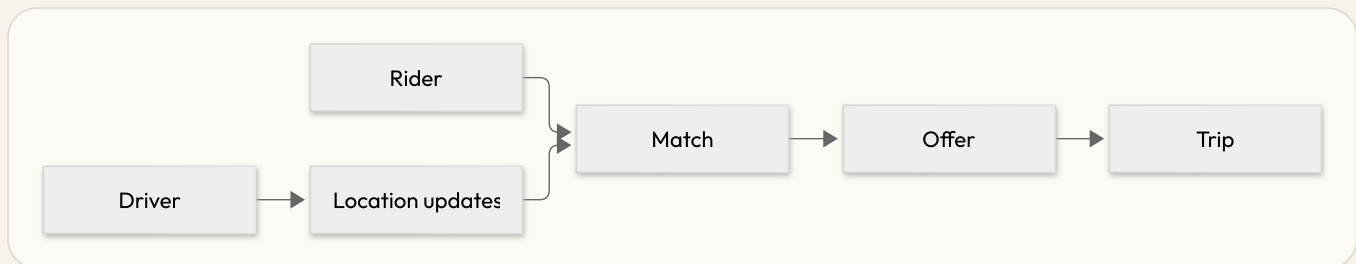
Write the lease condition. It is the scheduler.

Ride booking

The trick is matching under contention. Two riders must not take the same driver.

1. Location is a hot write
2. Match is a short transaction
3. One driver, one offer
4. The trip is a state machine

Figure



What to cut

Pricing, maps, and surge can be a single sentence: a price is computed and stored on the trip at match time. Spend the minutes on the race. This design is practice for any exclusive allocation, including a capacity reservation on a cluster.

Do it in this order

1. Requirements: request a ride, match a nearby driver, complete the trip.
2. Location updates are frequent and lossy. The latest point matters. A cache or a specialized store holds them.
3. Matching locks the driver row, or uses a conditional update: status = free. The loser retries with the next driver.
4. States: requested, matched, started, completed, cancelled.

Worked example

```

UPDATE driver SET status = 'offered'
WHERE id = ? AND status = 'free'
  
```

The usual miss

Finding the nearest driver with a full table scan and then hoping the update wins.

They will ask

Two matchers pick the same driver. Who gets the trip?

Before you close the laptop

Say the conditional update. That sentence is the design.

How every design is told

SDE II loops at Amazon and Microsoft put HLD next to DSA. You use the same spine every time so you do not freeze on the noun they picked.

1. Clarify read and write
2. One number for scale
3. API then data
4. Failure before you stop

Figure

Requirements



Scale



API



Data model



Architecture



DB



Cache



Queue



Consistency



Failure



Observe



Security



Tradeoffs

Do it in this order

1. Functional requirements in five bullets. Non-functional: latency, availability, consistency.
2. Back-of-envelope: QPS, storage per day, read/write ratio. Order of magnitude is enough.
3. End on trade-offs. A design with no trade-off is a drawing.

Worked example

40 minutes. Talk first. Draw second. No video until you have spoken.

The usual miss

Jumping to Kafka because it sounds senior.

They will ask

What did you refuse to build in v1?

Before you close the laptop

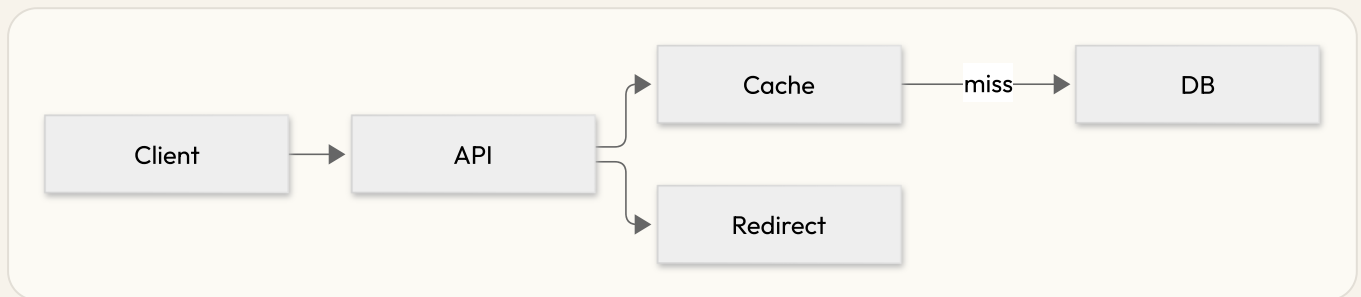
Run the spine on a system you already operate. Eight minutes.

URL shortener

The classic is a key-value write with a read-heavy cache. It teaches ids, redirects, and cache.

1. Generate an unguessable id
2. Store url by id
3. Cache the hot ids
4. 301 versus 302

Figure



Walk it

Create returns a short id. Read redirects. The id is random or a counter encoded in base 62. Reads dominate, so a cache sits in front. The database is the source. A collision retries. Analytics are an async event, not a write on the redirect path. Estimate: reads per second, row size, cache hit ratio. That estimate is a practice, and you say the assumptions.

Do it in this order

1. Write path: hash or an id service, unique constraint, return the short url.
2. Read path: cache then database. Redirect. Do not count analytics on the request thread.
3. Analytics is an async event. That is your first queue in this design.

Worked example

```
GET /{id} -> 302 Location
```

The usual miss

A sequential integer id that leaks volume and is easy to scrape.

They will ask

How do you keep one id from being issued twice?

Before you close the laptop

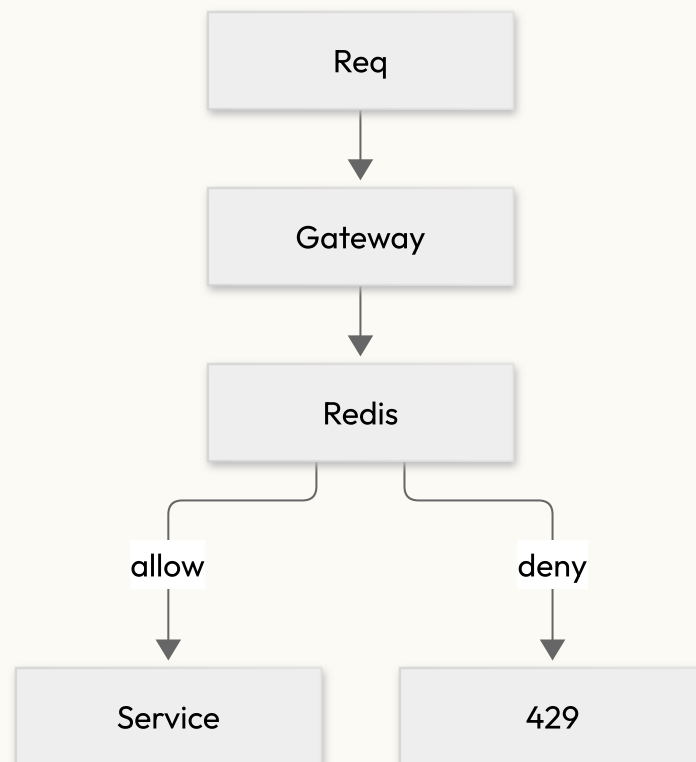
Speak it once with a cache and an analytics event.

Rate limiter

Apple and backend loops ask this because it is a small distributed system: counters, clocks, and failure.

1. Token bucket
2. Key is the client
3. Redis is shared
4. Fail open or closed on purpose

Figure



Walk it

Token bucket: tokens refill with time, a request takes one. Sliding window counts requests in the last N seconds. Store the counter in Redis so every app instance sees it. The key is the client. Failure of Redis is a product decision: fail open to keep the site up, or fail closed to keep a limit. Say which, for this API. The LLD lesson is the same limiter as classes.

Do it in this order

1. Token bucket: capacity and refill rate. Each request takes a token.
2. Key by user or token, not one global bucket, unless the limit is for the whole cluster.
3. If Redis is down, fail open for a product page and fail closed for a write that spends money. Say which.

Worked example

INCR + EXPIRE is a crude fixed window. Mention the boundary burst.

The usual miss

A limiter in one process memory when you have twenty pods.

They will ask

Two requests at the same millisecond. What happens?

Before you close the laptop

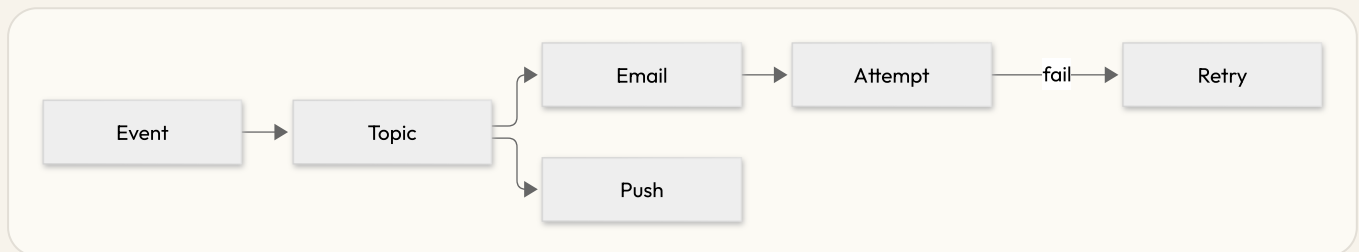
Put this in front of the agent tool-execution endpoint. That is the same design.

Notification system

Fanout, retries, and idempotent delivery. This is the Kafka design with a user-visible result.

1. Event in
2. One worker per channel
3. Retry with backoff
4. Do not double-send

Figure



Walk it

A request writes a notification row and an outbox row. Workers send email, push, or webhook. Each send has a provider id so retries do not double-send. A dead letter holds addresses that bounce. Fanout to many devices is a queue, not a loop in the request. Idempotency is the provider key plus the notification id.

Do it in this order

1. The API accepts the intent and returns. Delivery is async.
2. Idempotency key so a retry does not email twice.
3. Preferences and quiet hours are a filter before the provider call.

Worked example

`notification_id` unique at the provider adapter.

The usual miss

Calling the email vendor inside the user request.

They will ask

The worker crashes after the vendor accepts.
Did the user get two mails?

Before you close the laptop

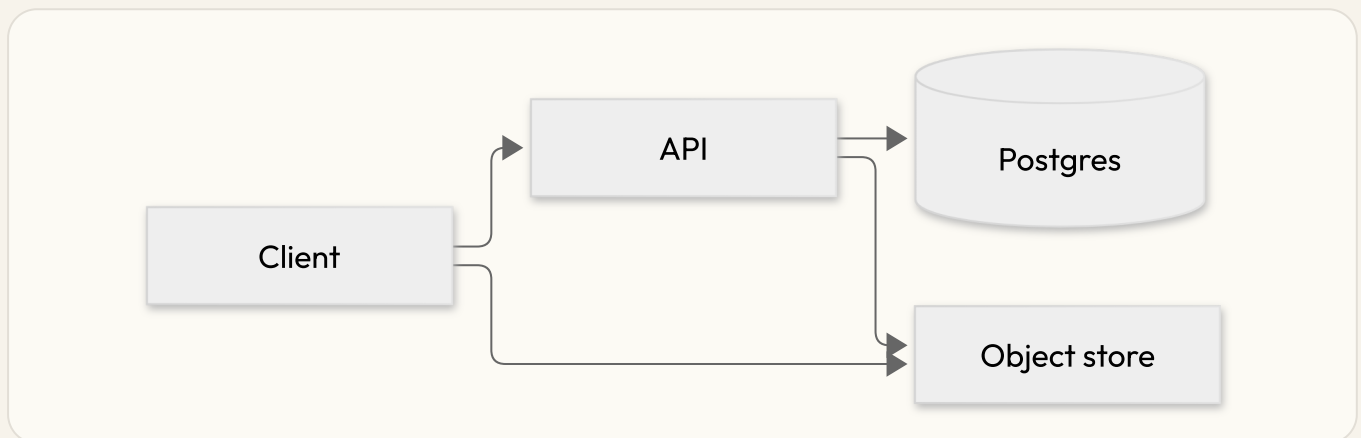
Map this onto an incident notice for a bucket, with synthetic users.

File and object storage

This is the design closest to your storage background. Metadata in a database, bytes in object storage, upload in parts.

1. Metadata is not the bytes
2. Multipart upload
3. Checksum
4. Lifecycle

Figure



Walk it

Bytes go to object storage in parts. Metadata and the part list go to Postgres. Complete assembles the object. A presigned URL lets the client upload without proxying the bytes through your app. This is the flagship's storage, synthetic. Dedup and encryption are sentences if time remains. The metadata write and the event use the outbox.

Do it in this order

1. The API issues an upload plan. The client sends parts to object storage.
2. Complete is a metadata transaction: all parts present, checksum matches, object becomes visible.
3. A garbage collector deletes abandoned parts. Say the delay.

Worked example

HEAD for existence. GET via a short-lived signed URL.

The usual miss

Storing the file bytes in Postgres.

They will ask

A part never arrives. What does the user see?

Before you close the laptop

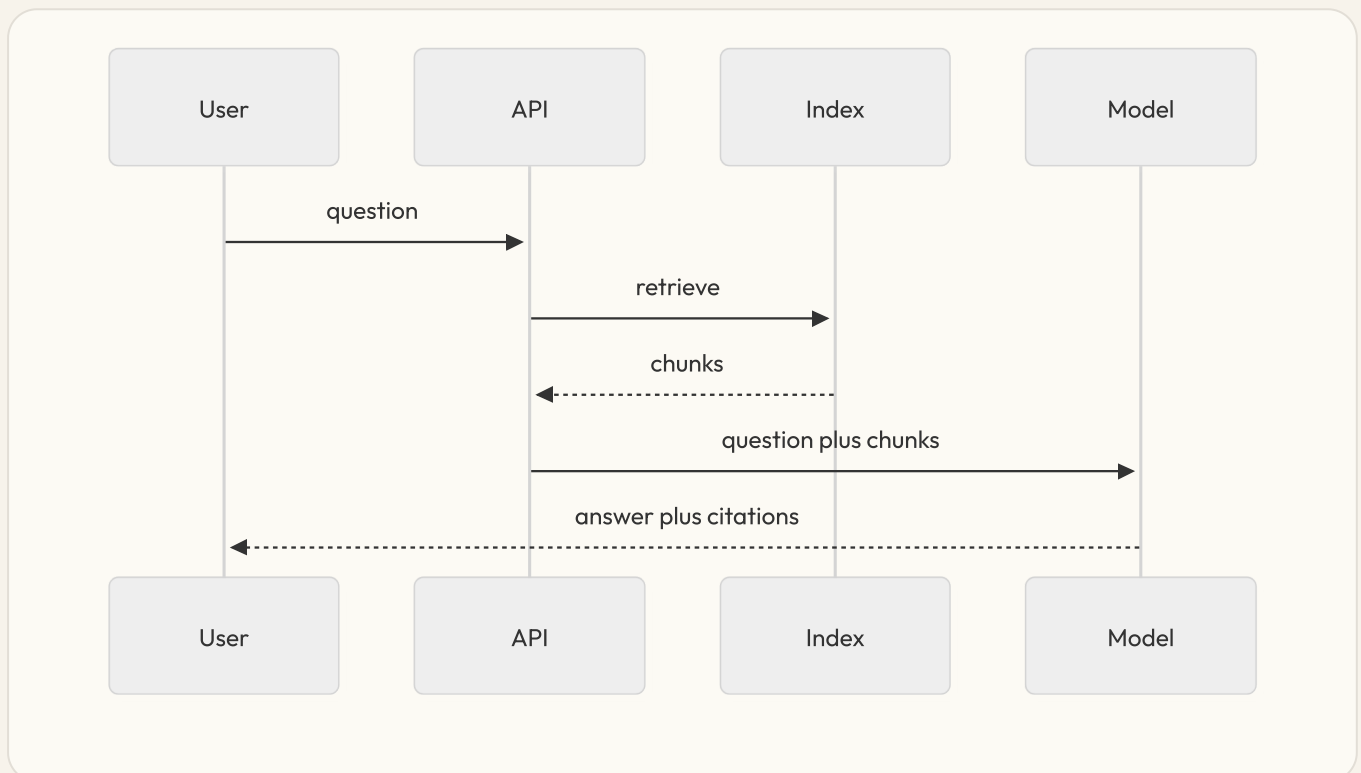
Tell this design with a bucket you invent. No Dell hostnames or schemas.

AI chat with RAG

The design interview version of your RAG work. Retrieval, citations, and what you do when the model is wrong.

1. Ingest offline
2. Retrieve before generate
3. Cite the chunk
4. Evaluate a fixed set

Figure



Walk it

The design answer is the pipeline plus failure: the model can be down, the retrieval can be empty, the citation can be wrong. Empty retrieval returns 'I do not have that' rather than a fluent guess. The eval set is part of the design, not a follow-up. Cost is a metric: tokens in, tokens out.

Do it in this order

1. Ingestion is a pipeline, not part of the user request.
2. The prompt may only use retrieved text for factual claims. If retrieval is empty, say so.
3. A set of questions with expected facts. Track pass rate, latency, and cost per answer.

Worked example

Answer: chunk 14 says the quota is 80 percent. Link the chunk.

The usual miss

A chatbot that answers from memory with no source.

They will ask

How do you know the assistant is not inventing a root cause?

Before you close the laptop

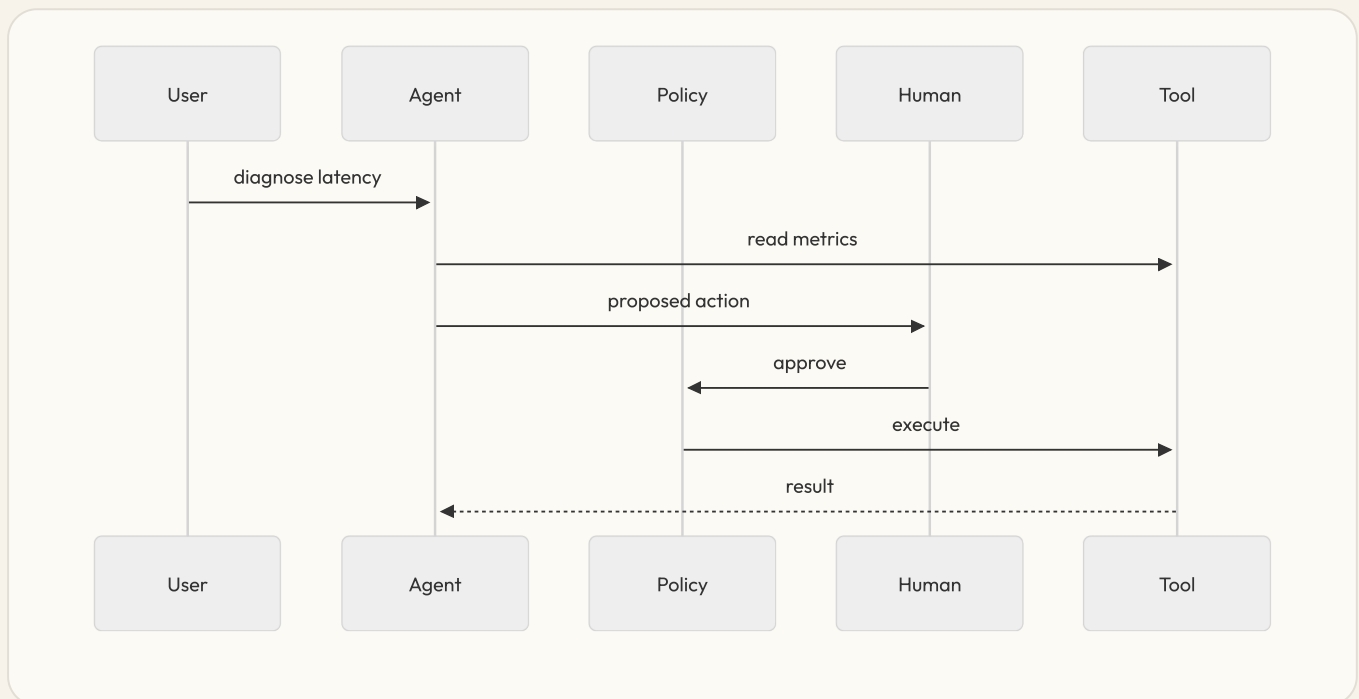
Write 5 evaluation questions for a fake storage runbook.

Agent orchestration with a human gate

The differentiator. An agent may suggest a change. It may not apply the change until a person approves, the tool is allow-listed, and the call is idempotent.

1. Read tools are wider
2. Write tools need approval
3. Timeouts and retries
4. Trace every tool call

Figure



Walk it

An orchestrator holds state. Specialist agents read logs, metrics, or config. A tool executor checks an allow-list and a human approval for anything that mutates. Every call has a timeout and a trace id. The platform is the project. In a design interview you draw the gate before you draw a second model.

Do it in this order

1. Split tools into read and write. Diagnostics are read. A config change is write.
2. The approval record stores who, what, and the idempotency key.
3. Prompt injection in a log line must not be able to call the write tool. The model does not hold the credential. The policy service does.

Worked example

Tool list: `getMetrics`, `getLogs`, `proposeScale`. Only `proposeScale` needs a human.

The usual miss

Giving the model a kubeconfig.

They will ask

How do you stop an agent from looping on a tool?

Before you close the laptop

Draw the approval box on the flagship diagram before you write more code.

Cache, chat, and feed in one sitting

Three designs share a spine. Learn the part that makes each one different, not three new religions.

1. Cache is freshness
2. Chat is a channel per conversation
3. Feed is fanout on write or read
4. All three need a queue

Figure

Cache: TTL and s

Chat: fanout and

Feed: push versu:

Do it in this order

1. Distributed cache: cache aside, TTL, single-flight on miss. Redis in front of Postgres.
2. Chat: a connection tier, a message log, fanout to online users, store for history. Presence is ephemeral.
3. Feed: push to followers' inboxes when celebrities are rare. Pull or a hybrid when one account has millions of followers.

Worked example

The hard part of chat is not the bubble. It is ordering in a conversation and not losing a

The usual miss

Designing all three as a single database table.

They will ask

When does feed fanout on write fall over?

Before you close the laptop

Pick feed. Say push, pull, or hybrid in one sentence and why.

Search, payments, scheduler, logs

Four more nouns. Each has one trick. Say the trick first.

1. Index asynchronously
2. Charge once
3. Only one worker owns a job
4. Logs are append-only

Figure

Search: index is n

Pay: idempotent

Scheduler: lease

Logs: buffer then

Do it in this order

1. Search: the database is the source. An index (inverted or a service) is built from a stream. Queries hit the index.
2. Payments: idempotency key, ledger entries that sum, and a state machine. Never a single balance update with no history.
3. Scheduler: a job row with a lease timestamp. A worker updates the lease. If it dies, another worker takes the expired lease.
4. Log aggregation: agents buffer, a queue absorbs bursts, object storage keeps the bulk, an index keeps the recent window.

Worked example

Payment states: created, authorized, captured, failed. Illegal jumps are rejected.

The usual miss

A cron on every pod so the job runs five times.

They will ask

Two API calls with the same idempotency key.
How many charges?

Before you close the laptop

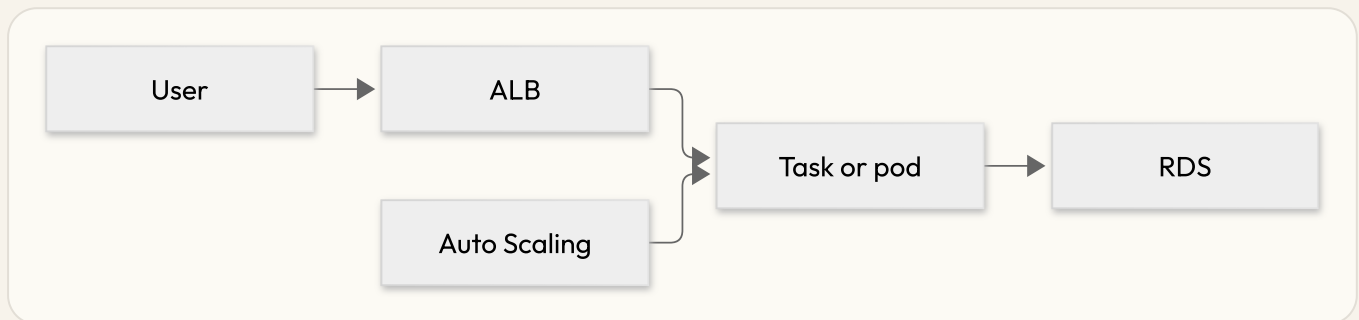
Write the payment states on a card. Carry it to one mock.

EC2, ALB, autoscaling, ECS, EKS

Compute is where the process runs. You pick it after you know the network, not before.

1. ALB checks health
2. Scale on a signal
3. ECS is tasks
4. EKS is Kubernetes

Figure



The path of a request

DNS names the ALB. The ALB is in public subnets. The target is a pod or task in private subnets. The security group on the target allows the app port only from the ALB's security group. The target calls RDS on 5432, and RDS allows that security group. If you can draw those four boxes, you can answer most of the compute section of an SAA-style question.

ECS or EKS

ECS is the smaller operational surface. EKS is the one that matches CKAD and your OpenShift work. Pick EKS for the flagship so the story stays one story. Say the trade: you operate more of Kubernetes, and you keep the skills the backend roles asked for.

Do it in this order

1. EC2 is a virtual machine. You patch it. A private subnet holds the app. A public subnet holds the load balancer.
2. An ALB routes HTTP and checks a health path. Unhealthy targets leave rotation.
3. Auto Scaling adds instances from a signal: CPU, request count, or queue depth. Queue depth is the honest signal for workers.
4. ECS runs containers without you managing the control plane. EKS is Kubernetes. Use EKS if the interview story is the Kubernetes one you are already learning. Do not run both for the project.

Worked example

Health check: GET /actuator/health

Unhealthy for three checks, then the target is out.

The pod or task restarts. The ALB does not send it traffic until it is healthy.

The usual miss

Scaling on CPU while the bottleneck is a full connection pool.

They will ask

Why would you scale the consumer on queue lag instead of CPU?

Before you close the laptop

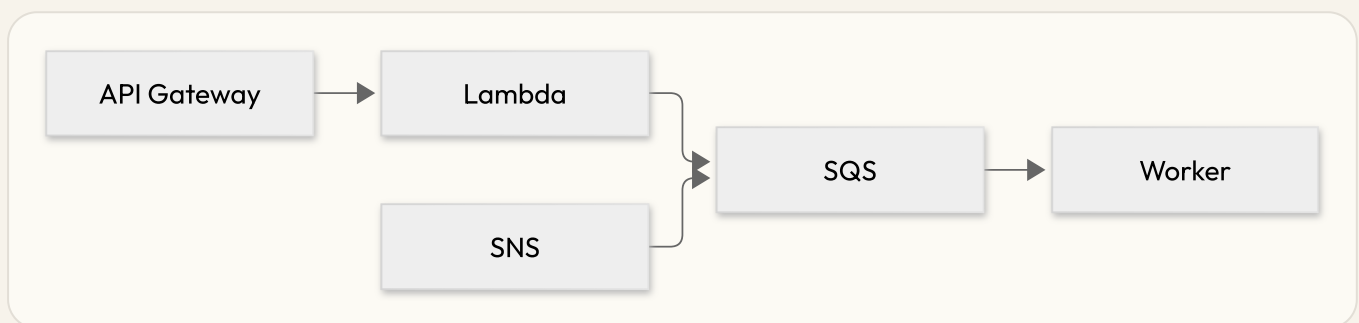
Write the health path and the one metric that should add a worker.

SQS, SNS, Lambda, API Gateway

AWS messaging is the managed version of the event story. Kafka remains the project log. SQS is the simpler queue when you do not need replay.

1. SQS is a queue
2. SNS is fanout
3. Lambda is a small function
4. API Gateway is the front door

Figure



When SQS is the right box

You need a buffer and a worker, you do not need to replay a year of events, and you do not need order across a key for the long term. Object-created notifications can land on SQS. The diagnostics stream that you want to replay belongs on Kafka. Say that distinction. It is the senior answer.

Lambda limits

A function is stateless, time-boxed, and billed in memory-time. It is a good fit for a thumbnail or a webhook ack. The agent that calls tools, waits for a human, and writes an audit row is a service. Put it in the FastAPI process you already planned.

Do it in this order

1. SQS: at-least-once, visibility timeout, dead-letter queue. The handler is idempotent. A message returns if you do not delete it.
2. SNS: one publish, many subscribers. Use it to fan out. Use SQS behind a subscriber when that subscriber must buffer.
3. Lambda fits a short transform. A long agent loop does not belong in a 15-minute function with a cold start you have not measured.
4. API Gateway terminates HTTP and can authorize. For the project, the ALB in front of Spring is enough until you have a reason.

Worked example

```
Visibility timeout 30s.  
Handler finishes in 5s and deletes.
```

If the handler dies, the message reappears.
The second run sees the idempotency key.

The usual miss

Using Lambda because the diagram looked modern.

They will ask

SQS delivered the same message twice. What in your code makes that safe?

Before you close the laptop

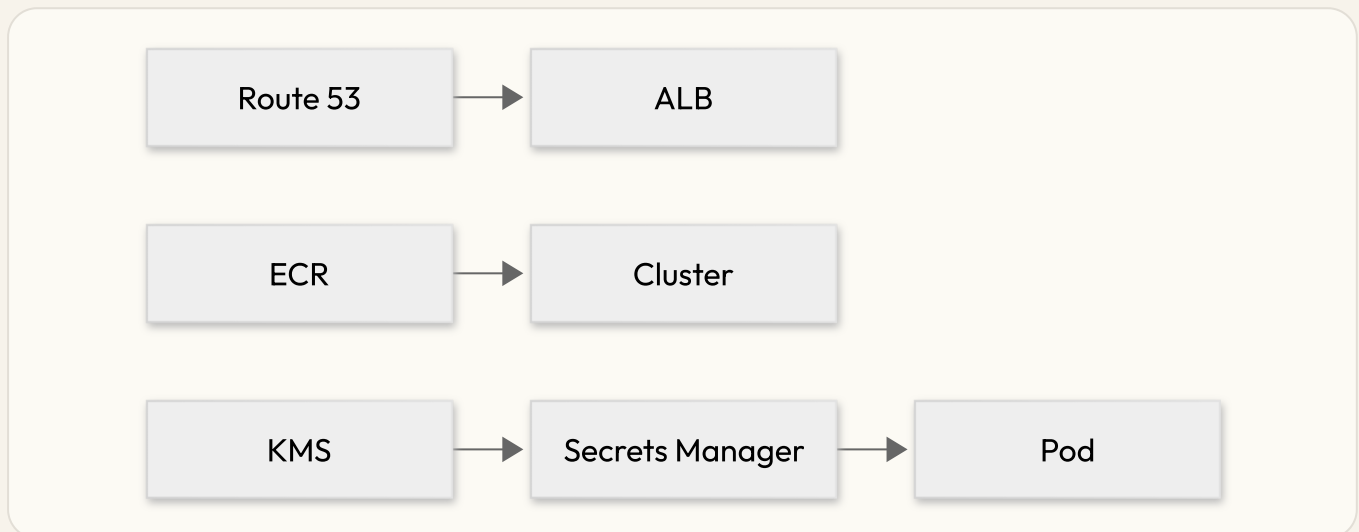
On paper, which project event is Kafka and which could be SQS, and why.

KMS, secrets, ECR, Route 53

The security half of AWS is who can call, where the secret lives, and how the image got there.

1. IAM role on the pod
2. Secrets are not env files in git
3. ECR holds the image
4. KMS wraps the key

Figure



Identity

IAM decides. A policy allows `s3:GetObject` on one bucket. The pod assumes a role through the cluster's identity mechanism. A human uses a role too. Root is not a daily login. CloudTrail records the calls. That is the security story that pairs with the agent: the tool the agent can call is a role with a short list of actions, not an admin key.

Images and names

You build a jar, then an image, then you push to ECR. The Deployment pins the digest or the tag you just pushed. Route 53 is only the name the laptop uses to reach the ALB. None of these require the Cloud Practitioner exam.

Do it in this order

1. A pod uses an IAM role. Long-lived access keys on a laptop are for the experiment, then they go away.
2. Secrets Manager or SSM holds the database password. The manifest references the secret. The git repo does not.
3. ECR stores the image. The cluster pulls it. Scan it. Pin the tag you deployed.
4. KMS encrypts S3, RDS, and the secret. Rotation is a feature you can name.
5. Route 53 maps the name to the load balancer. A health check removes a bad region later, not in version one.

Worked example

The repo contains a secret name.
The cluster injects the value.
git log never shows the password.

The usual miss

Committing an AWS key because the demo was on Friday.

They will ask

How does the pod reach S3 without an access key in the image?

Before you close the laptop

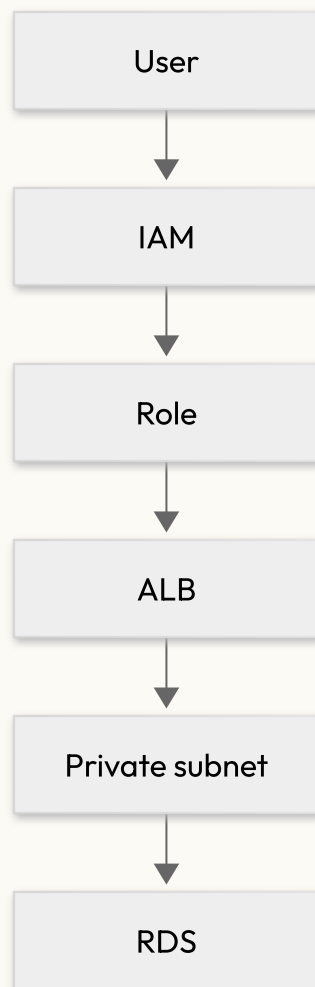
Check the project for a password in a file. Move the name into a note, not the value.

IAM and VPC before any service

Solutions Architect Associate is the credential, later. The knowledge starts with who can call what, and which network path exists. Cloud Practitioner is the wrong exam for you.

1. No long-lived keys on laptops
2. Roles for services
3. Database is private
4. Security groups are stateful

Figure



Draw this before SAA

A VPC. Public subnets for the load balancer. Private subnets for the tasks and the database. A NAT gateway if private tasks call the internet. IAM policies allow the task role to read one bucket and one secret. Nothing else. Cloud Practitioner does not teach this at the depth a backend loop expects. SAA is the credential after you can draw it from memory.

Do it in this order

1. IAM user is a person. A role is what EC2, EKS, or Lambda assumes. Policies are the allow list.
2. VPC, public subnet for the load balancer, private subnet for the app and the database.

3. Security group on the database allows the app group on 5432 and nothing from the internet.

Worked example

SAA is the supporting certificate after you can draw this. Do not start with Cloud Practiti

The usual miss

Opening port 22 to the world because a tutorial did.

They will ask

How does the pod reach S3 without an access key in the image?

Before you close the laptop

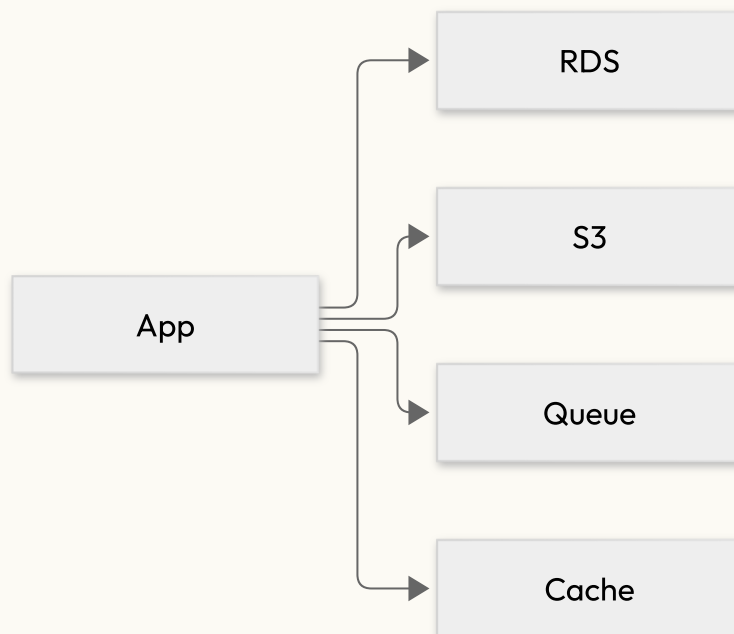
Draw the VPC once. Photograph it. That is this week's cloud note.

S3, RDS, queues, and cache

The flagship maps cleanly: bytes in S3, metadata in RDS, events on SQS or Kafka, hot state in ElastiCache.

1. S3 is objects
2. RDS is Postgres
3. SQS is a buffer
4. ElastiCache is Redis

Figure



Which store

S3 holds bytes: synthetic objects, log batches. It is durable and the request pays per call. RDS Postgres holds metadata and the outbox, with transactions. DynamoDB is for a key-value access pattern with a scale you can explain. Do not add it beside Postgres without a reason. ElastiCache is Redis. SQS is the simple queue. The project uses Kafka for the replayable log and can use SQS where replay does not matter.

Do it in this order

1. S3: bucket, key, storage class, lifecycle, pre-signed URL. Not a filesystem you mount and forget.
2. RDS Postgres for the source of truth. Multi-AZ is availability, not a write-scaling strategy.
3. SQS for a simple buffer. Kafka when you need replay and several consumers. Say the difference.
4. ElastiCache Redis for the rate limit and the cache. Same rules as the Redis topic.

Worked example

A pre-signed URL expires. The client uploads directly. Your API stays small.

The usual miss

Using SQS and also expecting to rewind the stream a week later.

They will ask

When do you pick SQS over Kafka?

Before you close the laptop

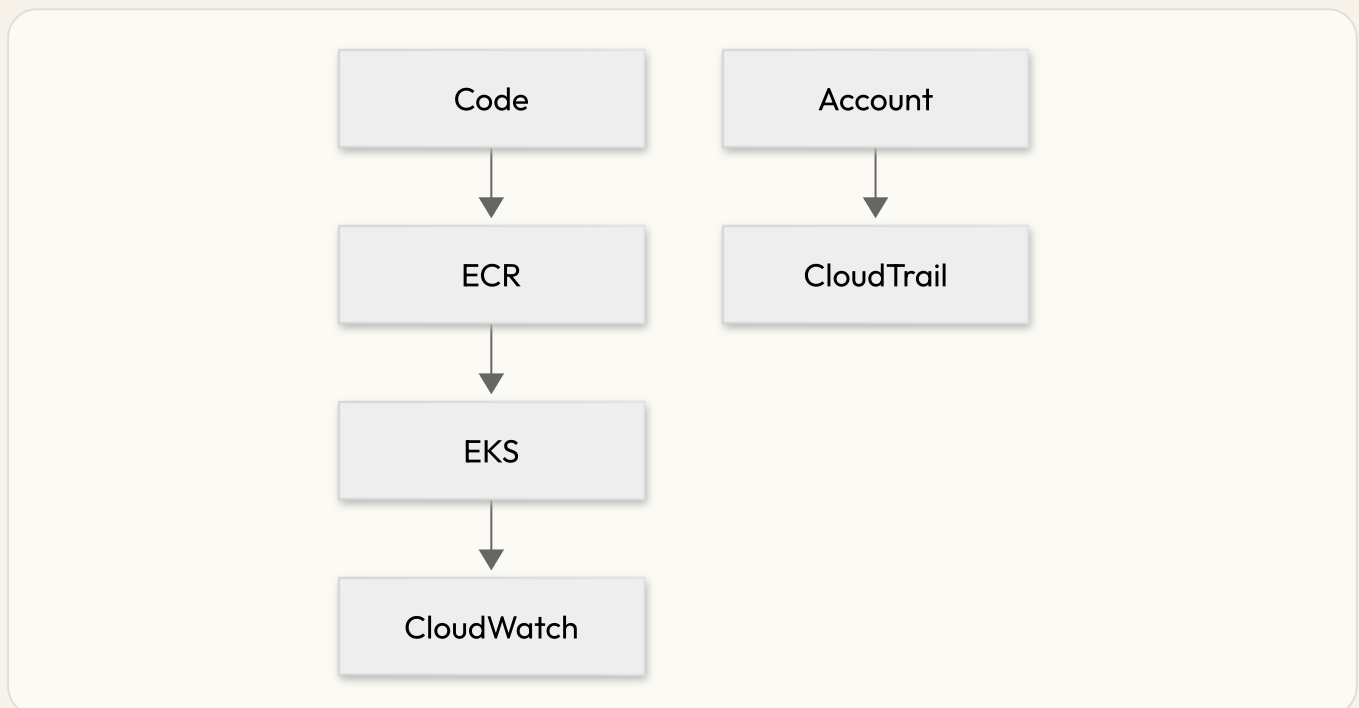
Label the four boxes on the flagship diagram with the AWS name.

EKS, images, and CloudWatch

EKS is Kubernetes with AWS as the cloud under it. You still need to understand pods. CloudWatch and CloudTrail tell you what the service and the account did.

1. Build an image
2. Push to ECR
3. Deploy to EKS
4. Logs and a trail

Figure



What you deploy

The image is in ECR. EKS runs the Deployment. CloudWatch holds logs and metrics. CloudTrail holds API calls such as who changed a security group. Alarms fire on error rate and on disk. The agent reads the same metrics. You can describe this without having the production account on day one. A local cluster plus a diagram is an honest version of the story until the apply happens.

Do it in this order

1. ECR stores the image. EKS runs it. IAM on the service account is how the pod calls AWS.
2. CloudWatch for logs and metrics. An alarm on error rate and on disk.
3. CloudTrail is the audit of API calls. Useful when someone changes a security group.

Worked example

You do not need Lambda, API Gateway, and Route53 in week one. Add them when the design needs them.

The usual miss

Putting the AWS key in the container environment.

They will ask

What is the difference between CloudWatch and CloudTrail?

Before you close the laptop

Write the alarm you would want on the diagnostics agent: error rate and token cost.

Docker, then the cluster

An image is the unit you ship. If the container does not run on your laptop, Kubernetes will not save it.

1. One process per container
2. Config by environment
3. A health URL
4. Non-root user

Figure



The image

A Dockerfile copies the jar or the venv and sets the user, the port, and the command. The image runs on your laptop with the same env the cluster will use. A health check is in the app, not only in Docker. Compose brings up Postgres, Kafka, Redis, and the two apps for the November milestone. If it does not run here, Kubernetes will not repair the image.

Do it in this order

1. Dockerfile: build the jar, copy it, expose the port, start the process.
2. Do not bake secrets into the image.
3. docker compose for Postgres and the app is the local rehearsal.

Worked example

HEALTHCHECK or a /health route the orchestrator can call.

The usual miss

A container that works only because it uses your laptop's files.

They will ask

What is in the image, and what is not?

Before you close the laptop

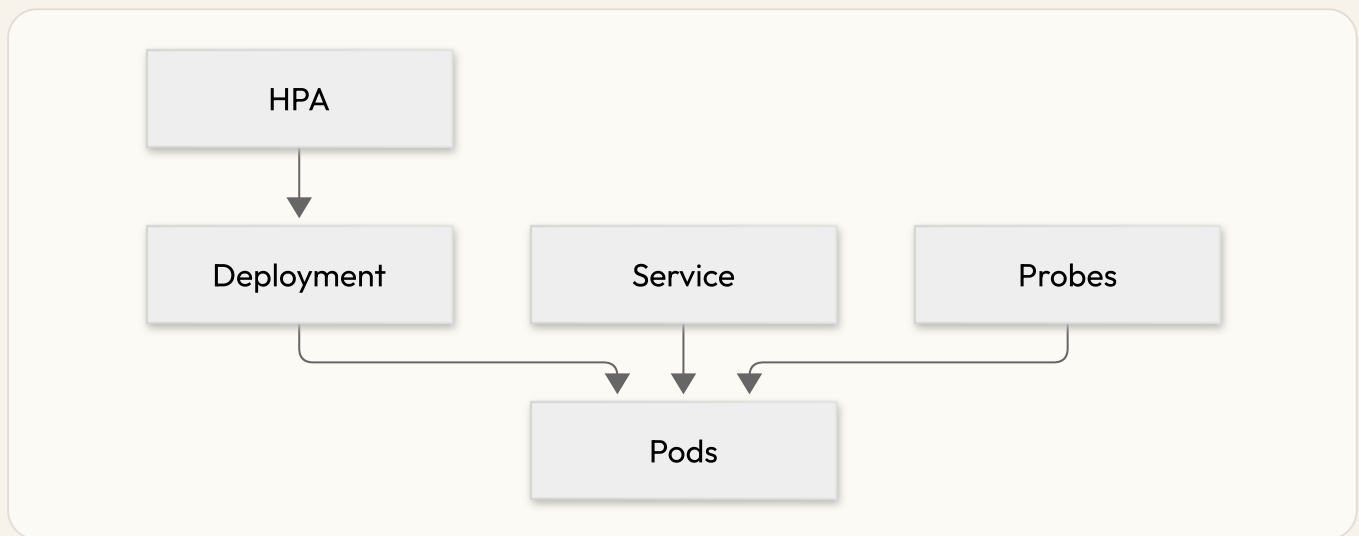
Run the Spring service in Docker before you read another Kubernetes page.

Deployment, Service, probes, HPA

CKAD is a hands-on exam and it is relevant later. It is not how you learn. These objects are how you learn.

1. Deployment keeps N pods
2. Service is the stable address
3. Readiness is traffic
4. Liveness is restart

Figure



What you debug

A Deployment keeps replicas. A Service selects pods by label and maps a port. A probe failure restarts the container or removes it from the Service. Readiness false means the pod is up and should not receive traffic. Liveness false means restart it. HPA adds pods from CPU or from a custom metric such as queue lag. CrashLoopBackOff means you read the previous log, not that you delete the pod in a loop.

Do it in this order

1. Deployment declares the desired count and the image.
2. Service selects pods by label. Ingress or a load balancer is the door from outside.
3. Readiness false means leave me alone. Liveness false means restart me. Mixing them up causes outages.
4. HPA scales on CPU or a custom metric. It does not fix a memory leak.

Worked example

CKAD order: Docker, these objects, ConfigMap and Secret, RBAC, volumes, Helm, then the exam

The usual miss

They will ask

Using liveness to check a downstream dependency, so a database blip restarts every pod.

Why did this pod restart, and why did it stop receiving traffic?

Before you close the laptop

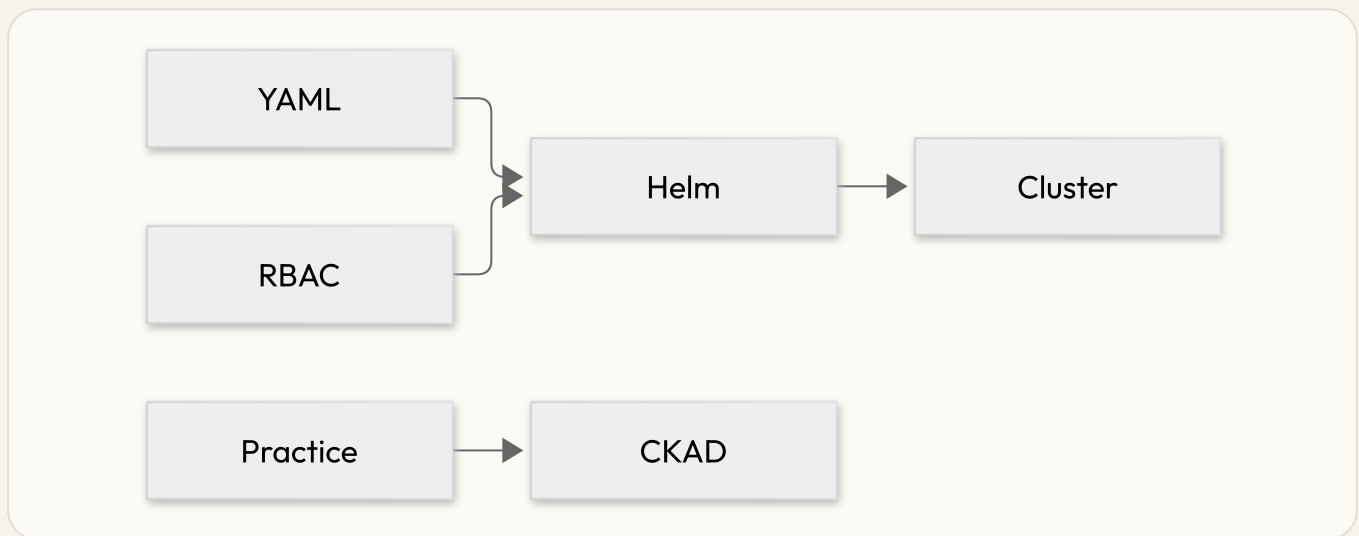
Write a Deployment and a Service for the FastAPI chunk service. Apply it locally if you have a cluster, or kind.

Config, RBAC, Helm, then CKAD

You already touch OpenShift at work. The public skill is the same objects with names you can explain, then Helm to stop copying YAML, then CKAD as proof.

1. ConfigMap for settings
2. Secret for credentials
3. RBAC is least privilege
4. CKAD after you can debug

Figure



Then the exam

ConfigMap is config. Secret is a credential, still not a place for a committed password in git. RBAC limits what the agent service account can do. A volume holds a scratch disk if you need one. Helm templates the manifests so the laptop and the cluster differ by values. CKAD is a timed performance exam over these objects. Book it after you can fix a crashing pod without notes.

Do it in this order

1. A ConfigMap changes the log level without a new image.
2. A Secret is not encryption by itself. You still restrict who can get secrets.
3. A Role that can only get and list pods in one namespace is the agent's read identity.
4. Book CKAD when you can fix a CrashLoopBackOff without notes. Not this month.

Worked example

```
kubectl logs, describe, and get events. That is the debug loop.
```

The usual miss

cluster-admin on the agent service account.

They will ask

The pod is CrashLoopBackOff. What do you type first?

Before you close the laptop

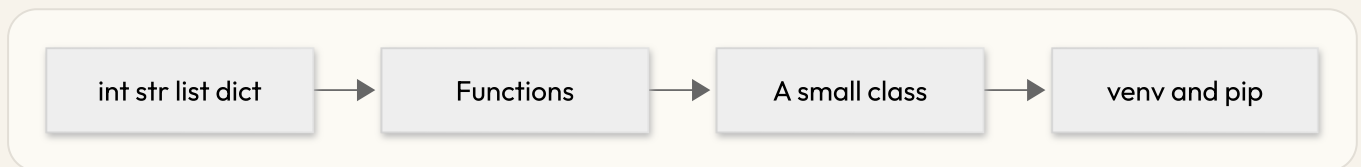
Break a probe on purpose and fix it with describe.

Python syntax and collections

Phase 1, inside the 30-minute block. You need to read and write the agent without translating every line from Java in your head.

1. Indentation is syntax
2. list, dict, set
3. A function can return several values
4. venv keeps the agent separate

Figure



The differences that bite a Java developer

There is no compile step that saves you. None is not a null you can call methods on. A default argument that is a list is shared across calls. Use None and create the list inside the function. Integer division is not the Java division you remember: $7 / 2$ is 3.5, $7 // 2$ is 3. Imports are files. `if __name__ == '__main__':` is how a script stays importable.

What phase 1 includes

Syntax, collections, functions, a class, venv, pip, typing, and requests or httpx calling your Java health endpoint. pytest comes as soon as the function exists. That is the whole month. FastAPI waits until the Java API exists to call.

Do it in this order

1. Lists are ordered. Dicts keep insertion order. Sets are for membership. A tuple is a fixed record.
2. `def ask(bucket: str) -> str` is the shape. Type hints are for you and the tools. They are not enforced at runtime unless you add that.
3. `python -m venv .venv` and `pip install -r requirements.txt`. The system Python stays alone.
4. Classes exist. You do not need a class for every function. A dataclass carries a tool result.

Worked example

```
def allow(client: str, now: float) -> bool:
    bucket = buckets.get(client)
    return bucket is not None and bucket.tokens > 0
```

The usual miss

They will ask

Installing packages into the global interpreter and then wondering why the laptop and CI differ.

What is the difference between a list and a tuple here?

Before you close the laptop

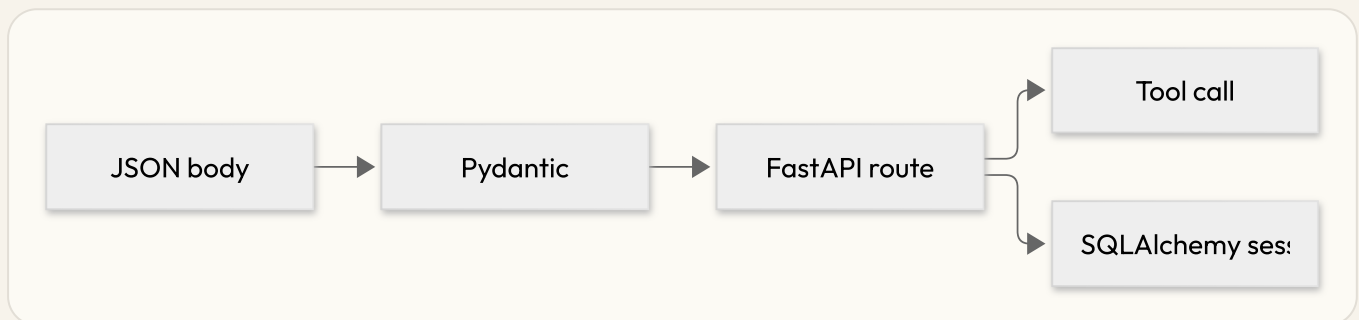
Create the agent folder, a venv, and a function that returns a fake diagnosis.

FastAPI, Pydantic, SQLAlchemy

The agent service is a small Python web app. This is phase 3, after you can call HTTP from a script.

1. Pydantic validates the body
2. The route stays thin
3. A session opens and closes
4. pytest hits the route

Figure



The shape of the service

FastAPI gives you HTTP and documentation. Pydantic gives you types at the boundary. SQLAlchemy is only there if the agent reads Postgres itself. It can also call the Java API and stay free of a second schema. Prefer calling Java for writes. Python may read for retrieval. Two writers to the same tables will produce a bug you will have to explain in the interview.

Async and tests

`httpx.AsyncClient` is worth `async`. A CPU-bound loop is not. `pytest` with `TestClient` calls the route in-process. The fake model returns a fixed answer so the test checks that evidence ids are passed through. That test is part of the eval story.

Do it in this order

1. A Pydantic model is the request and the response. Validation errors return 422.
2. The route calls a function. The function calls tools. The route does not embed a prompt the size of a file.
3. SQLAlchemy session is a unit of work. Close it. Do not keep a global session.
4. Async is for the HTTP client that is `async`. Do not mark every function `async` out of habit.

Worked example

```

@app.post("/diagnose")
def diagnose(body: Question) -> Answer:
    evidence = tools.metrics(body.bucket_id)
    return Answer(text=draft(evidence), evidence=evidence)
  
```

The usual miss

A route that opens a model client, a database, and a prompt, all inline.

They will ask

Where is the prompt, and how do you test the route without paying for a model?

Before you close the laptop

Add the diagnose route with a fake model and one pytest.

Python through FastAPI

You do not need to become a Python developer. You need to read and write the agent service: types, tests, HTTP, and async where the HTTP library is async.

1. Functions and dicts
2. Type hints
3. One pytest
4. One POST endpoint

Figure



The sequence

Syntax, collections, functions, classes, venv, pip, typing, requests. Then pytest, httpx, asyncio where the client is async, FastAPI, Pydantic, and SQLAlchemy only if Python reads the database. Docker last in this list so the image wraps a service that already runs. The detailed lessons are Syntax and collections, and FastAPI, Pydantic, SQLAlchemy.

Do it in this order

1. venv and pip. Pin requirements. Do not install into the system Python.
2. httpx or requests for the Java API. Pydantic for the body.
3. SQLAlchemy only if this service owns tables. Otherwise call the Java service.
4. asyncio matters when you wait on the model and the vector store together. Not before.

Worked example

```
def retrieve(question: str) -> list[str]:
    return index.search(question, k=3)
```

The usual miss

A 40-hour syntax course before a single endpoint exists.

They will ask

Show a typed function and a test.

Before you close the laptop

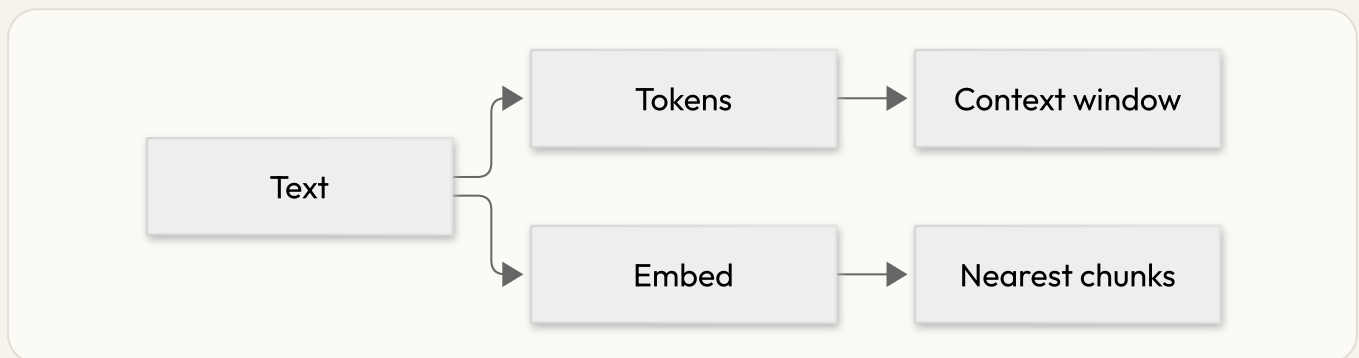
This week's Python is one file: read three markdown runbooks and return the overlapping lines.

Tokens, embeddings, retrieval

The engineering under the API. You should be able to say why a long log does not fit, and why similar vectors are not the same as a keyword match.

1. Tokens cost money and space
2. Temperature is randomness
3. Embeddings are coordinates
4. Retrieval is nearest chunks

Figure



The words you must own

Tokens are the model's units, not words. The context window is the budget for the prompt, the retrieved text, and the answer. Temperature and sampling change variety, not truth. Embeddings place texts in a vector space. Similarity is not a keyword match and not a citation. Structured output is a schema you validate. Tool calling is the model asking your code to run a function you defined. Streaming is how tokens arrive. A long log is chunked because it does not fit.

Do it in this order

1. A context window is finite. You retrieve a few chunks instead of pasting the cluster.
2. Temperature low for a root-cause draft. Higher only if you want variety, which you usually do not.
3. Chunk on headings and size with overlap. Write down why you picked the size.

Worked example

Structured output: ask for JSON with cause, evidence, action. Validate it. If it fails, ret

The usual miss

Turning the temperature up to make a diagnosis more creative.

They will ask

Why not send the whole log to the model?

Before you close the laptop

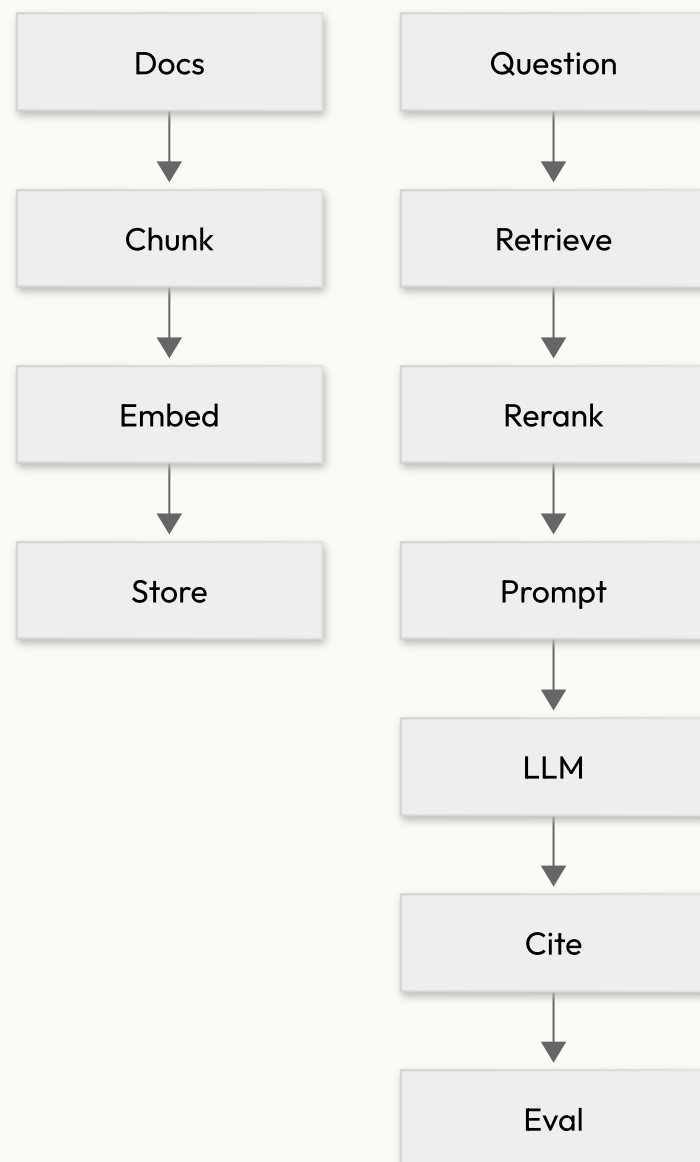
Count tokens on one synthetic log. Write the number in the README.

Production RAG

The pipeline is the product. A chat box without ingestion, citations, and an eval set is the project you were told not to build.

1. Ingest offline
2. Retrieve
3. Rerank if you need it
4. Cite
5. Score 20 questions

Figure



The pipeline

Ingest synthetic incidents. Chunk on paragraph or a fixed size with overlap, and write down why. Embed the chunks. Store them in a vector index with the incident id. Retrieve more than you need, rerank, then put the winners in the prompt with ids. The model answers with those ids as citations. The eval set is questions with

expected ids. A wrong citation fails the test. This is the December milestone: ten questions, a score in the README.

Do it in this order

1. Ingestion: clean text, chunk, embed, store with the source id.
2. Query: embed the question, fetch top K, optionally rerank, build the prompt, call the model.
3. The answer lists chunk ids. The eval set checks those facts.
4. Streaming is a nicer UI. It does not fix a bad index.

Worked example

If the index has no chunk about quota, the answer is I do not have that record.

The usual miss

Chat with PDF as the whole portfolio.

They will ask

How do you measure whether retrieval got worse?

Before you close the laptop

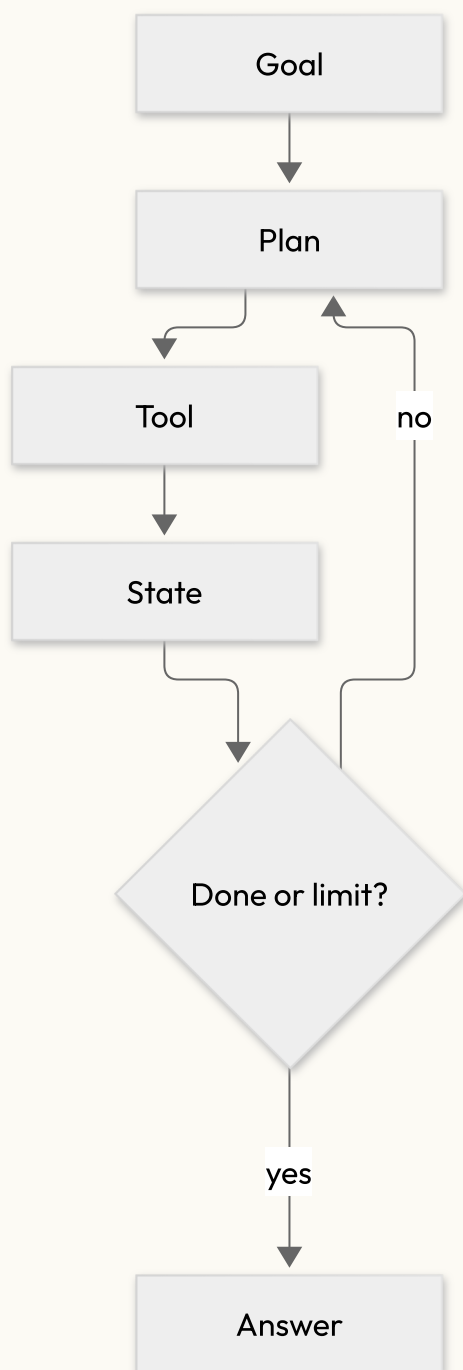
Create the 20-question file before you pick a vector database.

Agents, MCP, evals, and safety

An agent is a loop with tools, state, and a stop condition. MCP is a way to expose tools. Neither removes the need for permissions, traces, and a test set.

1. Allow-list the tools
2. Cap the steps
3. Trace each call
4. Eval the task not the vibe

Figure



A safe agent

An agent is a loop: state, a model, tools, a stop. Tools are allow-listed. A tool that changes the cluster requires a human approval, a timeout, a retry policy, and an idempotency key. MCP is a protocol for exposing those tools. It does not grant judgment. Traces show each tool call. Evals include a case where the tool should not be called. Prompt injection and data leakage are in the threat list: the incident text is untrusted, and the tool cannot read secrets it does not need.

Do it in this order

1. State is the messages plus tool results. Memory across sessions is a database, not a hope.
2. Timeouts, retries on reads, no retry on an unsafe write.
3. Prompt injection: untrusted logs are data, not instructions. The system prompt says so, and the write tool is not available to that call.
4. MCP lets a client discover tools. Your policy still decides which tools exist for this user.

Worked example

Max 8 tool calls. Then the agent must answer or ask a human.

The usual miss

An agent with a shell tool on a cluster.

They will ask

How do you make an agent safe enough for production?

Before you close the laptop

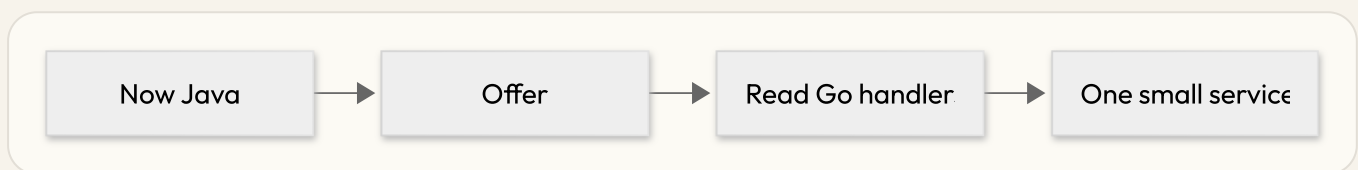
List read tools and write tools for the storage platform. Only the second list needs approval.

Go after the offer, at reading depth

Apple storage and Cisco distributed roles mention Go beside Java. That is a reason to read Go, not a reason to pause Java. After the switch, one small service is enough.

1. Do not context-switch this winter
2. After April, read net/http
3. Goroutines are the concurrency primitive
4. Errors are values

Figure



What reading Go means

After the offer, read a small service: package main, an HTTP handler, error returns instead of exceptions, goroutines, and a context cancellation. You can write a health endpoint. You do not start a Go curriculum in October. Java remains the language you solve the medium in.

```
func health(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
    _, _ = w.Write([]byte("ok"))
}
```

Do it in this order

1. A handler, a struct, and an error return are the first page.
2. You do not start with a framework tour.
3. Storage-infrastructure posts want Linux, Kubernetes, and a language. Your storage stories plus Java already qualify you to talk. Go is the gap you close on the job.

Worked example

```
func (s *Server) GetBucket(w http.ResponseWriter, r *http.Request)
```

The usual miss

Rewriting the flagship in Go in November.

They will ask

What will you learn in Go, and when?

Before you close the laptop

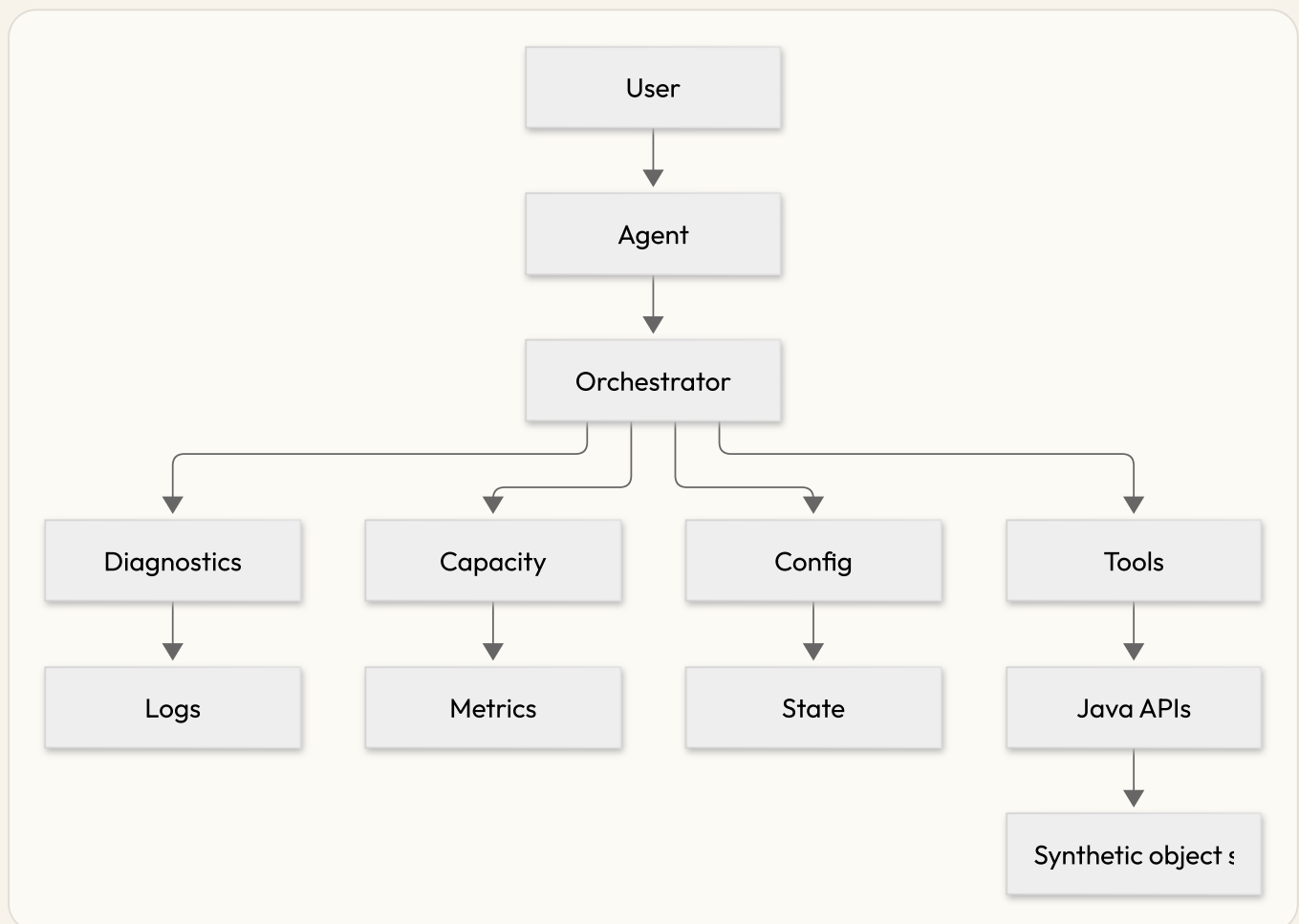
Add a single line to the April plan: read one Go HTTP file a week. Not before.

AI-native object storage operations

One project replaces the certificate pile. It is an operations console for a fake object store: Java services own metadata and events, a Python agent reads them, and a human approves anything that changes the cluster.

1. Synthetic data only
2. Java owns the system of record
3. Python owns the agent loop
4. Writes wait for a human

Figure



What the README must show

A stranger can run the synthetic cluster with Compose. The Java service owns buckets, events, and the outbox. The Python service answers a latency question with evidence ids. A mutation is proposed, approved, executed, and checked. The eval score is a number. No Dell data, no Dell code, no customer names.

Do it in this order

1. Do not copy Dell code, schemas, hostnames, or customer data. Invent buckets, latency series, and runbooks.
2. Services: metadata API in Spring Boot, event outbox, a worker, Postgres, Redis for the rate limit and session.

3. The README has a diagram, how to run it, and an eval score.

Worked example

Question the demo must answer: why is bucket latency high, using only the synthetic series.

The usual miss

A chat box with no tools and no score.

They will ask

What can the agent do without a person, and what can it not?

Before you close the laptop

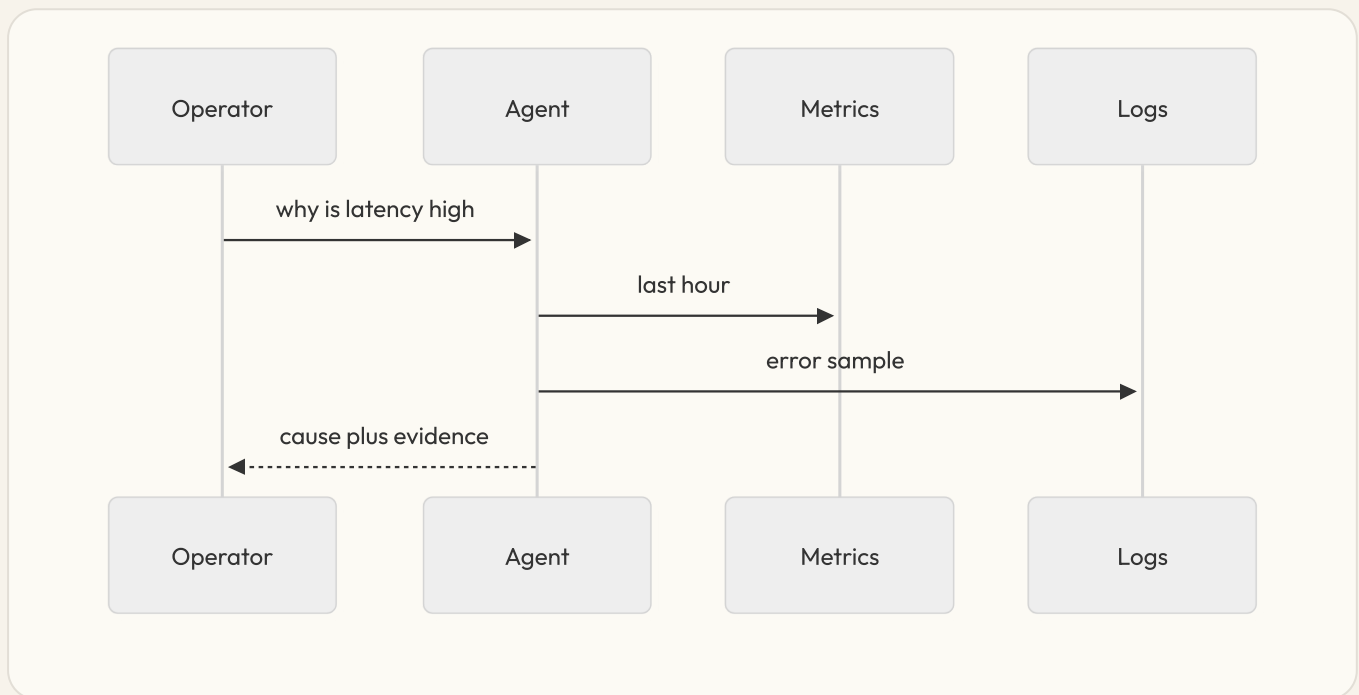
Create the repo and the architecture page. No model call yet.

Diagnostics agent

The first agent answers a latency question by reading metrics, logs, and traces, then naming a likely cause with the evidence attached.

1. Read tools only
2. Correlate time
3. Cite the series
4. Refuse if evidence is thin

Figure



The procedure

Question: why is bucket latency high? Collect the latency metric, the slow request ids, the traces, and the recent error logs. Correlate on bucket id and time. Name a likely cause and the evidence. Recommend an action. Do not take the action. If the evidence is thin, say so. The eval asks for the evidence ids, not for a poetic root cause.

Do it in this order

1. Tools: getLatency, getErrorRate, getRecentLogs. All read-only.
2. The answer template: observation, likely cause, what would disprove it.
3. Eval: ten scripted incidents with an expected cause label.

Worked example

Cause: retry storm. Evidence: error rate up, then latency up, same minute.

The usual miss

A confident cause with no series id.

They will ask

What would make you discard this hypothesis?

Before you close the laptop

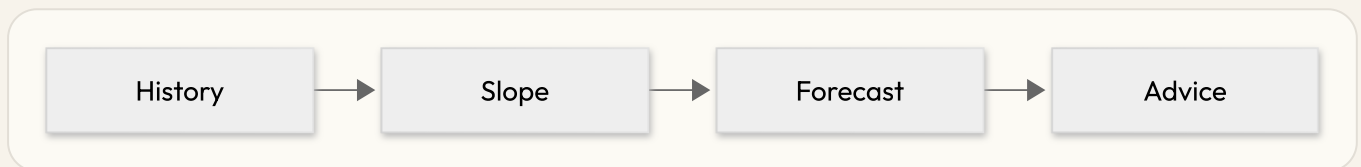
Seed three fake incidents and write the expected labels.

Capacity agent

The second agent forecasts whether a cluster crosses a threshold. It is a query and a sentence, not a research model.

1. Use the stored series
2. State the assumption
3. Recommend, do not resize
4. Show the date it crosses

Figure



The forecast you can defend

Read used bytes over time. Fit a simple trend. State the assumption: the last seven days continue. Answer whether the series crosses 80 percent inside the horizon. Recommend a scale step. A fake history in the repo is enough. A learned model is not required. The interview line is the assumption, not the library.

Do it in this order

1. A linear or seasonal baseline is enough. Say the assumption.
2. The tool returns the series. The model writes the sentence. The number comes from your code, not from the model's imagination.
3. If the series is shorter than two weeks, the agent says the forecast is weak.

Worked example

At the current slope, 80 percent full on day 40. Assumption: no new tenant.

The usual miss

Letting the model invent the percentage.

They will ask

Where is the number computed?

Before you close the laptop

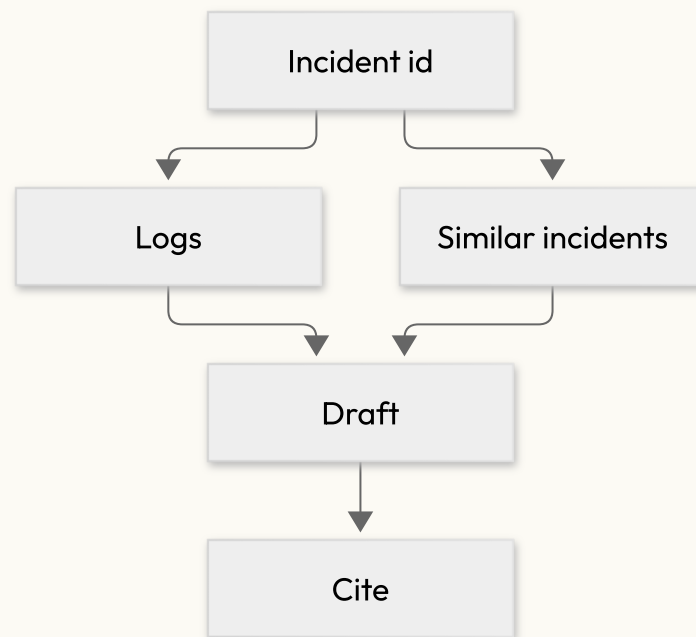
Implement the forecast in plain code. The model only explains it.

Incident RCA with retrieval

The third agent takes an incident id, pulls logs, and retrieves similar synthetic incidents and postmortems. That is RAG with a job to do.

1. Fetch by id
2. Retrieve similar text
3. Draft with citations
4. No fix is applied

Figure



Retrieval with a job

Input is an incident id. Load its logs. Retrieve similar synthetic incidents and postmortems. Answer with a suspected cause, a fix, and the ids you used. A missing citation fails the eval. Similar PRs, if you include them, are public or synthetic patches, not internal diffs.

Do it in this order

1. The corpus is synthetic postmortems you write. Twenty is enough.
2. Similar means embedding or keyword overlap. Record which you used.
3. The output is a draft for a human. It is not a merged pull request.

Worked example

Similar to incident 12: disk saturation. Your logs show the same queue time.

The usual miss

They will ask

Training a model on Dell tickets.

What is in the corpus, and what is forbidden to be in it?

Before you close the laptop

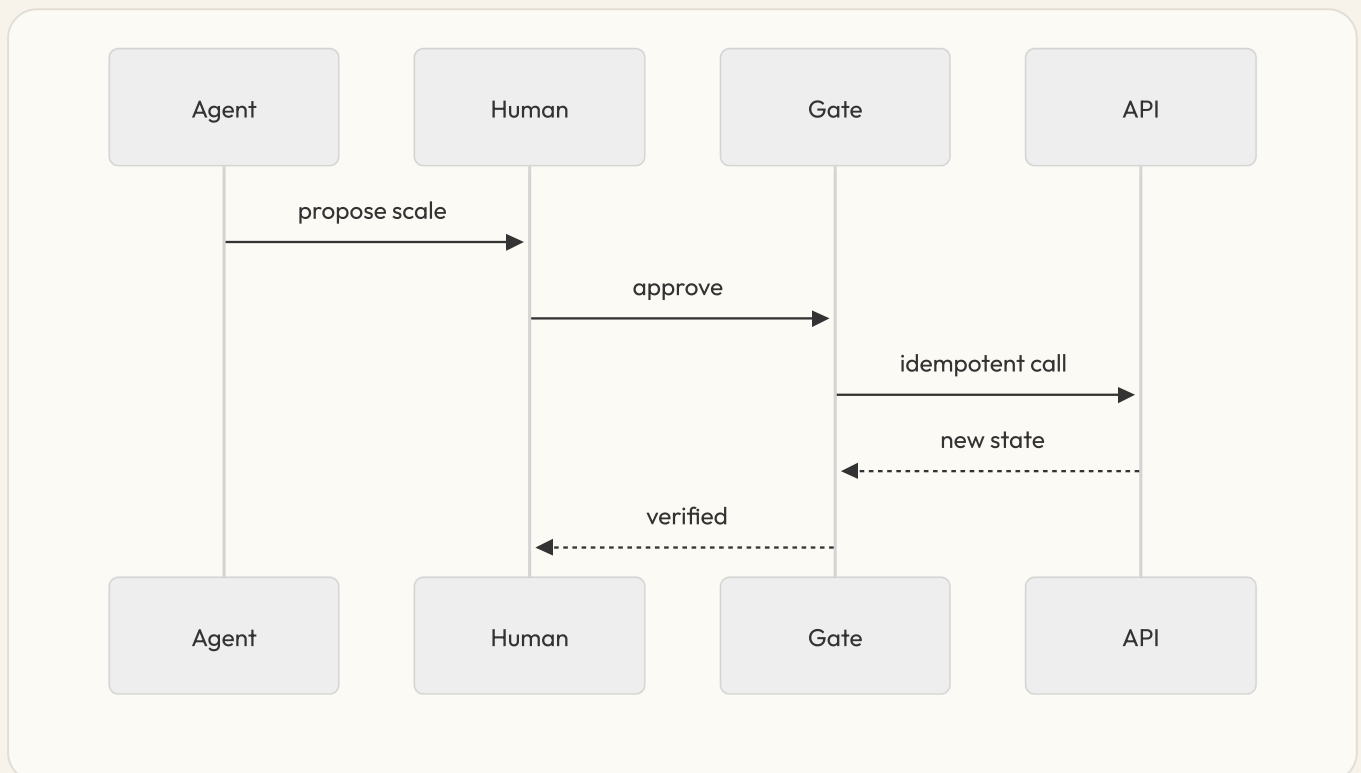
Write five fake postmortems. They are the index.

Safe executor

The fourth piece is the gate. The agent proposes. A person approves. A tool runs. A check verifies. The trace is the audit.

1. Proposal is data
2. Approval is a record
3. Tool is allow-listed
4. Verify after

Figure



The gate

The model proposes a tool call. The API stores the proposal as pending. A person approves. The executor runs the allow-listed tool with a timeout and an idempotency key. A verifier reads the metric or the config and records whether the change happened. A second approval does not double-apply. Denial is a recorded state. This is the January milestone.

Do it in this order

1. The proposal is a JSON document: action, target, reason, evidence ids.
2. Approve stores the user id. The Java API checks that row before it mutates.
3. Idempotency key so a double click does not scale twice.
4. Verification reads the new state and attaches it to the trace.

Worked example

POST /actions with Idempotency-Key. 200 on replay returns the same result.

The usual miss

The model calling kubectl with a string it composed.

They will ask

Who is allowed to approve, and where is that enforced?

Before you close the laptop

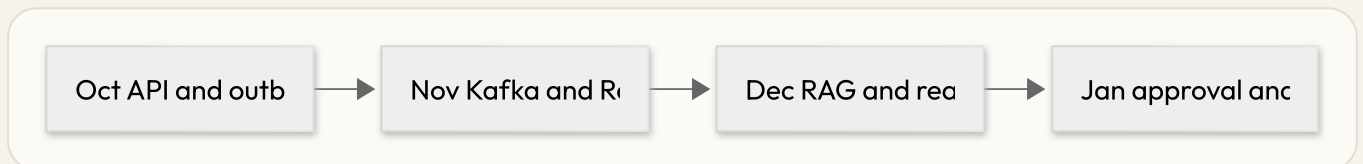
Implement the approval table in Java before the agent can ask for it.

Milestones through January

The project grows with the phases. It is not a December surprise.

1. October is the Java spine
2. November adds the bus and cache
3. December adds retrieval
4. January adds the gate and the score

Figure



Do it in this order

1. By 25 Oct: Spring service, Postgres, one idempotent POST, README run steps.
2. By 25 Nov: outbox, a consumer, Redis rate limit, Docker compose.
3. By 20 Dec: FastAPI retrieval over the fake postmortems, ten eval questions.
4. By 15 Jan: approval gate, twenty eval questions, a number in the README, a diagram you can redraw.

Worked example

Deploy to Kubernetes when compose is boring. AWS labels on the diagram can wait until the o

The usual miss

A Kubernetes cluster in October with no API.

They will ask

What is true in the repo on 25 November?

Before you close the laptop

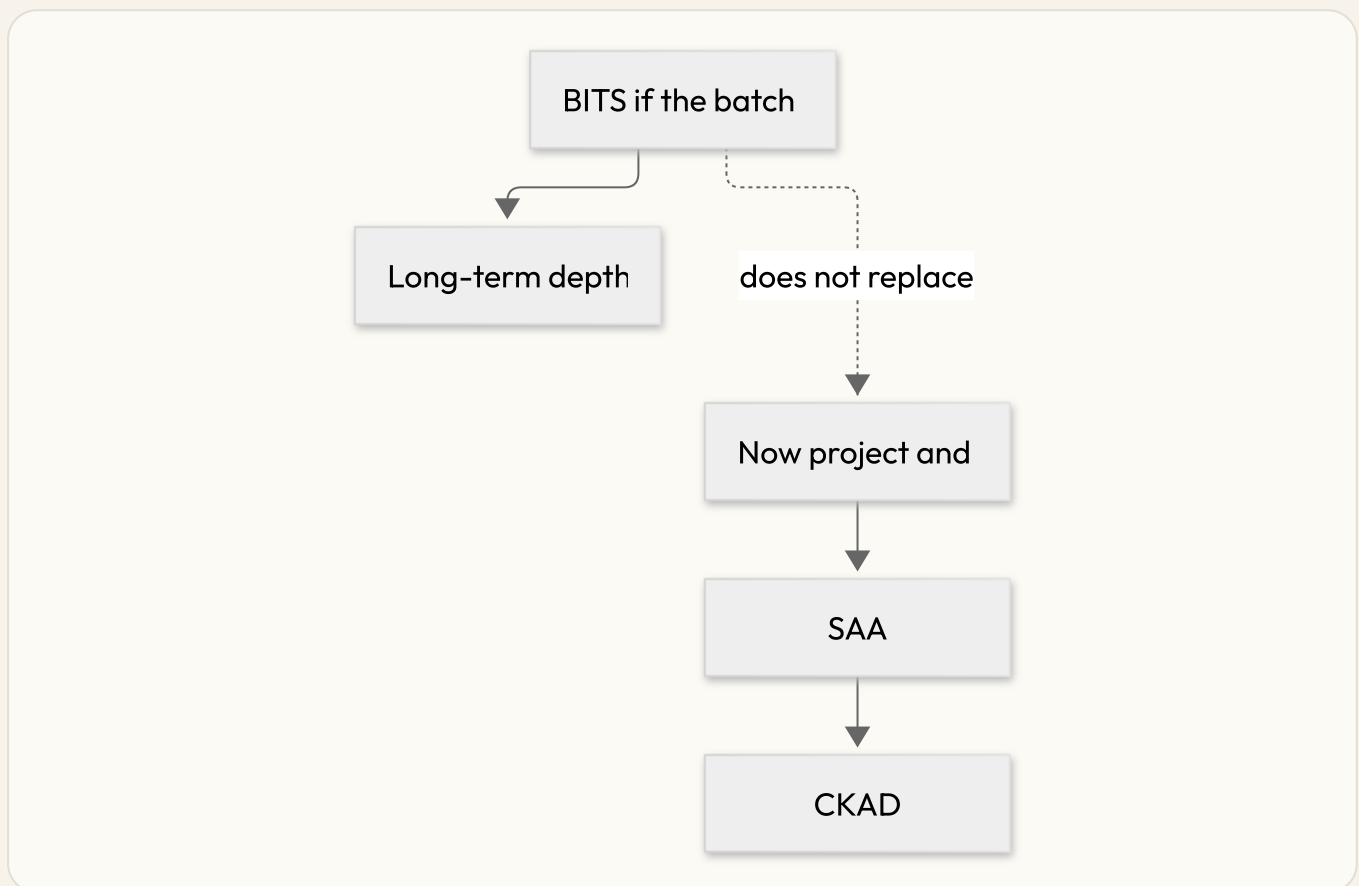
Put the four dates in the README now, unchecked.

SAA, CKAD, and BITS on the side

Two credentials earn a place: AWS Solutions Architect Associate after you can draw the VPC, and CKAD after you can debug a pod. BITS M.Tech AI and ML is a degree for the next two years, including marriage and a job, not the thing that gets you the January interview.

1. SAA after a real diagram
2. CKAD after CrashLoop practice
3. AI-901 optional
4. BITS is parallel

Figure



Do it in this order

1. Skip Cloud Practitioner, AI-900, LangChain badges, and Python completion certificates.
2. AI-901 replaced the English AI-900 exam in 2026. It is still optional.
3. AWS Generative AI Developer Professional is the later GenAI credential, after Bedrock or an equivalent pipeline is something you operated.
4. BITS WILP M.Tech AI and ML lists agentic systems, LLMs, cloud-native AI, and a dissertation. Public pages have quoted about ₹3.34 lakh across four semesters and have shown conflicting 2026 deadlines. It does not place you. If you join the October batch, confirm the date and the fee on the BITS site before you pay. It runs beside the job. It does not eat the 2.5 hours.

Worked example

If the degree load collides with a mock week, the mock wins.

The usual miss

Enrolling and then skipping DSA because the degree feels like progress.

They will ask

What does BITS not do for the April switch?

Before you close the laptop

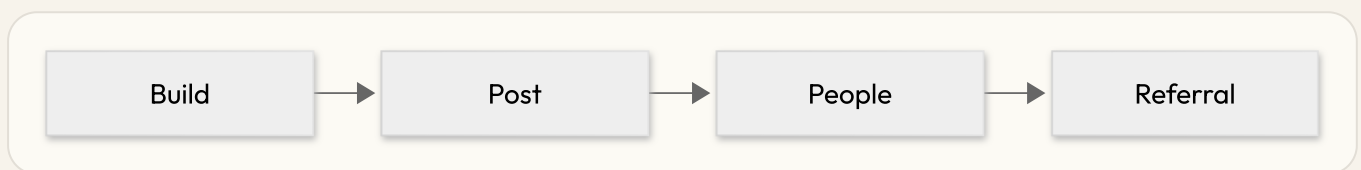
Email or check the BITS page for the live deadline. Do it once, then decide.

One post, a short list of people

Networking is a weekly quota, not a persona. One technical post and a handful of real conversations beat a daily influencer schedule.

1. One post a week
2. Five engineers
3. Two recruiters
4. Two people at target companies

Figure



Do it in this order

1. Post from the work you did that week: an idempotent consumer, a probe you broke, a RAG eval number.
2. No Dell internals. The synthetic platform is the public story.
3. Ask for a referral only after a specific conversation, not as the first message.

Worked example

This week I made a consumer safe to run twice. Here is the key, and here is the test.

The usual miss

Posting motivation quotes.

They will ask

Who did you talk to, and about which system?

Before you close the laptop

Draft the first post from the outbox sketch. Four sentences.